

GRADO EN INGENIERÍA ELECTRÓNICA
INDUSTRIAL



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

CARACTERIZACIÓN Y VALIDACIÓN DEL MODELO
DINÁMICO DE UN ROBOT MÓVIL

AUTOR:
IGNACIO PÉREZ SERRA

TUTOR:
VICENT GIRBÉS JUAN



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

CARACTERIZACIÓN Y VALIDACIÓN DEL MODELO DINÁMICO DE UN ROBOT MÓVIL

AUTOR: IGNACIO PÉREZ SERRA

TUTOR: VICENT GIRBÉS JUAN

Declaración de autoría:

Yo, Ignacio Pérez Serra, declaro la autoría del Trabajo Fin de Grado titulado “Caracterización y validación del modelo dinámico de un robot móvil” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 27 de junio de 2024

Fdo: Ignacio Pérez Serra

Resumen:

En este Trabajo de Fin de Grado (TFG), se ha llevado a cabo un proyecto centrado en la creación de un gemelo digital del robot Eddie. El proceso comenzó con la obtención de todas las medidas necesarias del robot físico. Utilizando estas medidas como base, se ha diseñado una réplica detallada, asegurando la captura de todos los elementos para mantener la precisión del modelo digital.

Posteriormente, se ha desarrollado un archivo URDF que describe tanto la estructura física como la cinemática del robot. Este archivo es esencial para la simulación del robot en Gazebo, donde se puede evaluar y ajustar el comportamiento del modelo digital para que se aproxime lo más posible al comportamiento real del robot Eddie. Además, se han integrado sensores como un Lidar y un Kinect al modelo digital, mejorando su capacidad de percepción y navegación en el entorno simulado.

Finalmente, para complementar el proyecto, se ha diseñado una carcasa-soporte utilizando tecnología de impresión 3D de Bambulab. Esta carcasa-soporte ha sido específicamente diseñada para alojar una pantalla en el robot Eddie real, facilitando la visualización de datos y la interacción con usuarios en aplicaciones prácticas.

Abstract:

In this Final Degree Project (TFG), a project focused on the creation of a digital twin of the Eddie robot has been carried out. The process began with obtaining all the necessary measurements of the physical robot. Using these measurements as a base, a detailed replica has been designed, ensuring the capture of all elements to maintain the accuracy of the digital model.

Subsequently, a URDF file has been developed that describes both the physical structure and the kinematics of the robot. This file is essential for the simulation of the robot in Gazebo, where the behavior of the digital model can be evaluated and adjusted to approximate the real behavior of the Eddie robot as closely as possible. Additionally, sensors such as a Lidar and a Kinect have been integrated into the digital model, enhancing its perception and navigation capabilities in the simulated environment.

Finally, to complement the project, a housing-support has been designed using BambuLab's 3D printing technology. This housing-support has been specifically designed to house a screen on the real Eddie robot, facilitating data visualization and user interaction in practical applications.

Resum:

En aquest Treball de Fi de Grau (TFG), s'ha dut a terme un projecte centrat en la creació d'un bessó digital del robot Eddie. El procés va començar amb l'obtenció de totes les mesures necessàries del robot físic. Utilitzant aquestes mesures com a base, s'ha dissenyat una rèplica detallada, assegurant la captura de tots els elements per a mantenir la precisió del model digital.

Posteriorment, s'ha desenvolupat un fitxer URDF que descriu tant l'estructura física com la cinemàtica del robot. Aquest fitxer és essencial per a la simulació del robot en Gazebo, on es pot avaluar i ajustar el comportament del model digital perquè s'aproxime el més possible al comportament real del robot Eddie. A més, s'han integrat sensors com un Lidar i un Kinect al model digital, millorant la seua capacitat de percepció i navegació en l'entorn simulat.

Finalment, per a complementar el projecte, s'ha dissenyat una carcassa-suport utilitzant tecnologia d'impressió 3D de Bambulab. Aquesta carcassa-suport ha estat específicament dissenyada per a allotjar una pantalla en el robot Eddie real, facilitant la visualització de dades i la interacció amb usuaris en aplicacions pràctiques.

Agradecimientos:

En primer lugar, me gustaría expresar mi más sincero agradecimiento a todos aquellos que me han apoyado a lo largo de estos años y han contribuido a hacer realidad este proyecto. Agradezco especialmente a Vicent por su dedicación, y orientación cuando me encontraba perdido y no sabía cómo avanzar con este proyecto. Por último, pero no menos importante, quiero agradecer a mi familia por su incondicional apoyo durante este prolongado y arduo proceso de aprendizaje continuo. Quiero destacar el papel fundamental de mi madre, quien con su paciencia y apoyo inquebrantable ha sido un pilar fundamental en este camino.

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Motivación	1
1.3. Objetivos	2
1.4. Organización de la memoria	3
2. Estado del Arte	5
2.1. Historia de la robótica	5
2.2. Clasificación de robots según su arquitectura	8
2.3. Robótica móvil terrestre	9
2.4. Tecnologías relacionadas con la robótica	14
2.4.1. ROS (Robot Operating System)	15
2.4.2. Herramientas de simulación	16
2.4.3. Herramientas de modelado 3D	18
2.4.4. Impresión 3D	19
3. Estudio de Alternativas	21
3.1. Software de modelado 3D	21
3.2. Plataforma robótica	24
3.3. Generación del URDF	28
3.4. Herramientas de simulación	28
4. Caracterización y Diseño	31
4.1. Medición y pesaje de las piezas del robot	31
4.2. Modelado en 3D	34
4.3. Generación del URDF	34
4.4. Modelado de la carcasa de la pantalla	38
5. Implementación y Validación del Sistema	41
5.1. Software	41

5.2. Creación del entorno de trabajo del Eddie Robot	41
5.3. Implementación del Gemelo Digital	42
5.4. Puesta a punto del modelo digital	43
5.5. Pruebas de rendimiento	43
5.6. Pruebas de sensorizado	47
5.7. Calculo del torque sin carga	50
6. Conclusiones	53
6.1. Conclusiones	53
6.2. Trabajo futuro	54
A. Diseño 2D	55
A.1. Kinect plate assembly	56
A.2. Base	58
A.3. Caster wheel kit	61
A.4. Top base	64
A.5. Wheel + motor	66
A.6. Carcasa de la pantalla	68
B. Diseño 3D	71
B.1. Kinect plate assembly	71
B.2. Base	72
B.3. Caster wheel kit	74
B.4. Top base	75
B.5. Wheel + motor	76
C. Código	79
C.1. URDF	79
C.2. Kinect URDF	91
C.3. Gazebo	92
C.4. Gazebo launch	97
C.5. RVIZ Launch	98
C.6. Script Bash para Guardar Datos de Pose 3D y Odom en archivos .txt . . .	98
D. Impresora Bambu Lab X1 Carbon	101
Bibliografía	107

Índice de figuras

2.1. El primer robot humanoide George era capaz de caminar agitando los pies y era teledirigido [1]	6
2.2. El robot Shakey con Charles Rosen en 1983 [1]	6
2.3. Spot Dog, robot móvil desarrollado por Boston Dynamics [1]	7
2.4. La estatua de Gundam de 18 m de altura frente al Museo de Tokio, en Odaiba. [2]	8
2.5. Ejemplo de locomoción ackermann [3]	10
2.6. Ejemplo de movimiento de la rueda mecanum [4]	11
2.7. Ejemplo de movimiento de la rueda omni [4]	11
2.8. Ejemplo de locomoción diferencial en un robot de tres ruedas [5]	12
2.9. Ejemplo de configuración triciclo [6]	12
2.10. Ejemplo de configuración skid de cuatro ruedas [7]	13
4.1. Robot Eddie	31
4.2. Piezas del robot	33
4.3. Ensamblaje completo del Eddie Robot - vista frontal y trasera	34
4.4. Ejemplo posición de los joints	35
4.5. Ejemplo de gestión de propiedades	35
4.6. Ejemplo de configuración de propiedades de los joints	36
4.7. Ejemplo de herramienta de propiedades físicas	36
4.8. Ejemplo de configuración de propiedades de los links	37
4.9. Ejemplo del visor de URDF	37
4.10. Pantalla del robot	38
4.11. Ensamblaje de la carcasa de la pantalla	38
4.12. Carcasa impresa con la pantalla	39
4.13. Ejemplo de versión anterior de la carcasa	40
5.1. Ejemplo de Eddie Robot en Gazebo	44
5.2. Ejemplo del comportamiento del Eddie Robot	45
5.3. Gráfica - Velocidad lineal en X	46
5.4. Gráfica - Velocidad angular en Z	46

5.5. Gráfica - Velocidad angular y lineal	47
5.6. Ejemplo funcionamiento del LIDAR en RVIZ	48
5.7. Ejemplo funcionamiento del LIDAR en Gazebo	49
5.8. Ejemplo funcionamiento de la Kinect en el robot	49
A.1. Kinect deck	56
A.2. Standoff kinect	57
A.3. First floor base	58
A.4. Battery	59
A.5. Battery shelf	60
A.6. Caster	61
A.7. Caster wheel holder	62
A.8. wheel (caster)	63
A.9. Standoff base	64
A.10.Top base	65
A.11.Wheel	66
A.12.Motor	67
A.13.Carcasa de la pantalla en 2D	68
A.14.Tapa de la carcasa en 2D	69
B.1. Kinect deck	71
B.2. Standoff kinect	72
B.3. First floor base	72
B.4. Battery	73
B.5. Battery shelf	73
B.6. Caster	74
B.7. Caster wheel holder	74
B.8. wheel (caster)	75
B.9. Standoff base	75
B.10.Top base	76
B.11.Wheel - Vista delantera	76
B.12.Wheel - Vista trasera	77
B.13.Motor	77
D.1. Bambu Lab X1 Carbon	101
D.2. Pantalla de inicio	102
D.3. Orientación de la pantalla	102
D.4. Ajuste de parámetros	103

D.5. Botón para exportar a G-Code	103
D.6. Objeto exportado a G-Code	104
D.7. Pantalla de inicio de la impresora	104
D.8. Selección del archivo	105
D.9. Pantalla previa a la impresión	105
D.10.Preparación de la impresión	106
D.11.Bambu Lab X1	106
D.12.Ejemplo de pieza impresa con sus soportes	107

Índice de tablas

3.1. Comparación de Solidworks, Fusion 360, FreeCAD y Blender	24
3.2. Comparación de CARMEN, YARP y ROS	26
3.3. Comparación de características entre ROS 1 y ROS 2	27
3.4. Comparación de características entre Gazebo, CoppeliaSim, Webots, Ro- boDK y Unity	30
4.1. Kinect plate assembly	32
4.2. Base	32
4.3. Caster Wheel kit	32
4.4. Top base	32
4.5. Wheel + motor	33
4.6. Componentes diversos	33

Capítulo 1

Introducción

*Cualquier tecnología suficientemente avanzada es
indistinguible de la magia*
Arthur C. Clarke

1.1. Introducción

En el ámbito de la robótica, debido a los avances causados por la última revolución industrial se ha tenido un avance sustancial tanto a nivel hardware como software, el cual ha repercutido en una mayor eficiencia y versatilidad en la implementación de sistemas autónomos y robots móviles. Sin embargo, uno de los desafíos persistentes en este campo es la necesidad de garantizar la seguridad y la fiabilidad de estos sistemas en entornos dinámicos y cambiantes. En este contexto, surge la idea de desarrollar un gemelo digital de un robot móvil, una réplica virtual que simula con precisión el comportamiento y el entorno del robot físico en tiempo real. Este enfoque ofrece numerosas ventajas, incluida la capacidad de realizar pruebas exhaustivas y de optimización antes de la implementación física, así como la posibilidad de realizar diagnósticos y mantenimiento predictivo. En este trabajo de fin de grado (TFG), se abordará el desarrollo y la implementación de un gemelo digital para un robot móvil, explorando sus aplicaciones potenciales y evaluando su rendimiento en diversos escenarios de operación.

1.2. Motivación

El estímulo que me ha llevado a realizar este estudio está marcado por una motivación personal, la cual me llevó a la realización del mismo.

El Trabajo de Fin de Grado (TFG) en el tema de **“Caracterización y Validación del Modelo Dinámico de un Robot Móvil”** surge a raíz de mi profundo interés en la robótica y la ingeniería de control. Este interés se encuentra estrechamente vinculado a mi fascinación por los mechas (robot o vehículo mecanizado que debe ser tripulado por un piloto humano) y mi pasión por la informática. A su vez, la razón fundamental por la cual opté por abordar este tema específicamente en el contexto de robots móviles, en lugar de bípedos, hexápodos, vehículos aéreos o marinos, se debe principalmente a consideraciones profesionales, vinculadas al tipo de empresa en la que aspiro desarrollar mi carrera laboral. En este proyecto, me propongo explorar y comprender a fondo el comportamiento dinámico de un robot móvil, centrándome específicamente en su caracterización y validación.

La creciente importancia de la robótica en diversas aplicaciones, como la industria, la

medicina y la exploración espacial, resalta la necesidad de contar con modelos dinámicos precisos para garantizar un rendimiento óptimo y una operación segura de los robots. A través de este trabajo, aspiro a contribuir al avance de la ingeniería robótica al mejorar la comprensión de los aspectos dinámicos fundamentales de los robots móviles.

Además, la oportunidad de aplicar y validar los modelos en un entorno práctico me permitirá adquirir habilidades y conocimientos valiosos en el diseño y control de sistemas robóticos. Este proyecto no solo representa un requisito académico, sino también una oportunidad para fusionar mi pasión por la tecnología con el aprendizaje práctico y la resolución de problemas del mundo real. Espero fortalecer mis habilidades técnicas en el sector de la robótica.

1.3. Objetivos

El trabajo tiene como objetivo principal desarrollar un gemelo digital de un robot móvil, estableciendo un enfoque integral que abarque desde la creación del modelo dinámico hasta la validación experimental. Los objetivos secundarios con los que se conseguirá el objetivo principal son:

1. **Creación del Gemelo Digital:** El objetivo principal consiste en construir un gemelo digital preciso del robot móvil, utilizando herramientas de modelado virtual. Este gemelo digital servirá como plataforma para la realización de simulaciones y pruebas en un entorno controlado.
2. **Caracterización y Validación del Modelo Dinámico:** Se busca caracterizar y validar el modelo dinámico del robot móvil, comprendiendo y modelando las interacciones entre sus componentes, como ruedas, sensores y actuadores, así como su interacción con el entorno. Este paso es crucial para asegurar la fiabilidad del gemelo digital.
3. **Programación con ROS:** Utilizar el Robot Operating System (ROS) como plataforma para programar y controlar el robot móvil en el entorno virtual. Se pretende aprovechar las capacidades de ROS para facilitar el desarrollo de software y la comunicación eficiente entre los distintos componentes del sistema robótico.
4. **Simulación en Gazebo:** Realizar simulaciones detalladas del robot móvil en el entorno de simulación Gazebo. Esta herramienta de simulación 3D permitirá emular el comportamiento del robot en diversas condiciones y escenarios, proporcionando insights valiosos para la optimización y mejora del rendimiento.
5. **Validación Experimental:** Comparar los resultados obtenidos de las simulaciones con pruebas experimentales llevadas a cabo en un robot móvil real. Este proceso de validación experimental tiene como objetivo verificar la precisión del modelo dinámico desarrollado, asegurando su utilidad y aplicabilidad en situaciones del mundo real.

Al alcanzar estos objetivos, se busca contribuir al avance en el entendimiento y desarrollo de sistemas robóticos, permitiendo una transición efectiva entre la simulación y la implementación práctica de robots móviles.

1.4. Organización de la memoria

Para situar al lector en la estructura de este proyecto, a continuación se resume el contenido de cada capítulo:

- **Capítulo 1.** Introduce el proyecto y lo contextualiza.
- **Capítulo 2.** Recopila la información necesaria para el proyecto sobre robótica, incluyendo la historia de la robótica, clasificación de robots y sus especificaciones, gemelos digitales, ROS (Robot Operating System), herramientas de simulación y sensores, apoyándose en la literatura especializada.
- **Capítulo 3.** Evaluación de opciones, descripción de las características y los aspectos de las diversas alternativas consideradas a lo largo de las diferentes fases del proyecto.
- **Capítulo 4.** Diseña, analiza e implementa los pasos llevados a cabo para la realización del gemelo digital de un robot móvil.
- **Capítulo 5.** Realiza las pruebas de verificación cuantitativas y cualitativas para la caracterización y realización del modelo dinámico, con discusión de los resultados.
- **Capítulo 6.** Resume las conclusiones obtenidas al final del proyecto y el trabajo futuro que motivan.

Capítulo 2

Estado del Arte

LAS TRES LEYES ROBÓTICAS

- 1. Un robot no debe dañar a un ser humano o, por su inacción, dejar que un ser humano sufra daño.*
- 2. Un robot debe obedecer las órdenes que le son dadas por un ser humano, excepto cuando estas órdenes están en oposición con la primera Ley.*
- 3. Un robot debe proteger su propia existencia, hasta donde esta protección no esté en conflicto con la primera o segunda Leyes.*

Manual de Robótica 56^a edición, año 2058

Asimov, Isaac. 1950. I, robot

2.1. Historia de la robótica

El campo de la robótica, tal como se lo conoce en la actualidad, emerge como una disciplina relativamente reciente. Los avances ocurridos a lo largo del siglo XX y XXI han sido sumamente significativos, los cuales han tenido repercusión en la tercera y cuarta revolución industrial.

Un robot inteligente y robusto es una maquina que opera en un entorno relativamente desconocido e impredecible. Cualquier robot será inteligente o autónomo si tiene el potencial de navegar libremente sin obstrucciones y la inteligencia para sortear cualquier obstáculo situado dentro del movimiento adyacente.

La definición más ampliamente conocida de robot, fue dada por el “**Robot Institute of America**”, la cual establece: “Un robot es un manipulador multifuncional y reprogramable diseñado para mover materiales, herramientas, dispositivos especializados o piezas, mediante movimientos integrados con programación variable para la realización de diferentes tareas”.

La historia de la robótica tiene sus raíces en la antigüedad, con los primeros autómatas mecánicos diseñados por inventores como Herón de Alejandría. Sin embargo, la robótica moderna comenzó a tomar forma en el siglo XX, impulsada por avances en electrónica, computación y tecnologías de control. La palabra “robot” fue popularizada por el escritor checo Karel Čapek en su obra de teatro “R.U.R.” (Rossum’s Universal Robots) en 1920, derivada de la palabra checa “robota” que significa trabajo forzado.

En las décadas siguientes, la robótica se desarrolló principalmente en el ámbito industrial. El primer robot humanoide fue construido en 1950 por Tony Sale en el Reino Unido. Nombrado “George”, tenía una altura de 6 pies y era capaz de caminar y hablar. Ver en la Figura 2.1.

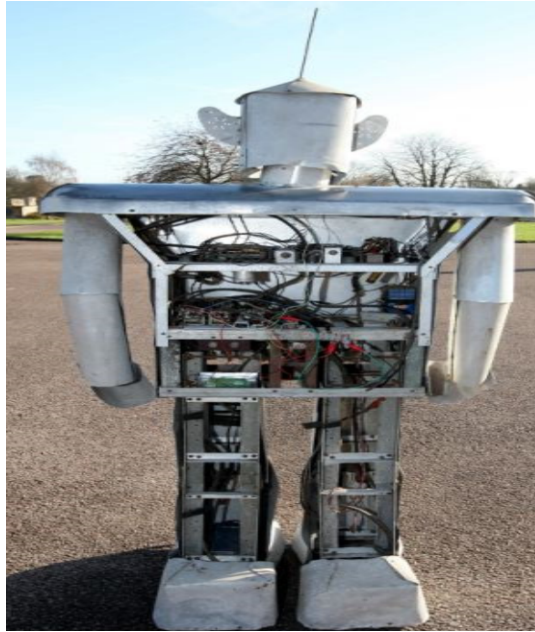


Figura 2.1: El primer robot humanoide George era capaz de caminar agitando los pies y era teledirigido [1]

Un aspecto crucial del desarrollo robótico es la evolución de los robots móviles. Estos robots están diseñados para moverse de manera autónoma en diversos entornos, utilizando sensores y algoritmos avanzados para navegar y evitar obstáculos.

La historia de los robots móviles no puede mencionarse sin hablar de Shakey, desarrollado a finales de la década de 1960. Fue el primer robot móvil capaz de comprender y reaccionar a su entorno. Este robot fue creado por un grupo de ingenieros del Stanford Research Institute (SRI) bajo la supervisión de Charles Rosen, con financiamiento de la Agencia de Proyectos de Investigación Avanzados de Defensa (DARPA). La Figura 2.2 muestra una imagen de Shakey con Charles Rosen en 1983.



Figura 2.2: El robot Shakey con Charles Rosen en 1983 [1]

En la última década, la robótica móvil ha experimentado avances significativos y una expansión notable en diversas áreas de aplicación. Desde aproximadamente 2010 hasta la actualidad, hemos presenciado un aumento en el desarrollo y la implementación de robots móviles en una variedad de sectores industriales y comerciales. Estos avances han sido impulsados por mejoras en la miniaturización de componentes, la computación en la nube, el aprendizaje automático y la inteligencia artificial, que han permitido a los robots moverse de manera más autónoma y realizar tareas más complejas.

En el ámbito de la logística y el transporte, los robots móviles han sido adoptados en almacenes y centros de distribución para la gestión automatizada de inventarios y la preparación de pedidos. Empresas líderes en comercio electrónico han implementado flotas de robots para optimizar el flujo de trabajo y reducir los tiempos de entrega. Además, en entornos de manufactura, los robots móviles colaborativos (cobots) han comenzado a trabajar junto a humanos en líneas de producción, mejorando la eficiencia y seguridad laboral.

En la investigación espacial, la última década ha visto el despliegue de nuevos rovers como el Perseverance de la NASA en Marte, equipados con sistemas avanzados de navegación y recolección de muestras. Estos robots no solo exploran terrenos difíciles de alcanzar para los humanos, sino que también realizan análisis científicos fundamentales para comprender el cosmos.

Boston Dynamics, una empresa pionera en el campo de la robótica avanzada, ha capturado la atención mundial con su robot cuadrúpedo Spot. Lanzado en la última década se ha destacado por su capacidad para moverse con agilidad en diferentes tipos de terrenos. Equipado con sensores y cámaras, Spot puede realizar tareas variadas. La continua evolución de Spot y las innovaciones de Boston Dynamics han demostrado el potencial de la robótica móvil para transformar industrias y mejorar la eficiencia en diferentes sectores [1]. Ver Spot dog en Figura 2.3.



Figura 2.3: Spot Dog, robot móvil desarrollado por Boston Dynamics [1]

Por ultimo, pero no menos importante, se hace referencia a los mechas, o robots gigantes tripulados, los cuales son un genero popularizado por la ciencia ficción, especialmente en la cultura japonesa. Estos enormes robots, pilotados por humanos, son protagonistas en muchas series de anime y manga, como “Mobile Suit Gundam”, “Mazinger Z”, “Macross” y “Neon Genesis Evangelion”. Aunque los mechas aún no son una realidad en el mundo

actual, su concepto ha inspirado el desarrollo de exoesqueletos y robots de asistencia que ayudan a las personas con discapacidades a moverse y realizar tareas físicas [2]. Ver un ejemplo en la Figura 2.4.



Figura 2.4: La estatua de Gundam de 18 m de altura frente al Museo de Tokio, en Odaiba. [2]

2.2. Clasificación de robots según su arquitectura

La clasificación de robots según su arquitectura se refiere a la configuración general de los componentes que constituyen el robot.

Esta clasificación puede dividirse en distintas categorías:

1. **Poliarticulados:** Tienen múltiples articulaciones, lo que les confiere una notable flexibilidad, permitiéndoles ejecutar tareas complejas que requieren una alta precisión. Estas articulaciones rotativas (juntas) posibilitan movimientos semejantes a los de un brazo humano, otorgándoles una amplia gama de movimientos y la capacidad de operar en diversos entornos. Están diseñados para moverse dentro de un espacio específico, a menos que se coloquen sobre un robot móvil.

2. **Móviles:** es un dispositivo robótico diseñado para desplazarse en su entorno y realizar diversas tareas.
- **Terrestres:** Robots móviles que se desplazan sobre la superficie terrestre utilizando ruedas, patas u otros sistemas de locomoción.
 - **Aéreos:** Drones y vehículos aéreos no tripulados que se desplazan en el aire mediante hélices o alas.
 - **Acuáticos:** Robots submarinos o de superficie que se mueven en el agua utilizando propulsores o aletas.

Estos robots pueden ser autónomos, semi autónomos o controlados remotamente.

3. **Androides:** Están diseñados con una estructura humanoide, con el objetivo de emular el comportamiento, y poder desempeñar las tareas de un ser humano. Desde mediados del siglo XX, Japón ha sido un líder en la investigación y desarrollo de robots, y muchos de sus esfuerzos se han centrado en crear robots con apariencia y comportamientos similares a los humanos. Este enfoque se debe en parte a la cultura japonesa, que tiene una larga tradición de historias sobre robots y personajes animados que han influido en la percepción y la aceptación de los robots en la sociedad japonesa.
4. **Zoomórficos:** Son robots diseñados para imitar o inspirarse en formas y comportamientos de animales. Estos robots a menudo imitan características físicas y movimientos de animales reales para cumplir una función específica. Por ejemplo, se puede imitar la locomoción de un reptil para moverse eficientemente en terrenos difíciles, o imitar el vuelo de un pájaro para realizar tareas de vigilancia aérea. La inspiración en la naturaleza y la biomecánica animal puede proporcionar soluciones innovadoras para desafíos en robótica, como la adaptación a entornos complejos y la mejora de la eficiencia en el movimiento.
5. **Híbridos:** Se refiere a aquellos autómatas que son de difícil clasificación, cuya estructura combina características y capacidades de diferentes tipos en una sola plataforma. Estos pueden integrar componentes y tecnologías de las categorías mencionadas anteriormente, para aprovechar las fortalezas de cada uno y adaptarse a una variedad de tareas y entornos. Estos robots son diseñados para ser versátiles y flexibles, pudiendo adaptarse a diferentes aplicaciones y desafíos.

2.3. Robótica móvil terrestre

Un robot móvil terrestre es un tipo de robot diseñado para desplazarse y operar sobre la superficie del suelo. Estos robots pueden ser autónomos o controlados a distancia y están equipados con diversos sistemas de locomoción, sensores y algoritmos de navegación que les permiten interactuar con su entorno y cumplir con diversas tareas en diferentes ubicaciones.

Características de los Robots Móviles Terrestres:

1. Tipos de tracción

- **Locomoción Ackerman:** La geometría de dirección, utilizada principalmente en vehículos con ruedas, es un sistema similar al de un automóvil diseñado para resolver el problema de las ruedas dentro y fuera de una curva. En este sistema, las ruedas delanteras giran en ángulos distintos para asegurar que todas las ruedas sigan trayectorias circulares concéntricas, lo que permite una mejor maniobrabilidad y estabilidad al tomar curvas. Las ruedas traseras, que son dos ruedas tractoras, se montan de forma paralela en el chasis principal del vehículo, proporcionando la tracción necesaria [3]. En la Figura 2.5 se detalla un ejemplo.

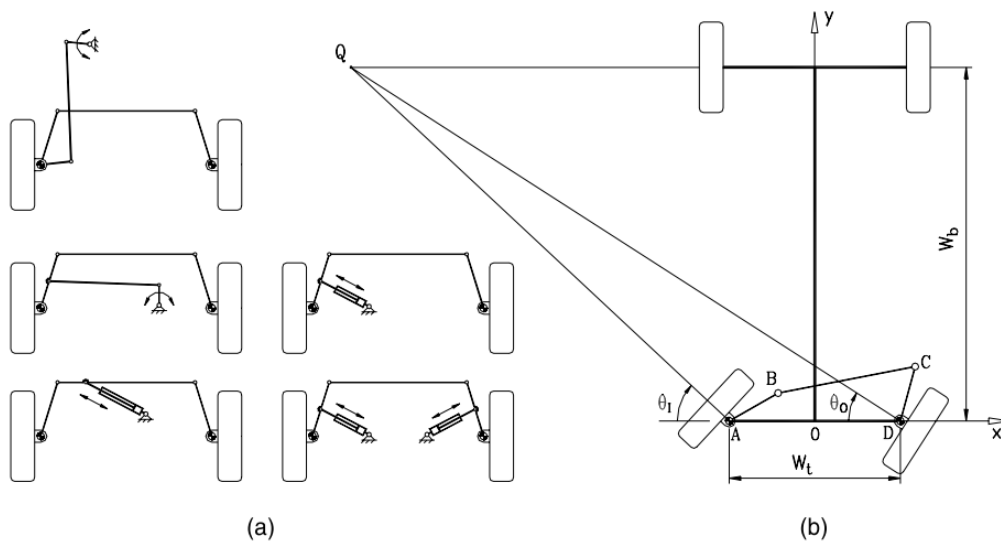


Figura 2.5: Ejemplo de locomoción ackermann [3]

- **Configuración omnidireccional:** Permite al robot moverse en cualquier dirección sin necesidad de cambiar la orientación. Esto se consigue utilizando ruedas especiales, como pueden ser las ruedas Mecanum o las ruedas omni, las dos pueden generar fuerzas en múltiples direcciones, ofreciendo una gran movilidad en espacios reducidos [4].
 - **Ruedas Mecanum:** Estas ruedas tienen una serie de rodillos montados en ángulo alrededor de su perímetro. Los rodillos están inclinados a 45 grados respecto al eje de la rueda. Cuando las ruedas giran en combinación, los rodillos generan fuerzas laterales, permitiendo que el robot se desplace lateralmente, diagonalmente y en cualquier dirección deseada.
 - **Ruedas omni:** Estas ruedas están construidas con una serie de rodillos dispuestos a lo largo de su circunferencia en ángulo recto con el eje de la rueda. Los rodillos pueden girar libremente, permitiendo que la rueda se desplace no solo hacia adelante y hacia atrás, sino también lateralmente.

En las Figuras 2.6, 2.7 se detallan dos ejemplos.

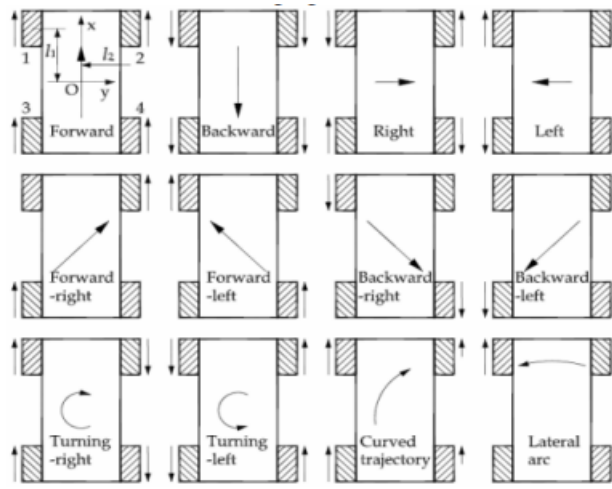


Figura 2.6: Ejemplo de movimiento de la rueda mecanum [4]

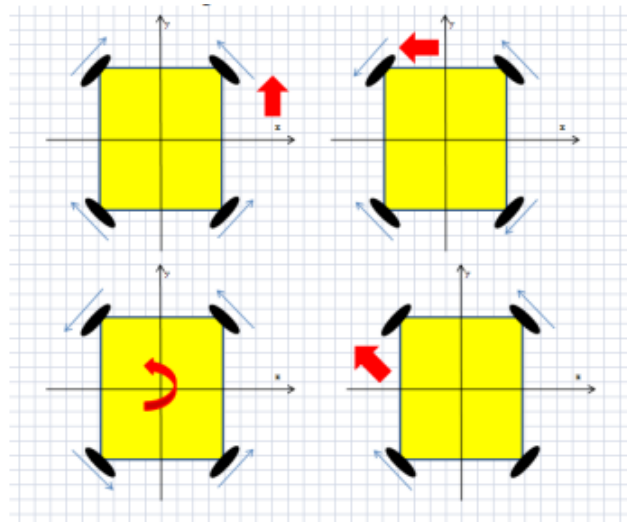


Figura 2.7: Ejemplo de movimiento de la rueda omni [4]

- Configuración diferencial:** Es un sistema de tracción en el que el movimiento se controla mediante dos ruedas motrices independientes, una a cada lado del robot. Este tipo de configuración permite al robot moverse hacia adelante y hacia atrás y girar sobre su propio eje al variar la velocidad y dirección de cada rueda de forma independiente. Por ejemplo, si ambas ruedas giran a la misma velocidad en la misma dirección, el robot avanza en línea recta; si giran a velocidades diferentes, el robot gira en la dirección de la rueda que gira más lentamente [5]. En la Figura 2.8 se muestra un ejemplo.

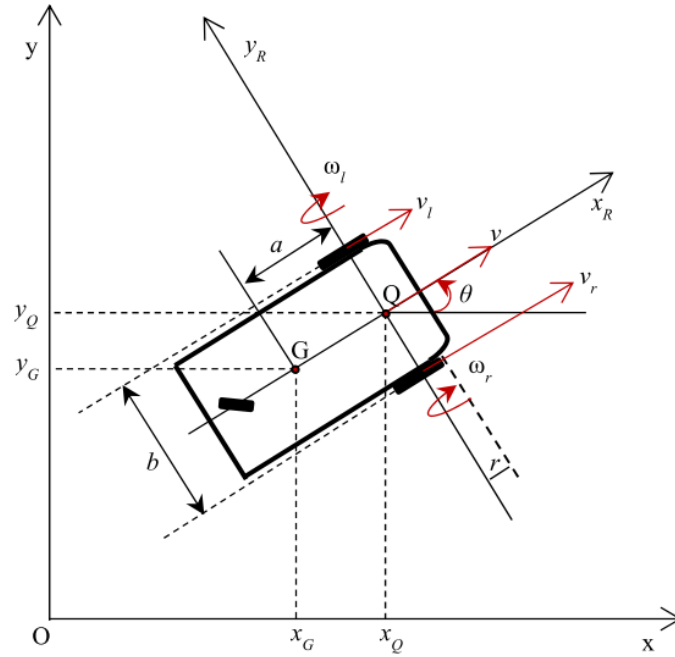


Figura 2.8: Ejemplo de locomoción diferencial en un robot de tres ruedas [5]

- Configuración triciclo:** Consiste en un robot que suele tener tres ruedas, dos ruedas traseras motrices y una rueda delantera que es la responsable de controlar la dirección del robot, mientras que las ruedas traseras proporcionan la tracción. Este sistema es similar al de una bicicleta o un triciclo, donde la dirección y el movimiento hacia adelante o hacia atrás son controlados de manera separada. Esta configuración es simple y proporciona una buena maniobrabilidad, especialmente en superficies planas [6]. En la Figura 2.9 se muestra un ejemplo.

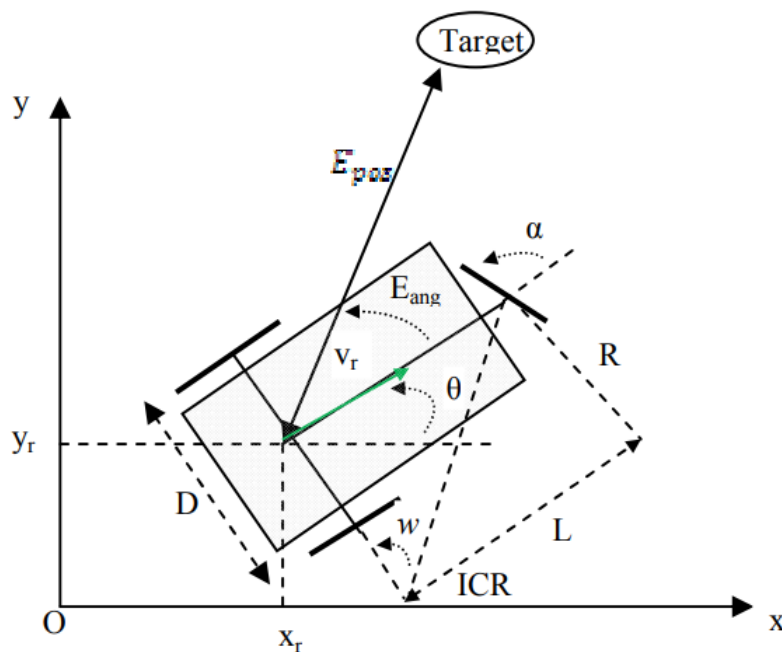


Figura 2.9: Ejemplo de configuración triciclo [6]

- **Configuración skid de cuatro ruedas:** Es un sistema en el que todas las ruedas del robot están fijas y no giran de manera independiente a la dirección. El movimiento y la dirección se controlan variando las velocidades de las ruedas en cada lado del robot, permitiendo giros mediante el deslizamiento de las ruedas. Este sistema permite al robot girar sobre su propio eje, es mecánicamente simple, y ideal para operar en terrenos difíciles y en espacios reducidos Sin embargo, puede generar un mayor desgaste de los neumáticos y ser menos eficiente debido a la fricción adicional durante los giros. Esta configuración también es conocida como tracción diferencial de cuatro ruedas [7]. En la Figura 2.10 se muestra un ejemplo.

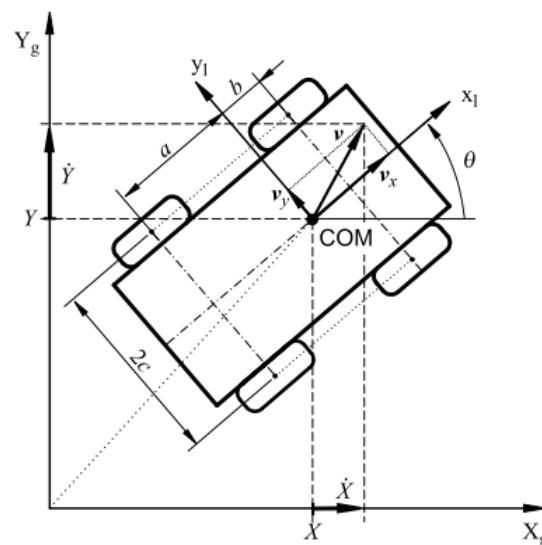


Figura 2.10: Ejemplo de configuración skid de cuatro ruedas [7]

2. Navegación y Control

- **Mapeo y Localización Simultánea SLAM (simultaneous localization and mapping):** Esta técnica permite crear y actualizar mapas de su entorno, utilizando los sensores de a bordo (LIDAR, cámaras, IMU), los cuales usa de manera simultánea para localizarse a sí mismo.
- **Navegación Basada en LIDAR (Light Detection and Ranging):** Estos sensores emiten una luz láser que es utilizada para escanear el entorno 3D, creando un mapa detallado del entorno a partir de los datos obtenidos, y así poder detectar obstáculos y planear rutas de manera precisa.
- **Navegación basada en GPS (Global Positioning System):** Se utilizan señales de satélites para determinar su posición geográfica exacta en cualquier momentos, permitiendo su seguimiento en rutas predefinidas y ajustes en tiempo real para alcanzar destinos con precisión.
- **Visión por Computadora:** Utiliza cámaras y algoritmos avanzados de procesamiento de imágenes por computadora para permitir que el robot perciba y comprenda su entorno. Mediante la captura de imágenes en tiempo real extrae información relevante sobre la posición y el movimiento, permitiendo que pueda planificar rutas y evitar colisiones de manera eficiente.

- **Control Reactivo:** Esta estrategia de control utiliza sensores de proximidad, infrarrojos o táctiles para detectar obstáculos y otros elementos en su camino, permitiéndole al robot responder de manera inmediata a los estímulos del entorno sin depender de un modelo complejo del mismo. Este enfoque es eficiente en entornos dinámicos y desconocidos, aunque puede ser menos efectivo en tareas que requieren planificación y decisión a largo plazo.

3. Aplicaciones en la industria y logística

- **AGV (Automated Guided Vehicles):** Se desplazan siguiendo rutas establecidas previamente en el suelo de una fábrica o almacén. Estos vehículos emplean varios sistemas de orientación, como cintas magnéticas, marcas en el suelo o sensores láser. Son especialmente eficaces para realizar tareas repetitivas y predecibles, como el transporte de materiales dentro de un entorno de producción. Sin embargo, su principal desventaja es la falta de flexibilidad, ya que cualquier cambio en las rutas de trabajo requiere una modificación física del entorno o de las guías utilizadas.
- **AMR (Autonomous Mobile Robots):** Son robots más avanzados, los cuales emplean sensores, cámaras y algoritmos de inteligencia artificial para desplazarse de manera autónoma en entornos dinámicos. Tienen la capacidad de tomar decisiones en tiempo real, evitar obstáculos y ajustarse a cambios en su entorno sin necesidad de reconfiguración manual. Esto los hace ideales para entornos laborales más complejos y variables, donde se necesita una mayor adaptabilidad y autonomía.

2.4. Tecnologías relacionadas con la robótica

Antes de la llegada del Robot Operating System (ROS) en 2007, el campo de la robótica se apoyaba en una serie de frameworks que buscaban estandarizar y facilitar el desarrollo de sistemas robóticos. Estos frameworks, como Player/Stage, CARMEN, ORCA, OROCOS y YARP, jugaron un papel fundamental en la evolución de la robótica, permitiendo la creación de sistemas reutilizables [8]. Cada uno de ellos ofrecía distintas herramientas y enfoques para la navegación, control y comunicación de robots, preparando el terreno para el desarrollo de una plataforma más unificada y robusta como ROS, que revolucionaría el campo de la robótica al proporcionar una extensa biblioteca de recursos y una comunidad activa.

1. Player/Stage

- **Player:** Desarrollado a principios de la década de 2000, Player es un servidor de dispositivos de red para robots móviles. Proporciona una interfaz estándar para controlar robots y sensores [9].
- **Stage:** Acompañando a Player, Stage es un simulador que emula los sensores y movimientos de robots móviles en un entorno 2D. Player y Stage permitieron la investigación y el desarrollo en robótica con una arquitectura cliente-servidor.

2. CARMEN (Carnegie Mellon Navigation Toolkit)

Desarrollado en la Universidad Carnegie Mellon, CARMEN es una colección de software modular para la

navegación y control de robots móviles. Fue diseñado para ser fácil de usar y modificar, facilitando la investigación en algoritmos de navegación [10].

3. **ORCA (Open Robot Control Software)** ORCA fue un proyecto de software libre para el control de robots, basado en componentes. Permitía a los investigadores crear sistemas robóticos distribuidos y modulares, con un enfoque en la reutilización y la interoperabilidad [11].
4. **YARP (Yet Another Robot Platform)** YARP es una plataforma para la comunicación entre módulos de software robótico. Desarrollada en el Instituto Italiano de Tecnología, YARP se centra en la flexibilidad y la eficiencia en la comunicación, siendo especialmente útil para robots humanoides [12].
5. **OROCOS (Open Robot Control Software)** Es un software de código abierto que comenzó en 2001, fue diseñado para el control en tiempo real de robots. Desarrollado por la comunidad de investigación y desarrollo robótico, proporciona una colección de bibliotecas y herramientas para el desarrollo de sistemas de control avanzados [13].

2.4.1. ROS (Robot Operating System)

2.4.1.1 Definición:

ROS, o Robot Operating System, se define como un conjunto de herramientas y librerías (software libre bajo términos de licencia BSD) que ayudan al desarrollo de aplicaciones robóticas.

Ofreciendo una arquitectura flexible y modular que permite a los desarrolladores crear sistemas robóticos complejos mediante la combinación de componentes individuales (nodos), un sistema de mensajería para la comunicación asíncrona entre nodos, herramientas de simulación para probar y depurar algoritmos y aplicaciones en entornos virtuales, y más soporte para una amplia gama de plataformas de hardware y sistemas operativos, facilitando el desarrollo, la implementación y la colaboración en proyectos de robótica.

2.4.1.2 Versiones de ROS

ROS ha experimentado varias versiones a lo largo de su evolución para adaptarse a las crecientes demandas y necesidades en el campo de la robótica. La versión original de ROS, conocida como ROS 1, estableció las bases del sistema y se convirtió en un estándar en la comunidad. Durante su desarrollo, ROS 1 ha recibido a lo largo de los años numerosas actualizaciones y mejoras, incluyendo nuevas características, correcciones de errores y soporte para una gran variedad de plataformas y hardware.

Sin embargo, para abordar ciertas limitaciones y desafíos encontrados en ROS 1 en el desarrollo industrial, se inició el desarrollo de una nueva versión, denominada ROS 2.

En resumen, estas versiones y distribuciones continuamente evolucionan para brindar a los desarrolladores las herramientas necesarias para crear sistemas robóticos avanzados.

2.4.1.3 Ventajas de ROS

1. **Amplia gama de herramientas y librerías:** Proporciona una gran cantidad de herramientas que facilitan desarrollar software de robots, lo que permite aprovechar funcionalidades preexistentes y acelerar el proceso de desarrollo del gemelo digital.
2. **Arquitectura modular y flexible:** La arquitectura de ROS permite construir el gemelo digital como un conjunto de nodos modulares interconectados, lo que facilita la creación, modificación y prueba de diferentes componentes de manera independiente, y permite una mayor reutilización de código.
3. **Simulación integrada:** ROS incluye herramientas específicas diseñadas para la simulación de robots y entornos robóticos. Como Gazebo, que se trata de una de las herramientas más conocidas para simulación de robots en ROS, que permite simular el comportamiento del robot en diferentes entornos y condiciones. Esto facilita el desarrollo y la validación del gemelo digital sin necesidad de hardware físico.
4. **Facilita la integración de sensores y actuadores:** ROS proporciona interfaces estandarizadas para una amplia variedad de sensores y actuadores, lo que permite simular su comportamiento de manera realista.
5. **Comunidad activa y recursos abundantes:** Cuenta con una gran comunidad de desarrolladores y usuarios que comparten conocimientos, experiencias y recursos, como paquetes de software, tutoriales y documentación.
6. **Escalabilidad y compatibilidad:** Es altamente escalable y compatible con una gran variedad de plataformas de hardware y sistemas operativos.

Utilizar ROS para el desarrollo de un gemelo digital de un robot proporciona una serie de ventajas, incluyendo una amplia gama de herramientas y librerías, una arquitectura modular y flexible, herramientas de simulación integradas, facilidad de integración de sensores y actuadores, una comunidad activa y recursos abundantes, escalabilidad y compatibilidad con diferentes plataformas.

2.4.2. Herramientas de simulación

Las distintas herramientas de simulación que se pueden utilizar para la creación de un gemelo digital se detallan a continuación destacando sus características y aplicaciones.

2.4.2.1 Gazebo

Este simulador de código abierto, se utiliza para simular robots en entornos tridimensionales, lo que la convierte en una herramienta útil para el desarrollo, prueba y validación de algoritmos de control, planificación de movimientos y percepción.

Este programa se ha vuelto muy popular debido a que está respaldado por la OSFR (Open Source Robotics Foundation). El simulador se emplea para simular una amplia gama de robots, desde robots móviles hasta brazos robóticos, drones e incluso sistemas multirobot. Proporciona una interfaz gráfica que permite a los usuarios crear entornos complejos y personalizados, así como también simular sensores y actuadores.

Cabe destacar que Gazebo está especialmente ligado a ROS debido a que le proporciona simulaciones físicas realistas para probar algoritmos de control en un entorno virtual.

Además, la disponibilidad de numerosos plugins y herramientas personalizables diseñadas específicamente para ROS facilita la creación de simulaciones complejas. La gran comunidad de desarrolladores, el soporte, y los flujos de trabajo estandarizados dentro del ecosistema ROS, junto con las actualizaciones y mantenimiento constantes, hacen que Gazebo sea una herramienta ideal para la simulación robótica en el ecosistema ROS [14].

2.4.2.2 CoppeliaSim

CoppeliaSim, anteriormente conocido como V-REP, es una plataforma de simulación robótica avanzada y versátil. Esta aplicación de código abierto permite la simulación de robots y entornos complejos, ofreciendo un entorno de desarrollo integrado para modelar, programar y probar sistemas robóticos.

Incluye una amplia gama de características, como motores físicos realistas, una interfaz de usuario intuitiva, soporte para múltiples lenguajes de programación (python, C++, C, etc) y compatibilidad con diversos tipos de robots y sensores. Es ampliamente utilizada en investigación, desarrollo y educación en el campo de la robótica [15].

2.4.2.3 Webots

Es una aplicación de escritorio de código abierto y multiplataforma utilizada para simular robots, a partir de un entorno moderno (GUI) para poder editar tanto las simulaciones como controladores del robot.

Una de sus características principales es que facilita el diseño de simulaciones de robots, gracias a una biblioteca que incluye robots, sensores, actuadores, objetos y materiales. Además, permite la importación de modelos CAD desde Blender o URDF.

En resumen, Webots es una excelente herramienta para crear prototipos de robots (como robots de dos ruedas, brazos industriales, robots modulares, drones, entre otros), así como para desarrollar, probar y validar algoritmos. Esto la convierte en una herramienta ideal para la enseñanza de la robótica [16].

2.4.2.4 RoboDk

RoboDK es un software de simulación y programación para robots industriales. Permite a los usuarios simular el movimiento de robots y sistemas de fabricación automatizados en entornos virtuales fuera del entorno de producción, para poder probarlos antes de implementarlos en el mundo real. Con RoboDK, los usuarios pueden crear, simular y generar programas de control para una amplia variedad de robots y aplicaciones de fabricación, lo que les permite optimizar el rendimiento y la eficiencia de sus procesos de automatización [17].

2.4.2.5 Unity

Unity es un motor de videojuegos que integra todas las herramientas necesarias para desarrollar videojuegos en diversas plataformas como PC, videoconsolas y dispositivos

móviles. Aunque su principal uso es el desarrollo de videojuegos, se ha destacado como una excelente herramienta para la simulación de robots debido a su versatilidad y robustez.

Empresas como Siemens y IA Cross Compass utilizan Unity para realizar pruebas con gemelos digitales, validar y entrenar algoritmos de IA para robots industriales antes de su implementación. Una de las principales ventajas de Unity es su capacidad para integrarse con ROS y ROS 2, lo que facilita el desarrollo de simulaciones robóticas avanzadas. Además, las actualizaciones constantes en sus motores de física permiten obtener resultados más realistas durante las simulaciones.

Por último, Unity ofrece acceso a diversos productos de inteligencia artificial (IA) y aprendizaje automático, proporcionando soluciones efectivas para una amplia gama de problemas en la simulación y desarrollo de robots [18].

2.4.3. Herramientas de modelado 3D

Se han seleccionado dos grupos de software como posibles opciones de herramientas de modelado 3D. El primer grupo es el software de mallas, con herramientas destacadas como Blender, Cinema 4D y Maya. El segundo es el software CAD/CAM, que incluye opciones como FreeCAD, Solidworks y Fusion 360.

El software de mallas se utiliza principalmente para crear modelos 3D mediante la manipulación de vértices, aristas y caras, siendo ideal para animaciones, efectos visuales y diseño de juegos. En contraste, el software CAD está diseñado para la creación de modelos con especificaciones exactas y funcionalidades para la fabricación y análisis estructural, siendo ampliamente utilizado en la ingeniería, arquitectura y manufactura.

Tras analizar todas las opciones existentes en el mercado a día de hoy, las más utilizadas son:

2.4.3.1 Solidworks

Solidworks de la empresa francesa Dassault es un software de diseño asistido por computadora (CAD), ampliamente utilizado en la industria, el cual ofrece herramientas avanzadas para la creación de modelos tanto en dos como tres dimensiones, estas herramientas permiten crear modelos precisos y detallados, desde componentes simples hasta complejas estructuras ensambladas [19].

2.4.3.2 Fusion 360

Es una plataforma de diseño asistido por computadora (CAD) desarrollada por Autodesk. Es conocida por su enfoque integrado que combina modelado 3D, diseño paramétrico, simulación, visualización y fabricación en un único entorno colaborativo basado en la nube. Fusion 360 se utiliza principalmente en industrias como la ingeniería mecánica, diseño industrial, arquitectura y fabricación para crear modelos 3D precisos, prototipos y productos finales [20].

2.4.3.3 FreeCAD

Es un software de diseño asistido por computadora (CAD) de código abierto y multiplataforma, orientado principalmente a la ingeniería y el diseño mecánico. Permite a los usuarios crear modelos 3D precisos mediante herramientas de modelado paramétrico, lo que facilita la modificación y personalización de diseños. Es utilizado tanto por profesionales como por aficionados para una amplia gama de proyectos, desde la creación de piezas mecánicas hasta la planificación de estructuras complejas [21].

2.4.3.4 Blender

Es un software de mallas de código abierto y multiplataforma dedicado al diseño 3D, animación, modelado, renderizado, composición y creación de gráficos en movimiento. Es conocido por su amplia gama de herramientas y su comunidad activa de usuarios y desarrolladores. Se utiliza en diversas industrias, como la animación, el cine, los videojuegos, la arquitectura y la producción de medios, para crear contenido visual de alta calidad. Su interfaz personalizable y su conjunto de características avanzadas lo convierten en una opción popular tanto para artistas independientes como para estudios de animación y diseño [22].

2.4.3.5 Cinema4D

Cinema 4D es un software de modelado, animación y renderizado en 3D desarrollado por Maxon. Es conocido por su interfaz intuitiva y su facilidad de uso, lo que lo hace popular entre artistas gráficos, diseñadores y animadores. Ofrece potentes herramientas para crear efectos visuales, gráficos en movimiento y modelos 3D detallados, siendo utilizado en la industria del cine, la televisión y la publicidad [23].

2.4.3.6 Maya

Maya es un software de modelado, animación, simulación y renderizado en 3D desarrollado por Autodesk. Es ampliamente utilizado en la industria del cine, los videojuegos y la televisión para crear gráficos y efectos visuales de alta calidad. Ofrece un conjunto completo de herramientas avanzadas para la animación de personajes, simulaciones físicas, modelado y texturizado, lo que permite a los artistas y diseñadores producir contenido 3D complejo y realista [24].

2.4.4. Impresión 3D

Los grandes avances en impresión 3D llevados a cabo durante la última década, han transformado la fabricación y el diseño al posibilitar la creación de objetos tridimensionales. En este apartado, se abordarán las dos tecnologías más empleadas fuera del sector industrial, las cuales han ampliado el acceso a la impresión 3D y están cambiando la forma en que se fabrican y diseñan objetos en la vida diaria.

1. El modelado por deposición fundida (FDM)

Es un método en el que se calienta el filamento de manera que pueda ser extruido

en una plataforma, para formar la figura 3D. El filamento se extruye capa por capa de manera sucesiva hasta completar la impresión. Este proceso permite el uso de una amplia variedad de materiales, cada uno con propiedades diferentes, siendo los más importantes.

- **PLA (ácido poliláctico):** Un termoplástico biodegradable derivado de recursos renovables como el almidón de maíz o la caña de azúcar. Es el material por excelencia utilizado por principiantes y expertos, debido a que se trata de un material fácil de imprimir, ya que requiere una baja temperatura de impresión (190 - 220 °C), tiene baja contracción y warping y no emite olores. También es biodegradable y existen una gran cantidad de colores y acabados. Ofrece una buena rigidez y es ideal para prototipos y piezas no sometidas a altas tensiones mecánicas o térmicas.
- **ABS (acrilonitrilo butadieno estireno):** Un termoplástico robusto conocido por su resistencia a la abrasión, al impacto y a la deformación. Requiere una temperatura de fusión más alta (220 - 250 °C), y una cama caliente para prevenir deformaciones y agrietamiento durante la impresión, también se necesita ventilación debido a que emite vapores tóxicos durante la impresión. Es ampliamente utilizado en aplicaciones industriales y en productos de consumo, como carcasas de electrodomésticos y piezas de automóviles.
- **TPU (Poliuretano termoplástico):** Un material derivado de la combinación de caucho y plástico. Resalta por ser flexible y elástico, es conocido por su alta resistencia a los agentes químicos, la rotura, la abrasión y el desgaste, también es utilizado como material antiadherente. Requiere una temperatura de impresión de aproximadamente 230°C, pero no necesita cama caliente. Es ideal para aplicaciones que requieren flexibilidad, como carcasas de teléfonos, juntas, correas y piezas que necesiten propiedades similares al caucho.
- **PETG (tereftalato de polietileno glicolizado):** Un copoliéster que combina la facilidad de uso del PLA con la durabilidad del ABS. Tiene una mayor resistencia a los golpes que el PLA, es químicamente resistente y tiene una resistencia a la temperatura de 80 - 90°C, también permite que las impresiones puedan estar en contacto con los alimentos, es reciclable y no desprende olores. Ofrece buena adherencia entre capas y es menos propenso a deformarse que el ABS. Es ideal para piezas funcionales y aplicaciones industriales.

2. El modelado por estereolitografía (SLA)

Esta técnica utiliza un láser ultravioleta para curar y solidificar capas sucesivas de resina líquida. El proceso se realiza capa por capa, con el láser trazando el diseño de cada capa en la superficie de la resina, lo que provoca su endurecimiento. La plataforma de construcción se desplaza hacia abajo, permitiendo que una nueva capa de resina líquida cubra la capa solidificada previamente. Este ciclo se repite hasta completar el modelo 3D. La estereolitografía (SLA) es conocida por su alta precisión y su capacidad para producir detalles finos y superficies lisas, lo que la hace ideal para figuras, prototipos, piezas de joyería y otras aplicaciones que requieren un alto nivel de detalle. Sin embargo, esta técnica requiere precauciones especiales debido a la posible toxicidad y sensibilidad a la luz de las resinas. Además, se necesita post-procesamiento, que incluye un post-curado bajo luz UV y limpieza con solventes. Las resinas fotopolimerizables también pueden ser más costosas que otros materiales.

Capítulo 3

Estudio de Alternativas

El objetivo principal de este estudio es proporcionar una visión detallada de las opciones disponibles, destacando las ventajas y desventajas de cada una. A través de un análisis comparativo, se buscará identificar la alternativa más adecuada para alcanzar los objetivos del proyecto.

3.1. Software de modelado 3D

Se han considerado tres opciones: Solidworks, Fusion 360 y Blender, las cuales se describieron en la subsección 2.4.3. A continuación, se explicarán las razones por las que se planteó cada una de ellas.

1. Solidworks

Ventajas

- Cuenta con avanzadas herramientas de modelado 3D, que permiten crear diseños detallados y precisos. Los usuarios pueden aprovechar una amplia gama de funciones para modelar piezas desde formas simples hasta geometrías complejas, asegurando un alto nivel de precisión y detalle.
- Proporciona potentes herramientas de ensamblaje que permiten crear complejos ensamblajes mecánicos. Estas herramientas facilitan la verificación de errores y la simulación de la interacción entre diferentes componentes.
- Una de las funcionalidades de este software es la creación de dibujos en 2D conforme a los estándares. Permite generar vistas automáticas, agregar anotaciones y cotas.
- Cuenta con la herramienta de propiedades físicas como una de sus ventajas, permitiendo calcular de manera precisa atributos como masa, volumen, centro de gravedad e inercia de las piezas y ensamblajes. Esta funcionalidad es crucial para el diseño del gemelo digital, ya que facilita la evaluación de las características físicas y el comportamiento estructural del modelo.
- Permite integrar plugins adicionales, lo que amplía las capacidades del software para adaptarlo a las necesidades específicas del proyecto.

Desventajas

- Tiene una curva de aprendizaje pronunciada, especialmente para aquellos que no lo han utilizado antes.
- Es un software caro, tanto en términos de licencia inicial como de suscripciones y actualizaciones.
- Necesita hardware potente para funcionar de manera óptima, lo que puede implicar costos adicionales.

2. Fusion 360

Ventajas

- Tiene una versión gratuita para uso personal y educativo, lo que es beneficioso para empezar a utilizarlo desde cero.
- Es conocido por tener una curva de aprendizaje más suave en comparación con Solidworks.
- Almacena los proyectos en la nube, facilitando el acceso y la colaboración desde diferentes ubicaciones.
- Combina CAD, CAM y CAE en un solo software.
- Incluye potentes herramientas de ensamblaje como Solidworks.
- Cuenta con herramientas de modelado 3D integradas, que facilitan la creación de diseños detallados con facilidad.
- Permite añadir extensiones adicionales a través de su App Store.

Desventajas

- Aunque es robusto, algunas funciones avanzadas pueden ser limitadas en comparación con Solidworks.
- Tiene una comunidad creciente, pero que no es tan amplia como la de Solidworks.
- El trabajo en la nube puede ser un problema si no se dispone de una conexión a internet confiable.

3. FreeCAD

Ventajas

- Es gratuito y de código abierto, lo que lo hace accesible para cualquier usuario sin ningún coste de licencia.
- Es compatible con múltiples sistemas operativos, incluyendo Windows, macOS y Linux, lo que facilita su uso en diversas plataformas.
- Debido a su arquitectura modular, permite agregar o eliminar módulos según sus necesidades. Los usuarios también pueden crear sus propios scripts y macros para automatizar tareas repetitivas.
- Al ser de código abierto, cuenta con una comunidad activa de usuarios y desarrolladores que proporcionan soporte, documentación, tutoriales y plugins adicionales.
- Ofrece herramientas robustas para el modelado 3D paramétrico, lo que permite realizar cambios en los diseños fácilmente ajustando los parámetros iniciales.

Desventajas

- Se trata de un software flexible, pero tiene una curva de aprendizaje pronunciada, especialmente si no se está familiarizado con el modelado 3D o el software CAD.
- La interfaz de usuario es menos intuitiva y pulida en comparación con software comercial de CAD como Solidworks o Fusion 360, lo que puede dificultar su uso para principiantes.
- Carece de algunas de las herramientas avanzadas y funcionalidad específicas que se encuentran en software CAD comercial.
- Al ser un proyecto de código abierto, puede tener problemas ocasionales de estabilidad, y las actualizaciones pueden no ser tan frecuentes o integradas como en los software comerciales.

4. Blender

Ventajas

- Es completamente gratuito y de código abierto, lo que elimina los costos de licencia y permite modificar el software según las necesidades del usuario.
- Ofrece un conjunto completo de herramientas de modelado 3D que permiten la creación de geometrías complejas con un alto nivel de detalle.
- Proporciona herramientas avanzadas para animación y renderizado, haciendo que sea útil para visualizar el funcionamiento y el montaje de las piezas mecánicas en un entorno realista.
- Tiene una gran comunidad de usuarios que comparten recursos, tutoriales y plugins, facilitando el aprendizaje y la resolución de problemas.
- Permite la adición de plugins y scripts personalizados para extender su funcionalidad.

Desventajas

- Es potente para el modelado artístico y de animación, pero no está diseñado para ingeniería o CAD. Carece de algunas herramientas avanzadas necesarias para el diseño mecánico, como análisis de tolerancia, simulaciones de estrés y herramientas de ensamblaje precisas.
- Tiene una curva de aprendizaje muy pronunciada, especialmente si no se tiene experiencia en modelado 3D.
- Para tareas de ingeniería, los modelos creados en Blender pueden necesitar ser exportados a otros programas de CAD para análisis adicionales.

Características	Solidworks	Fusion 360	FreeCAD	Blender
Tipo de software	CAD	CAD	CAD	Mallas
Licencia	Propietaria (Pago por licencia)	Propietaria (Pago por suscripción o versión gratuita para estudiantes y startups)	Código abierto (Licencia GPL)	Código abierto (Licencia GPL)
Cantidad de herramientas	Alta	Alta	Alta	Alta
Comunidad	Muy alta	Alta	Moderada	Alta
Curva de aprendizaje	Moderada a alta	Moderada	Moderada a alta	Alta
Plugins	Si	Si	Si	Si
Soporte para diferentes plataformas	Windows	Windows y macOS	Windows, macOS y Linux	Windows, macOS y Linux
Especializado para tareas de ingeniería	Si	Si	Si	No

Tabla 3.1: Comparación de Solidworks, Fusion 360, FreeCAD y Blender

Tras haber hecho una comparativa de todas las opciones disponibles, se ha optado por Solidworks debido a que se disponía de la licencia correspondiente. Y aunque también se contaba con la opción de obtener una licencia de Fusion 360, el cual presentaba una curva de aprendizaje más suave, ya se contaba con experiencia previa en el uso de Solidworks. Esto lo convirtió en la mejor opción para ser más eficientes al crear los modelos y para consolidar los conocimientos previamente adquiridos con esta herramienta.

3.2. Plataforma robótica

Al inicio del capítulo del Estado del Arte 2.4, se han explorado diversas alternativas a la plataforma ROS. Sin embargo, se van a comparar CARMEN, YARP y ROS, por considerarse las más importantes.

1. **Modularidad:** Todos los frameworks son modulares, permitiendo a los desarrolladores combinar diferentes módulos según las necesidades del proyecto.
2. **Soporte para múltiples robots:** YARP y ROS soportan aplicaciones con múltiples robots, lo que los hace adecuados para entornos complejos. CARMEN está más enfocado en la navegación de un solo robot.

3. **Comunidad:** ROS tiene una comunidad muy activa, proporcionando una gran cantidad de recursos y soporte. YARP tiene una comunidad moderada y activa. CARMEN tiene una comunidad más pequeña y menos activa.
4. **Lenguaje de programación:** CARMEN utiliza C y C++, YARP se basa en C++, y ROS soporta tanto C++ como Python.
5. **Curva de aprendizaje:** CARMEN tiene una curva de aprendizaje más baja debido a su menor complejidad. YARP y ROS tienen una curva de aprendizaje más alta debido a su flexibilidad y capacidades avanzadas.
6. **Cantidad de paquetes y herramientas:** ROS ofrece una extensa colección de paquetes y herramientas. YARP tiene una cantidad moderada de herramientas y CARMEN es la que menos tiene.
7. **Soporte para sistemas distribuidos:** YARP y ROS están diseñados para soportar sistemas distribuidos, permitiendo la comunicación entre múltiples componentes. CARMEN no tiene esta capacidad.
8. **Documentación y recursos de aprendizaje:** ROS tiene una extensa documentación y recursos de aprendizaje disponibles. YARP y CARMEN tienen documentación y recursos más limitados.
9. **Uso en la investigación y la industria:** ROS es ampliamente utilizado tanto en la investigación como en la industria. YARP tiene un uso moderado en estos campos, mientras que CARMEN está más limitado a aplicaciones académicas y específicas de navegación.

Característica	CARMEN	YARP	ROS
Enfoque	Navegación	Comunicación distribuida	General (navegación, manipulación, etc.)
Lenguaje Principal	C, C++	C++	C++, Python
Modularidad	Moderada	Alta	Alta
Comunidad	Pequeña	Moderada	Muy amplia
Flexibilidad	Baja	Alta	Muy alta
Curva de Aprendizaje	Baja	Moderada	Alta
Interoperabilidad	Baja	Alta	Muy alta
Soporte para múltiples robots	No	Sí	Sí
Uso en la investigación y la industria	Limitado	Moderado	Alta
Cantidad de paquetes y herramientas	baja	Moderada	Alta

Tabla 3.2: Comparación de CARMEN, YARP y ROS

Tras la comparativa, se ha concluido que para el desarrollo de robots, ROS es la plataforma estándar utilizada en el sector de la robótica. Al igual que las opciones anteriores, ROS es de código abierto, lo que implica que proporciona todas las herramientas y funciones necesarias para el desarrollo, incluyendo bibliotecas, controladores, visualizadores, herramientas de construcción y más. Por último, se ha seleccionado por ser la herramienta de código abierto más potente y con la comunidad más grande.

Se ha considerado utilizar ROS o ROS 2, las cuales se mencionaron en la subsección 2.4.1. A continuación, se realiza una comparativa de las dos.

1. **Arquitectura de comunicación:** ROS 1 utiliza el middleware ROS 1, basado en el protocolo de comunicación TCP/IP, que no cumple con estándares de tiempo real y el sistema de mensajería de ROS (ROS message passing system). En cambio, ROS 2 utiliza el middleware de comunicación DDS (Data Distribution Service), que ofrece una comunicación más flexible y con soporte nativo para tiempo real.
2. **Soporte para sistemas en tiempo real:** ROS 2 ofrece soporte nativo para sistemas en tiempo real, lo que significa que puede cumplir con requisitos de tiempo determinado (hard real-time) o tiempo aproximado (soft real-time). Esto es crucial para aplicaciones robóticas que requieren tiempos de respuesta rápidos y predecibles, como la navegación autónoma y el control de movimiento.
3. **Modularidad:** ROS 2 ha sido diseñado con una arquitectura más modular que ROS 1, lo que facilita la reutilización de componentes de software y la integración con

otros sistemas. Esto permite a los desarrolladores construir sistemas más complejos y personalizados de manera más eficiente.

4. **Seguridad:** ROS 2 incluye características adicionales para mejorar la seguridad de los sistemas, como una gestión de errores más robusta, la autenticación de nodo, la encriptación de comunicaciones y control de acceso, lo que lo hace más adecuado para aplicaciones críticas en términos de seguridad.
5. **Soporte para diferentes plataformas:** Aunque ROS 1 es compatible con una amplia gama de plataformas, ROS 2 ha mejorado aún más esta compatibilidad, lo que permite ejecutar aplicaciones en una gran variedad de sistemas operativos y arquitecturas de hardware, incluidos sistemas embebidos y dispositivos con recursos limitados.

Características	ROS 1	ROS 2
Arquitectura de comunicación	Basado en el protocolo de comunicación TCP/IP y ROS Master, no soporta fácilmente sistemas distribuidos	Basado en DDS, soporta comunicación distribuida y robusta
Soporte para sistemas en tiempo real	Limitado, no está diseñado para sistemas en tiempo real	Mejora significativa, diseñado con soporte para tiempo real usando DDS
Modularidad	Modular pero con dependencias más rígidas	Más modular con mejores capacidades para componentes desacoplados
Seguridad	Seguridad básica y no integrada	Seguridad integrada a nivel de DDS con autenticación, cifrado y control de acceso
Soporte para diferentes plataformas	Principalmente Ubuntu, soporte limitado para Windows y macOS.	Soporte oficial para Ubuntu, Windows, macOS, y Fedora. Y soporte limitado para Raspbian (compilación manual), Docker (contenedores)

Tabla 3.3: Comparación de características entre ROS 1 y ROS 2

Como resultado, la elección entre ROS y ROS 2 se ha llevado a cabo teniendo en cuenta las necesidades del proyecto. Si se requiere soporte para sistemas en tiempo real, una arquitectura más modular o características avanzadas de seguridad y fiabilidad, ROS 2 podía ser la mejor opción. Sin embargo, ha sido preferible utilizar ROS en su decimotercera distribución ROS Noetic Ninjemys debido a su madurez, estabilidad y la amplia base de conocimientos y recursos disponibles.

3.3. Generación del URDF

Una vez completado el modelado del robot en Solidworks y seleccionada la plataforma robótica ROS, es necesario describir la estructura física y los componentes del robot. Para simular y visualizar el robot en el entorno de ROS, se utilizará el URDF (Unified Robot Description Format), un formato de archivo basado en XML. Este formato permite especificar detalles como las dimensiones, la geometría, las articulaciones y los enlaces del robot, así como sus propiedades físicas, como la masa y el centro de gravedad. Se han explorado dos métodos al momento de plantear la realización del URDF. A continuación se detalla una comparativa.

1. La primera opción es realizar el URDF manualmente, esto implica, una vez obtenidos los archivos STL, comenzar a construir el archivo XML desde cero. Este proceso requiere realizar cálculos para especificar las dimensiones, la geometría, las articulaciones y los enlaces del robot, así como sus propiedades físicas, como la masa y el centro de gravedad.
2. La segunda alternativa consiste en hacer uso de un plugin para la exportación de Solidworks a URDF (SolidWorks to URDF Exporter). Este plugin facilita la integración de modelos creados en SolidWorks con simuladores de robots como Gazebo, permitiendo diseñar y simular sistemas robóticos completos de manera más eficiente [25].

Después de comparar ambas opciones, se rechazó la primera opción porque, para la creación del gemelo digital, esta técnica resultaba en una solución más lenta y menos precisa. La segunda opción permite obtener el URDF completo con una precisión considerable, siempre y cuando se introduzcan correctamente todos los datos en SolidWorks. Sin embargo, una vez generado el URDF con el plugin, no está completamente listo, ya que es necesario realizar algunas modificaciones manuales. A pesar de esto, esta opción ofrece una mayor precisión y un ahorro de tiempo significativo.

3.4. Herramientas de simulación

Al inicio de la subsección 2.4.2 se han explorado diversas alternativas para la simulación del Eddie Robot. A continuación, se comparan Gazebo, CoppeliaSim, Webots, RoboDK y Unity.

1. Simulación de robots

- **Gazebo:** Ofrece una simulación robusta de robots con soporte para una amplia variedad de plataformas y sensores.
- **CoppeliaSim:** Proporciona una simulación detallada y precisa de robots, con una amplia gama de modelos predefinidos y capacidad para importar modelos personalizados.
- **Webots:** Permite la simulación de robots móviles y manipuladores con un enfoque en la simulación física y la interacción con el entorno.
- **RoboDK:** Especializado en la simulación y programación de robots industriales, con herramientas para la simulación de células de fabricación completas.

- **Unity:** No ofrece una funcionalidad de simulación de robots integrada, pero es ampliamente utilizado para el desarrollo de juegos y aplicaciones de realidad virtual que podrían incluir elementos de simulación de robots.

2. Simulación móvil

- Todas las herramientas mencionadas proporcionan simulación para robots móviles, aunque no sea su función principal, permitiendo probar algoritmos de navegación y control en un entorno virtual antes de implementarlos en hardware real.

3. Lenguajes de programación

- **Gazebo:** Se utiliza con C++ y Python para el desarrollo de controladores de robots y la integración con ROS.
- **CoppeliaSim:** Soporta Lua, Python, C++ y MATLAB.
- **Webots:** Ofrece soporte para Python, C, Java y MATLAB.
- **RoboDK:** Permite la programación de robots utilizando Python, C# y LUA.
- **Unity:** Utiliza principalmente C#, pero también soporta JavaScript y C++ para el desarrollo de aplicaciones.

4. Plataformas:

- Todas las herramientas son multiplataforma, con soporte para Windows, Linux y macOS en diferentes grados.

5. Licencia:

- **Gazebo:** Es de código abierto bajo la licencia Apache 2.0.
- **CoppeliaSim, Webots, RoboDK y Unity:** Son herramientas propietarias con diferentes modelos de licencia dependiendo de las características y el uso.

6. Integración con ROS:

- **Gazebo:** Está integrada con ROS, siendo la herramienta de simulación predefinida para muchos paquetes de robots de ROS.
- **CoppeliaSim y Webots:** Ambas herramientas ofrecen soporte para ROS a través de plugins y pueden integrarse con sistemas basados en ROS.
- **RoboDK:** Ofrece integración con ROS a través de una interfaz de usuario dedicada para la importación y exportación de modelos y programas.
- **Unity:** No tiene una integración directa con ROS, pero se pueden encontrar plugins y herramientas de terceros para facilitar la comunicación con sistemas ROS.

7. Simulación del entorno:

- Todas las herramientas proporcionan capacidades de simulación de entornos virtuales, permitiendo la creación y modificación de escenarios para probar algoritmos de control y planificación de robots.

8. Curva de aprendizaje:

- La curva de aprendizaje puede variar dependiendo de la complejidad y las características específicas de cada herramienta, pero en general, todas tienen una curva de aprendizaje moderada que requiere tiempo para controlarlas completamente.

9. Personalización:

- Todas las herramientas ofrecen un alto grado de personalización, permitiendo a los usuarios crear y modificar escenarios, modelos de robots y algoritmos de control.

Características	Gazebo	Coppelia	Webots	RoboDK	Unity
Simulación de robots	Sí	Sí	Sí	Sí	No
Simulación móvil	Sí	Sí	Sí	Sí	Sí
Lenguajes de programación	C++, Python	Lua, Python, C++, MATLAB	Python, C, Java, MATLAB	Python, C#, MATLAB, LUA	C#, JavaScript, Python, C++
Plataformas	Windows, Linux, macOS	Windows, Linux, macOS	Windows, Linux, macOS	Windows, Linux y macOS	Windows, Linux y macOS
Licencia	Open Source	Propietaria	Propietaria	Propietaria	Propietaria
Integración con ROS	Sí	Sí	Sí	Sí	No (Hacen falta plugins)
Curva de aprendizaje	Moderada	Moderada	Moderada	Moderada	Moderada
Personalización	Alta	Alta	Alta	Alta	Alta

Tabla 3.4: Comparación de características entre Gazebo, CoppeliaSim, Webots, RoboDK y Unity

Después de comparar todas las herramientas, se considera que Gazebo es la mejor elección para la simulación del robot debido a su amplia comunidad y soporte, su estrecha integración con ROS que facilita el desarrollo, y su naturaleza de código abierto que lo hace accesible para el desarrollo del gemelo digital.

Capítulo 4

Caracterización y Diseño

4.1. Medición y pesaje de las piezas del robot

Para realizar la medición y el pesado exacto de todas las piezas, en primer lugar se ha realizado el desmontaje de todo el robot Eddie (Ver Figura 4.1).

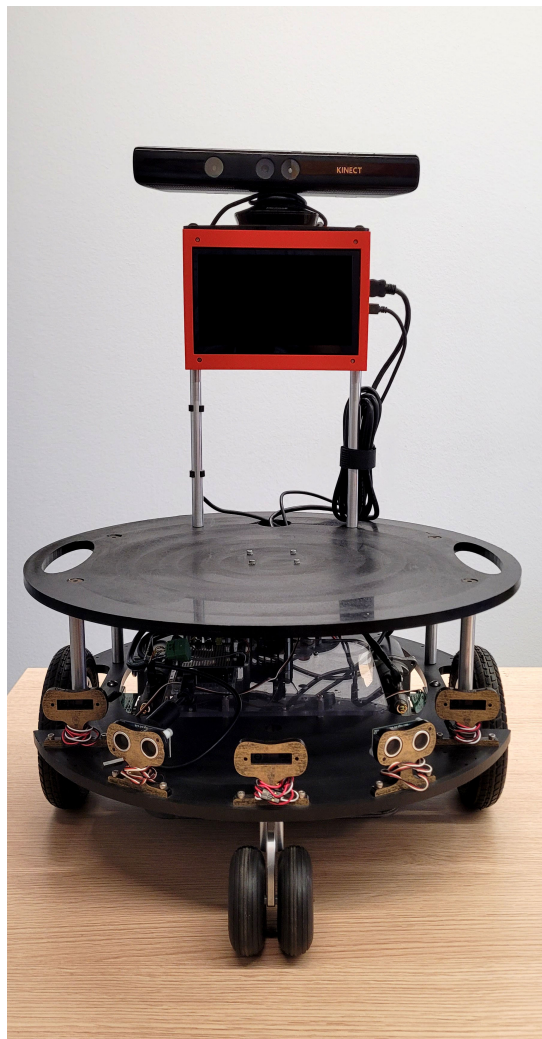


Figura 4.1: Robot Eddie

En las tablas 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, se detallan los datos obtenidos del peso para su posterior uso en el URDF.

Kinect plate assembly	peso
Standoff Kinect	107 g
Standoff Kinect x 2	214 g
Kinect deck	58 g
Total	875 g
Total sin kinect	272 g

Tabla 4.1: Kinect plate assembly

Base	peso
First deck (5 sensors + mainpower)	1351 g
Battery shelf	455 g
Battery (1)	2480 g
Battery (2)	2549 g
Average batteries	2514,5 g

Tabla 4.2: Base

Caster Wheel kit	peso
Caster wheel holder + caster + wheel	190 g
Wheels (caster) (30 %)	38 g
Caster wheel holder (35 %)	76 g
Caster (35 %)	76 g

Tabla 4.3: Caster Wheel kit

Top base	peso
Standoff base	46 g
Standoff base x4	184 g
Second deck	1540 g
Robot Control board	498 g

Tabla 4.4: Top base

wheel + motor	Peso
Soporte ruedas + motor entero pesado without battery	3357 g
Motor + reductor	1046 g
Wheel motor	388 g
Total motor + wheel	1434 g

Tabla 4.5: Wheel + motor

Componentes diversos	Peso
Carcasa pantalla + pantalla	403 g
Kinect	603 g

Tabla 4.6: Componentes diversos

En segundo, lugar se han realizado las medidas y fotos necesarias para el diseño 2D de las piezas del robot.

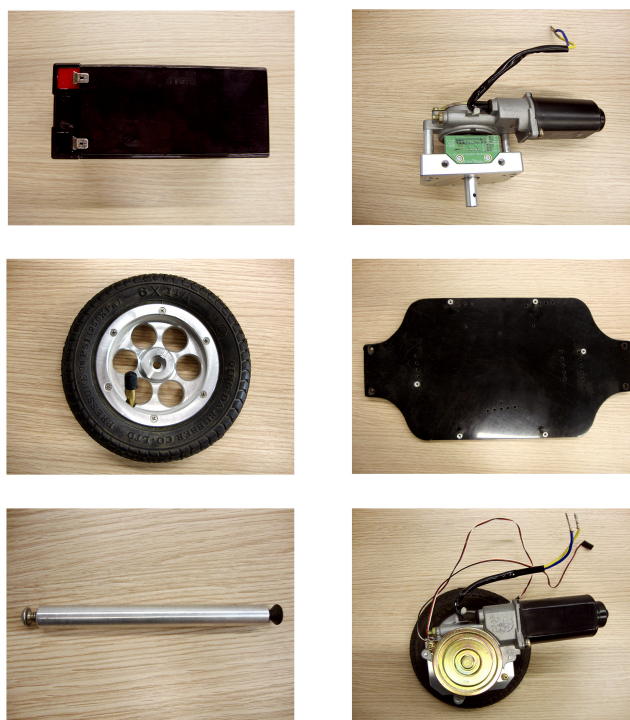


Figura 4.2: Piezas del robot

Más allá del croquis realizado a mano, se ha utilizado la herramienta de dibujo en SolidWorks, la cual permite crear representaciones técnicas detalladas de los modelos 3D. Esta herramienta facilita la generación de vistas ortográficas, secciones, detalles ampliados y anotaciones precisas, como dimensiones, tolerancias y símbolos de acabado superficial. En las Figuras A.1, A.2, A.3, A.4, A.5, A.6, A.7, A.8, A.9, A.10, A.11 y A.12, se muestran las piezas en 2D que forman en su conjunto el Robot Eddie.

4.2. Modelado en 3D

En el modelado del Eddie Robot, se han diseñado todas las piezas por separado para su posterior ensamblaje. Las mismas se detallan en las Figuras B.1, B.2, B.3, B.4, B.5, B.6, B.7, B.8, B.9, B.10, B.11, B.12 y B.13.

La figura 4.3 muestra una vista frontal y trasera, respectivamente, del ensamblaje final en SolidWorks del Eddie Robot.



Figura 4.3: Ensamblaje completo del Eddie Robot - vista frontal y trasera

4.3. Generación del URDF

La herramienta utilizada ha sido SolidWorks to URDF Exporter [25]. La misma, permite exportar cómodamente partes y ensamblajes de SolidWorks a un archivo URDF, creando un paquete similar a ROS que contiene un directorio para mallas, texturas y robots.

Para ejecutar el proceso de exportación de un ensamblaje de SolidWorks a URDF utilizando el exportador, se han realizado los siguientes pasos:

1. Abrir el ensamblaje en Solidworks
2. Establecer la posición de los joints tal y como se desea exportarlos. En la Figura 4.4 se muestra cómo se han posicionado los joints en un sistema de coordenadas.

5. A continuación, se abre una pestaña donde se muestra la configuración de las propiedades de los joints, para poder modificar los orígenes de articulación debido a que no suelen crearse en la ubicación más deseable. Se pueden cambiar los sistemas de coordenadas de referencia y los ejes para adaptarlos a las necesidades fuera del exportador. Hay que asegurarse de que se guardan los nombres correctos de los sistemas de coordenadas y ejes para cada enlace. En la Figura 4.6 se muestra un ejemplo de la pantalla de configuración.

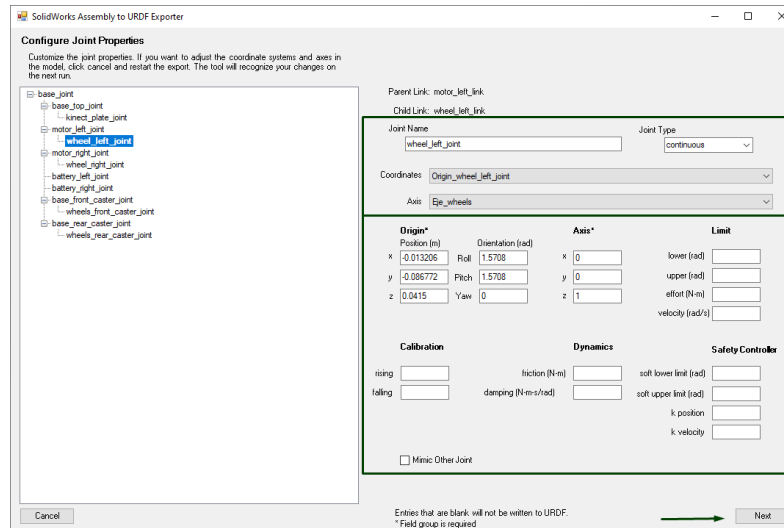


Figura 4.6: Ejemplo de configuración de propiedades de los joints

6. Para finalizar, se abre una pestaña que permite configurar los links. Se debe de modificar en esta pestaña la información de los momentos de inercia y la masa correspondientes a cada link, debido a que no sale la información correcta. Los datos de las masas se obtuvieron en la Sección 4.1, y los momentos de inercia se obtienen utilizando la herramienta de Propiedades físicas en SolidWorks. Ver Figura 4.7.

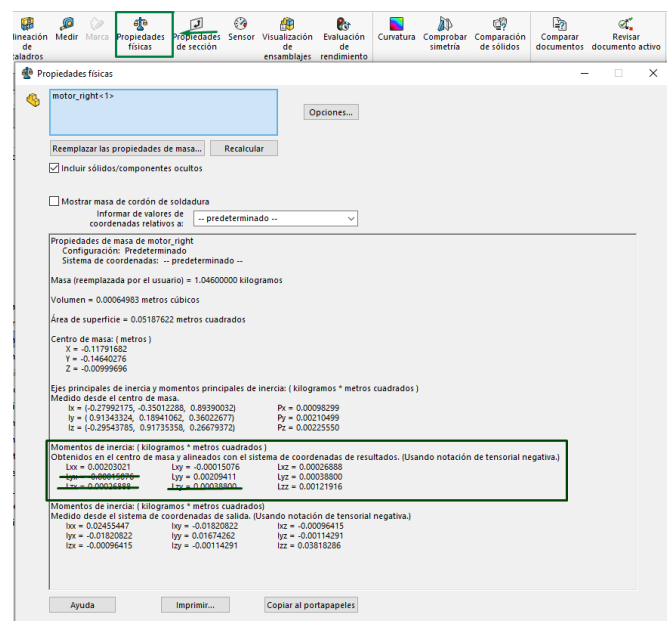


Figura 4.7: Ejemplo de herramienta de propiedades físicas

Tras haber finalizado la configuración de los datos, por último, pulsamos sobre Export URDF and meshes. Detalle en la Figura 4.8.

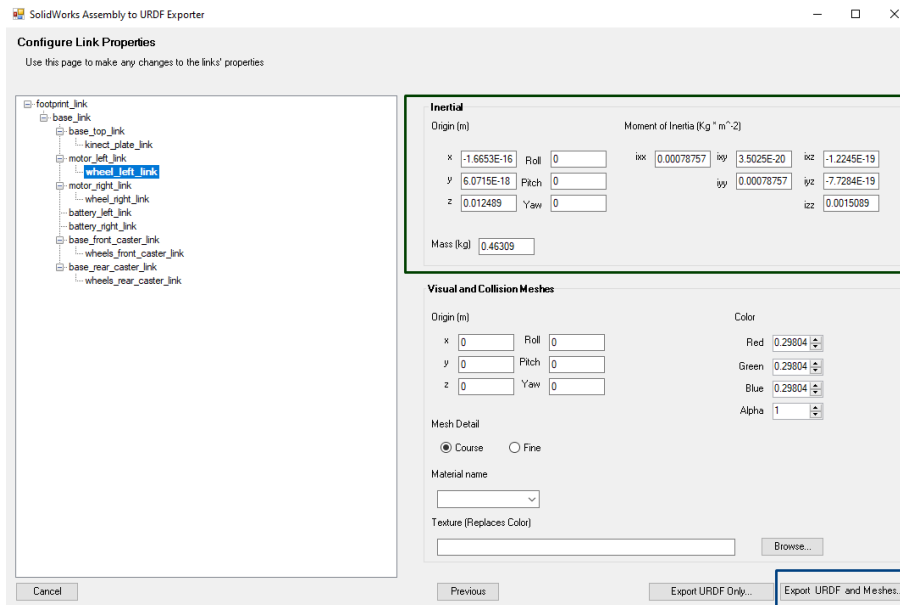


Figura 4.8: Ejemplo de configuración de propiedades de los links

7. Después de obtener el URDF, se ha utilizado una herramienta online (URDF viewer) [26] que funciona como un visor de URDF para verificar y ajustar correctamente los joints y los ejes. En la Figura 4.9 se muestra un ejemplo. Esta herramienta permite realizar diversas pruebas sin la necesidad de cambiar de sistema operativo de forma continua.



Figura 4.9: Ejemplo del visor de URDF

4.4. Modelado de la carcasa de la pantalla

Adicionalmente a la creación del Gemelo Digital, se aprovechó para realizar el diseño de una carcasa para poder incluir una pantalla en el Robot Eddie. En las Figuras 4.10, A.13, A.14, se muestra la pantalla a integrar en la carcasa, y el diseño 2D.



Figura 4.10: Pantalla del robot

Ver en las Figuras A.13 y A.14, el diseño en 2D de las dos piezas que forman el ensamblaje de la carcasa.

El ensamblaje en 3D de las dos piezas se detalla en la Figura 4.11.

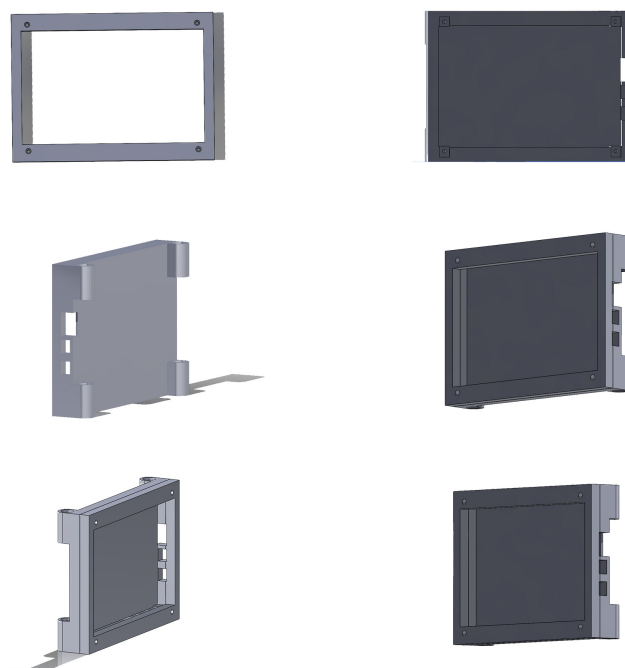


Figura 4.11: Ensamblaje de la carcasa de la pantalla

La carcasa impresa junto a su pantalla, se muestran en la Figura 4.12.



Figura 4.12: Carcasa impresa con la pantalla

Durante el proceso de diseño de la pieza, se encontraron las siguientes dificultades.

- Obtención incorrecta de las medidas de la pantalla, lo que obligó a realizar numerosas modificaciones.
- Incertidumbre sobre cómo plantear exactamente la entrada de los tornillos, ya que, había que tener en cuenta el tipo de cabeza del tornillo (hexagonal, redonda, cilíndrica o avellanada), longitud y diámetro, para poder hacer correctamente los cuatro agujeros.
- Rediseño repetido de las entradas del HDMI y los dos puertos USB, para conseguir que entrasen correctamente.
- Para reducir el gasto de filamento durante la impresión, se decidió no hacer completa la entrada de las varillas. En la Figura 4.13 se muestra un ejemplo de cómo era anteriormente, y la parte que se eliminó.

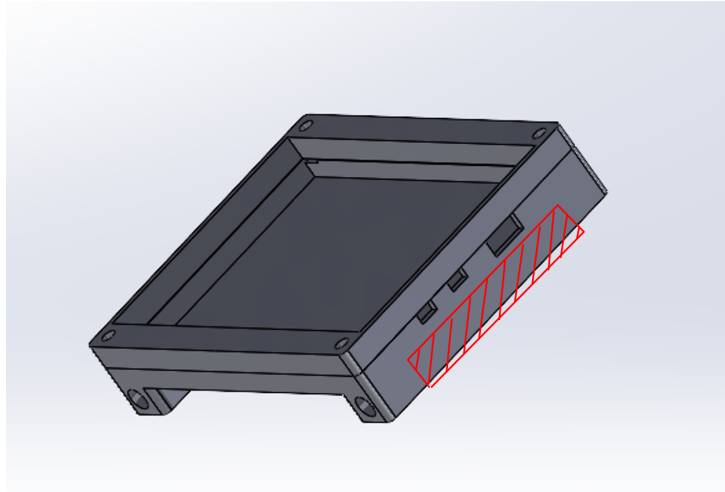


Figura 4.13: Ejemplo de versión anterior de la carcasa

- Inicialmente, se planteó usar bridas o abrazaderas en los extremos para mantener el soporte en las varillas, pero una vez impreso, el soporte se mantenía por sí solo en las varillas.

Capítulo 5

Implementación y Validación del Sistema

5.1. Software

Para utilizar ROS Noetic, primero se ha instalado el sistema operativo Ubuntu en su versión 20.04, utilizando el arranque dual (Dual Boot).

Ubuntu es una distribución GNU/Linux basada en Debian GNU/Linux, que incluye principalmente software libre y de código abierto.

A continuación, para realizar la instalación de ROS y Gazebo, se deben de seguir una serie de pasos que se encuentran en la página oficial de ROS [27]. La instalación se realiza mediante comandos a través del terminal. Previamente, es necesario configurar los repositorios de Ubuntu para permitir el acceso a los mismos. Finalmente, se han descargado e instalado varios paquetes necesarios que se detallaran en siguientes secciones.

5.2. Creación del entorno de trabajo del Eddie Robot

Para crear el workspace, se han ejecutado los siguientes comandos:

Comandos para crear el workspace

```
mkdir -p ~/eddiebot_ws/src  
cd ~/eddiebot_ws/  
catkin_make
```

Después de ejecutar los comandos, se genera una carpeta llamada `eddiebot_ws`, que incluye tres subcarpetas: `build`, `devel` y `src`.

Para comprender mejor el contenido y la función de cada una de ellas, se ofrece una breve descripción:

- La carpeta `build` es el espacio de compilación del workspace. En esta carpeta, CMake se invoca para compilar los paquetes. Aquí se almacena toda la información

de compilación y los archivos temporales necesarios durante este proceso, incluyendo la información de caché que CMake utiliza para optimizar las compilaciones futuras.

- La carpeta `devel` es el espacio de desarrollo del workspace. En esta carpeta se colocan las librerías y bibliotecas generadas durante la compilación, pero antes de que se instalen formalmente. Esta carpeta contiene todos los archivos necesarios para probar y ejecutar los paquetes dentro del workspace sin necesidad de instalarlos globalmente en el sistema.
- La carpeta `src` es la carpeta principal del workspace. Contiene los códigos fuente de todos los paquetes que se desean compilar y desarrollar. Aquí es donde se pueden verificar, extraer y clonar los paquetes que forman parte del proyecto. En esta carpeta se alojará el contenido principal del trabajo de fin de grado, incluyendo todos los paquetes y scripts necesarios para el correcto desarrollo y funcionamiento del proyecto.

Una vez creado el entorno del proyecto, la carpeta `robot_description` generalmente se utiliza para contener todos los archivos relacionados con la descripción del robot. Esta carpeta es importante porque define cómo es y cómo se comporta el robot en el espacio simulado o en el mundo real.

Dentro de la carpeta `robot_description`, se pueden encontrar varios tipos de archivos, incluyendo:

- **URDF (Unified Robot Description Format):** Es un formato XML utilizado para describir el modelo del robot, incluyendo sus enlaces, uniones, geometría, y otras propiedades físicas. Por ejemplo, un archivo `robot.urdf` podría definir la estructura y la articulación del robot.
- **Xacro (XML Macros):** Xacro es una extensión de URDF que permite utilizar macros para simplificar la creación de descripciones de robots más complejas. Los archivos Xacro (`.xacro`) se preprocesan para generar un archivo URDF.
- **Mallas 3D (Meshes):** Archivos de geometría 3D en formatos como `.dae`, `.stl` o `.obj` que representan las partes físicas del robot. Estas mallas se refieren en el URDF/Xacro para visualizar el robot en Rviz o Gazebo.
- **Archivos de configuración:** Pueden incluir archivos de configuración específicos del robot, como configuraciones de sensores, parámetros de control, etc.

5.3. Implementación del Gemelo Digital

Para la implementación del Robot Eddie se ha tenido que definir la estructura del robot, ver Código en C.1. Para añadir la kinect se ha dividido su descripción en otro componente, ver código en C.2.

Una vez definida la estructura, se ha tenido que definir el archivo que permite añadir configuraciones y plugins específicos que solo son relevantes en el entorno de simulación de Gazebo. Ver código en C.3.

Para poner iniciar Gazebo y RVIZ, se han tenido que configurar su archivos correspondientes, Ver en C.4 y C.5. Los cuales permiten automatizar el inicio de varios nodos y configuraciones con un solo comando, lo cual es muy útil para pruebas y desarrollo.

5.4. Puesta a punto del modelo digital

Para ajustar el modelo, se han configurado manualmente los archivos URDF durante la simulación para verificar que los parámetros de inercia y colisión, así como toda la estructura y los ejes estuvieran correctamente alineados.

Posteriormente, para mejorar la dinámica, ha sido necesario configurar diversas partes del archivo Gazebo.xacro.

- **Configuración del plugin de control:** Para configurarlo correctamente, se han ajustado los siguientes parámetros.
 - **wheelSeparation:** Distancia entre las ruedas izquierda y derecha.
 - **wheelDiameter:** Diámetro de las ruedas.
 - **torque:** Torque máximo aplicable a las ruedas. Este valor se ha tenido que aproximar por no contar con los datos suficientes. Ver información del calculo en 5.7.
 - **leftJoint:** Nombre de la unión de la rueda izquierda.
 - **rightJoint:** Nombre de la unión de la rueda derecha.
- **Configuración de las propiedades físicas de las ruedas:** Esta parte ha sido crucial para lograr un comportamiento realista del robot. Estas configuraciones incluyen parámetros como fricción, inercia, masa, y otros aspectos que afectan en cómo las ruedas interactúan con el entorno y cómo se mueven.

A continuación, se explica cómo son las propiedades físicas de las ruedas a configurar.

- **mu1 y mu2:** Coeficientes de fricción en las dos direcciones tangenciales a la superficie de contacto. Valores más altos significan mayor fricción.
- **kp y kd:** Constantes del resorte y amortiguador para el contacto. Controlan la rigidez y la amortiguación del contacto.
- **fdir1:** Dirección de la fricción en el primer eje tangencial. Útil para ruedas que deben deslizarse principalmente en una dirección específica.
- **maxVel:** Velocidad máxima permitida en el punto de contacto.
- **minDepth:** Profundidad mínima de penetración antes de que se considere un contacto. Valores más pequeños pueden mejorar la precisión del contacto.

5.5. Pruebas de rendimiento

En primer lugar, inicializamos ROS utilizando.

```
roscore
```

A continuación, se inicializa Gazebo. Ver un ejemplo de Gazebo inicializado con el EddieRobot en la Figura 5.1.

```
roslaunch eddiebot_description gazebo.launch
```

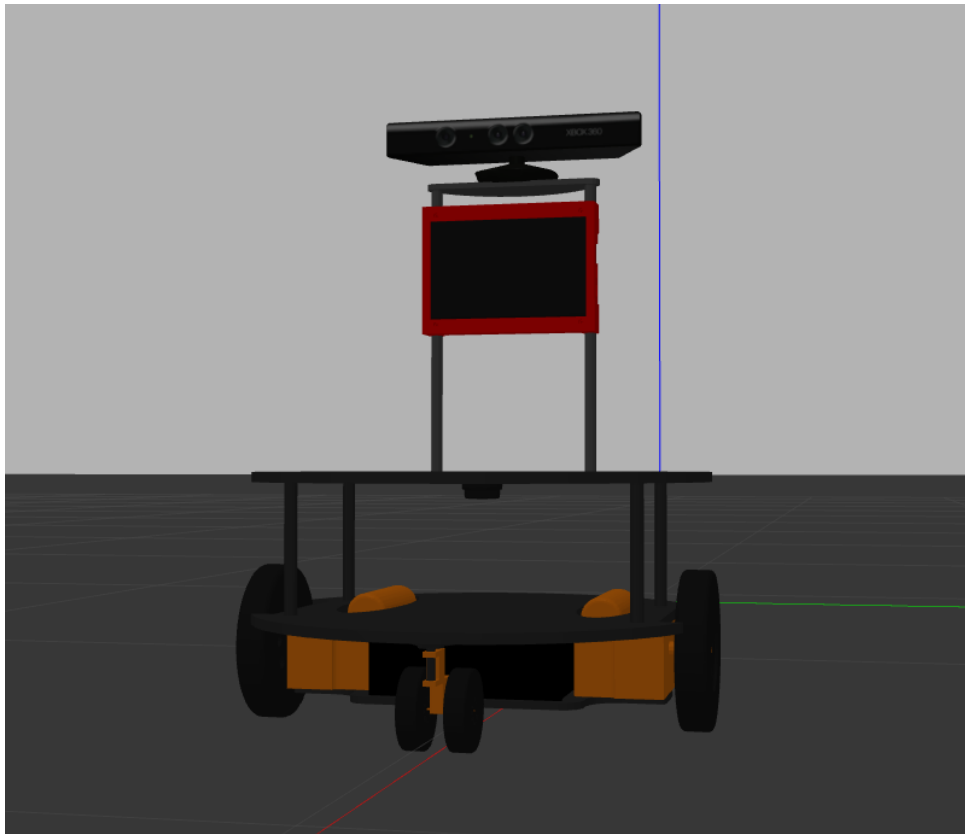


Figura 5.1: Ejemplo de Eddie Robot en Gazebo

Para verificar el rendimiento del ajuste del robot se han utilizado dos tópicos.

- **Odom:** Refleja una estimación de la posición y velocidad del robot basados en sus sensores (principalmente encoders), y no en la posición “real” exacta. Esta estimación es susceptible a errores.

Se utiliza el comando:

```
rostopic echo \odom
```

- **Pose3D:** Proporciona la posición y orientación precisas del modelo simulado del robot en el entorno virtual.

Se utiliza el comando:

```
rostopic echo \pose3d
```

Para verificar el correcto comportamiento se ejecuta el siguiente comando, el cual le indica al robot la velocidad a la que se debe de mover:

```
rostopic pub -r 10 /cmd_vel geometry_msgs/Twist'{  
  linear: {x: 0.1, y: 0.0, z: 0.0},  
  angular: {x: 0.0,y: 0.0,z: 0.3}}'
```

A continuación, se verifica visualmente que el comportamiento es el correcto. Para ver si se movía correctamente, una vez se ajustaron los parámetros correspondientes y se observó que se movía correctamente. Ver Figura 5.2.

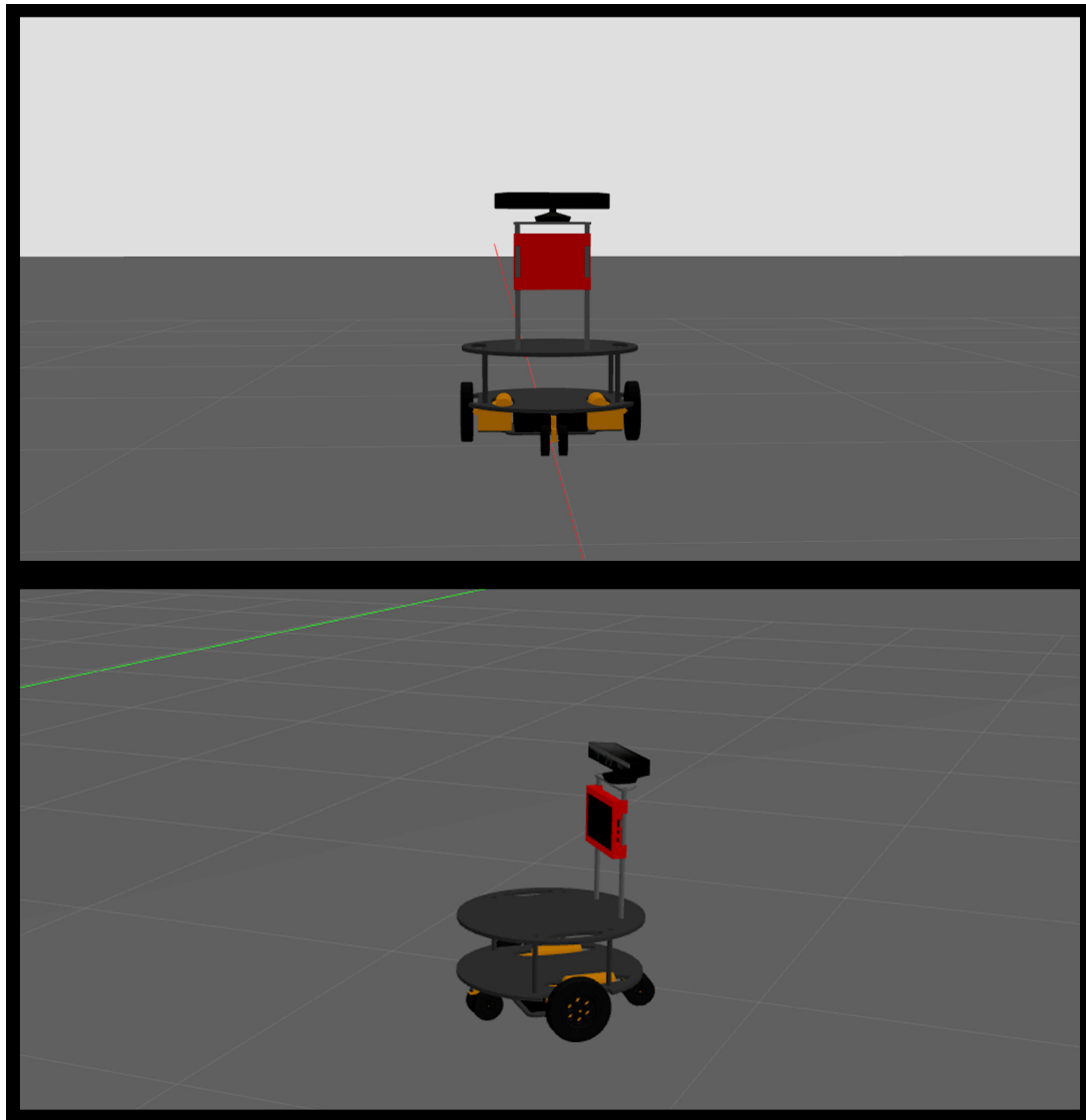


Figura 5.2: Ejemplo del comportamiento del Eddie Robot

Con el objetivo de efectuar los ajustes necesarios y verificar que la velocidad era la adecuada en los ejes x (lineal) y z (angular), según lo requerido, se han llevado a cabo tres pruebas utilizando los tópicos “odom” y “pose3D”. Estos tópicos muestran por terminal en tiempo de simulación los valores de velocidad y posición. La interactividad genera un problema para el correcto análisis comparativo de los datos. Por tanto, se ha creado un script de bash (ver C.6), para que los datos que generan interactivamente se guarden en dos archivos .txt. A continuación, con un script de Python se han leído y plotado los datos de interés para realizar la comparativa.

1. **Velocidad lineal:** Se obtuvieron datos durante 15 segundos al aplicar un escalón de 0 a 0.5 en la velocidad lineal en el eje X.

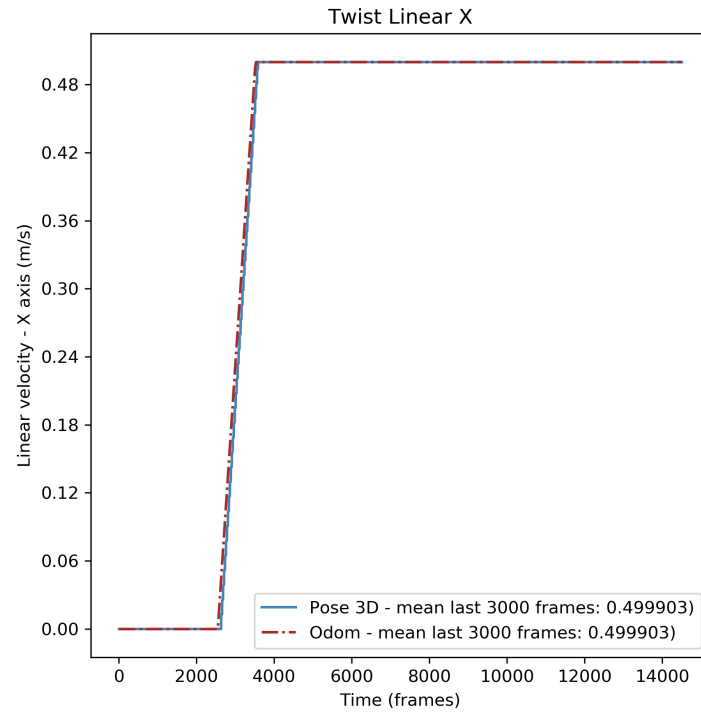


Figura 5.3: Gráfica - Velocidad lineal en X

En la Figura 5.3 se observa que la odometría del robot en el entorno virtual se comporta de manera consistente con los datos reales del robot.

2. **Velocidad angular:** Se obtuvieron datos durante 15 segundos al aplicar un escalón de 0 a 0.5 en la velocidad angular en el eje Z.

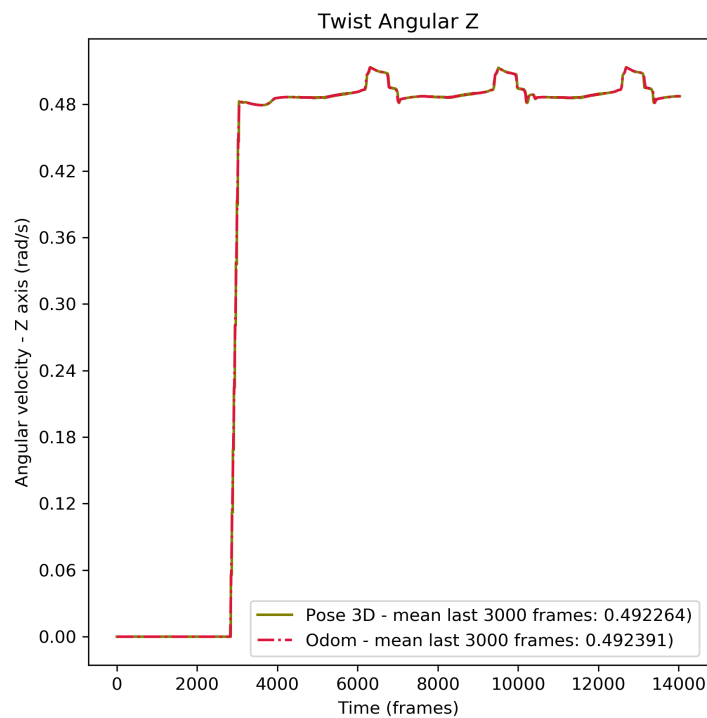


Figura 5.4: Gráfica - Velocidad angular en Z

La Figura 5.4 muestra que la odometría del robot en el entorno virtual mantiene una coherencia con los datos reales, aunque existe una ligera disminución en la precisión. La cual no es apreciable en la gráfica, pero es evidente al calcular el promedio de los últimos 3000 frames (ver leyenda Figura 5.4), donde se observa un pequeño error a partir del cuarto decimal.

3. **Velocidad angular y lineal:** Se obtuvieron datos durante 15 segundos al aplicar un escalón de 0 a 0.5 en la velocidad angular en el eje Z y en la velocidad lineal en el eje X.

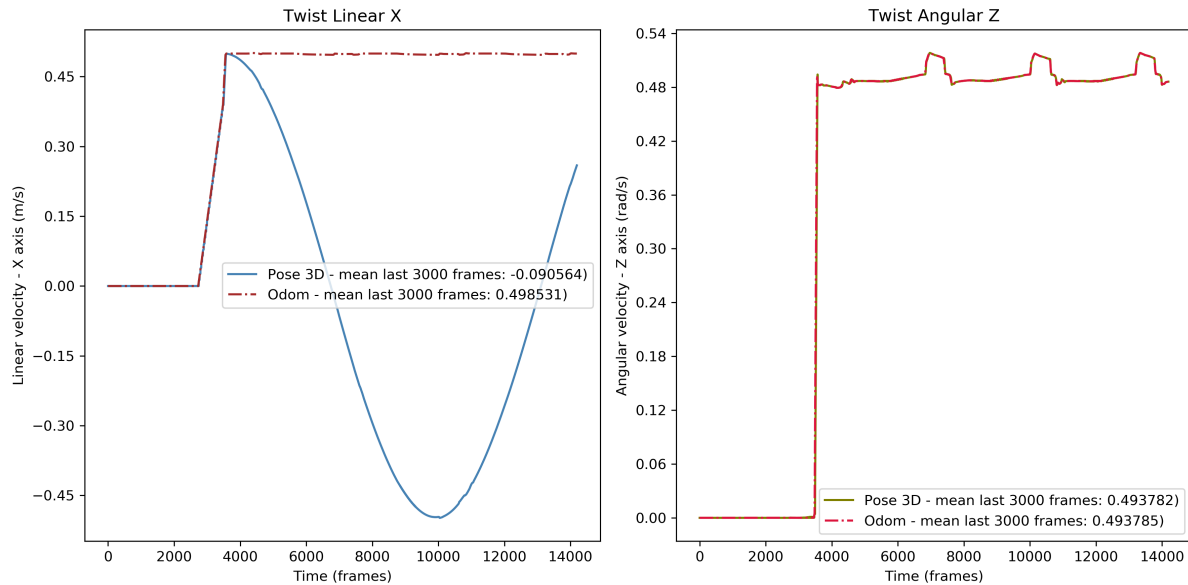


Figura 5.5: Gráfica - Velocidad angular y lineal

En la Figura 5.5, se observa una consistencia en la precisión de la velocidad general del sistema. Sin embargo, al examinar la velocidad lineal (ver gráfica izquierda en Figura 5.5), se nota que la odometría proporciona datos precisos, mientras que el pose3D no. Esta discrepancia no se debe a un error, sino más bien a cómo cada método calcula la velocidad del robot. La odometría se basa únicamente en los datos de las ruedas o actuadores para estimar la velocidad, mientras que el pose3D utiliza información absoluta sobre la posición y orientación del robot en el entorno virtual. Esto implica que el pose3D considera tanto los movimientos angulares como lineales del robot, lo cual puede diferir significativamente de la odometría en ciertos escenarios.

5.6. Pruebas de sensorizado

Se ha implementado una cámara Kinect y un sensor LIDAR en el robot. Para observar su funcionamiento, primero se debe inicializar Gazebo con el comando 5.5 detallado en la sección anterior. Además, se utiliza RViz (Robot Visualization), una herramienta de visualización 3D que forma parte de ROS, la cual es fundamental para visualizar la información de los sensores y los datos del estado del robot en tiempo real. Para inicializar RViz, se utiliza el siguiente comando:

```
roslaunch eddiebot_description rviz.launch
```

Para evaluar su funcionamiento, primero se ha creado un mapa en Gazebo con paredes y obstáculos, para realizar una serie de pruebas.

- **Lidar (Light Detection and Ranging):** Se utiliza el tópic `LaserScan` en ROS para transmitir los datos del sensor LIDAR.

En RVIZ, se puede observar la representación visual de los datos del LIDAR en la ventana 3D, viendo cómo los puntos reflejan las distancias a los objetos en el entorno. Se presenta un ejemplo en la Figura 5.6.

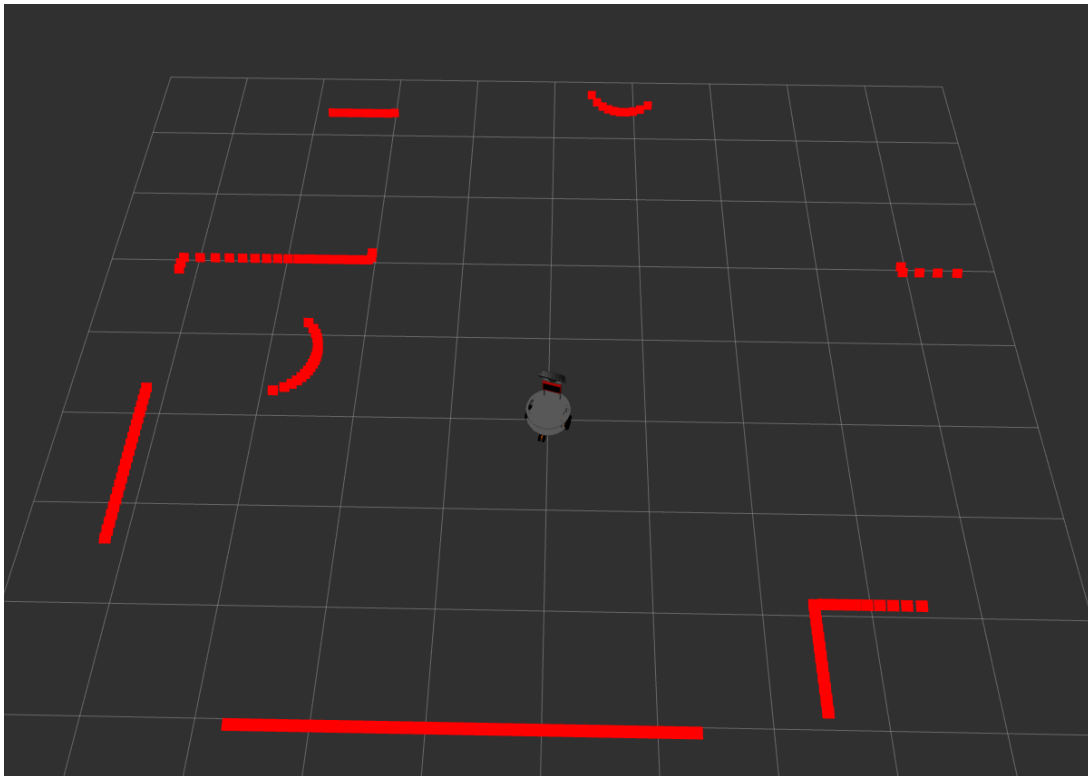


Figura 5.6: Ejemplo funcionamiento del LIDAR en RVIZ

Por otra parte, en Gazebo, se puede observar una representación visual de los datos del sensor LIDAR en el entorno simulado. Es decir, se pueden ver los rayos láser emitidos por el sensor, los cuales se visualizan como líneas o haces que se extienden desde el sensor hacia las superficies con las que interactúan. Los puntos donde impactan los rayos se muestran como puntos de impacto. Estos puntos indican la distancia medida desde el sensor hasta los objetos. Ver ejemplo en la Figura 5.6.

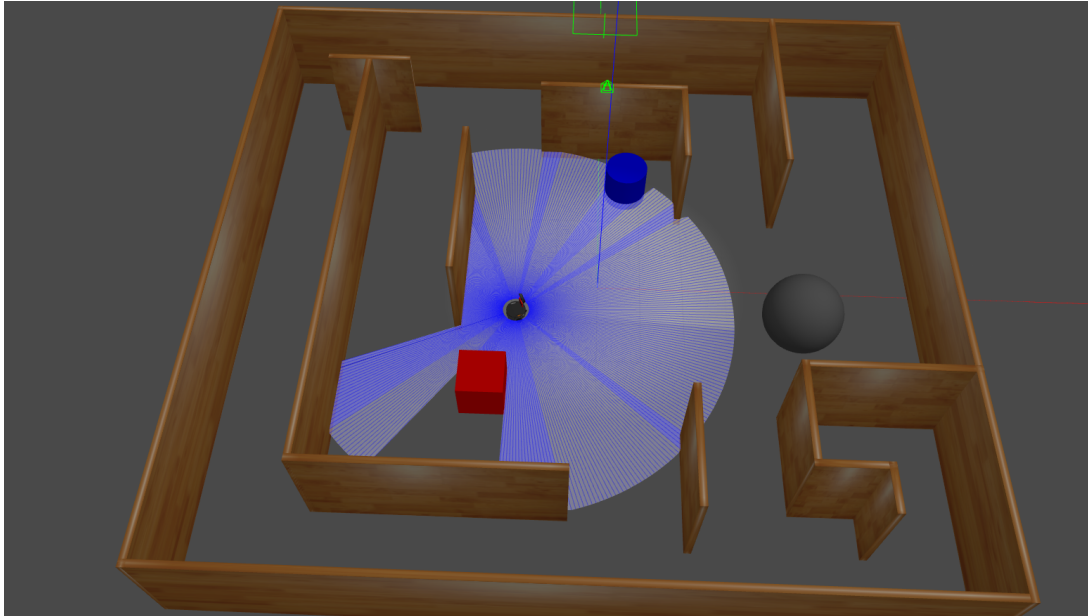


Figura 5.7: Ejemplo funcionamiento del LIDAR en Gazebo

- **Kinect:** En RViz, los tópicos Camera y PointCloud2 se utilizan para visualizar datos específicos obtenidos por la Kinect. Ver un ejemplo en la Figura 5.8.
1. **Image:** Visualiza las imágenes en tiempo real desde una cámara. Es útil para monitorear lo que la cámara del robot está viendo, generando una transmisión en vivo.
 2. **PointCloud2:** Visualiza la nube de puntos 3D capturada por la cámara. Es útil para entender la estructura 3D del entorno del robot.

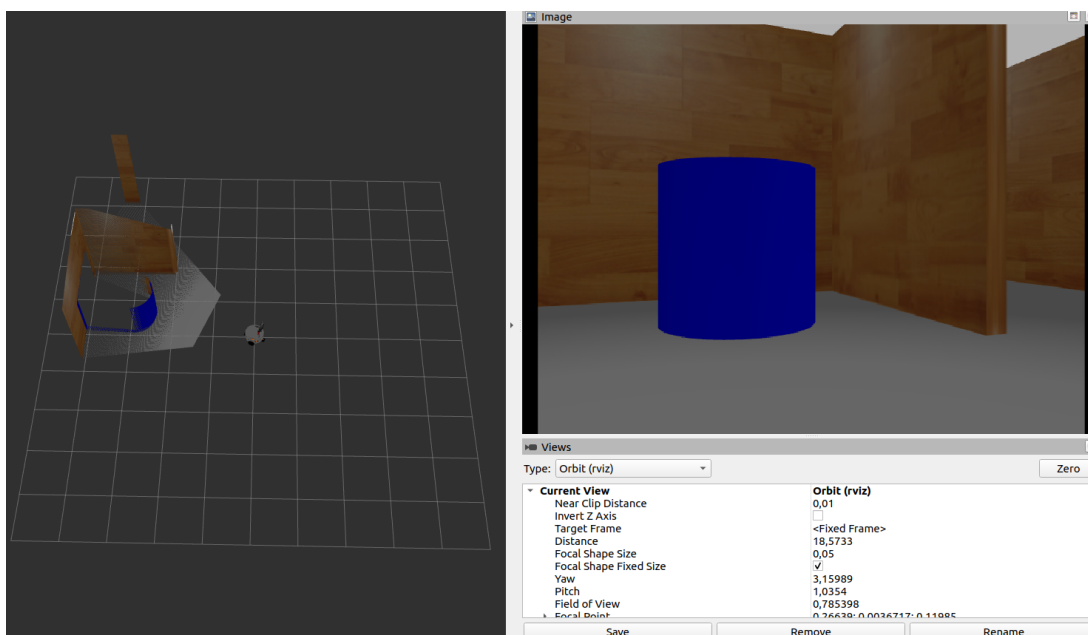


Figura 5.8: Ejemplo funcionamiento de la Kinect en el robot

5.7. Cálculo del torque sin carga

Para el cálculo del torque sin carga se han obtenido del datasheet del Eddie Robot los siguientes datos :

- **Voltaje nominal de operación del motor de corriente continua (DC):** 12.0 VDC
- **Corriente sin carga:** Aprox. 1.50 Amperios
- **Peso por conjunto de tracción:** 3.15 lbs (dos incluidos en este kit)
- **Velocidad aproximada:** 150 RPM @ 12.0 VDC, 1.50 A, sin carga

Con la información proporcionada sobre el motor, se puede utilizar la siguiente relación para estimar el torque del motor:

$$\text{Torque} = \frac{P}{\omega}$$

donde:

- P es la potencia en vatios (W).
- ω es la velocidad angular en radianes por segundo (rad/s).

Para estimar el torque, primero se necesita calcular la potencia del motor y luego la velocidad angular.

1. **Potencia del motor:** La potencia eléctrica P puede ser calculada usando la siguiente fórmula:

$$P = V \times I$$

donde:

- V es el voltaje (12 VDC).
- I es la corriente (1.50 A).

$$P = 12 \text{ V} \times 1,50 \text{ A} = 18 \text{ W}$$

2. **Velocidad angular:** La velocidad angular ω en radianes por segundo se puede obtener de la velocidad en RPM (revoluciones por minuto) usando la conversión:

$$\omega = \text{RPM} \times \frac{2\pi}{60}$$

Dado que la velocidad sin carga es de aproximadamente 150 RPM:

$$\omega = 150 \text{ RPM} \times \frac{2\pi}{60} \approx 15,71 \text{ rad/s}$$

3. Cálculo del torque:

$$\text{Torque} = \frac{P}{\omega} = \frac{18 \text{ W}}{15,71 \text{ rad/s}} \approx 1,15 \text{ Nm}$$

Esta estimación supone que la potencia del motor se convierte completamente en torque a la velocidad sin carga. Sin embargo, en la práctica, el torque real puede variar debido a la eficiencia del motor y otros factores, especialmente bajo carga. Para obtener un cálculo más preciso, se requiere información adicional sobre la eficiencia y las características del motor bajo carga, las cuales se obtienen mediante pruebas con el robot real.

Capítulo 6

Conclusiones

Es difícil decir que es imposible, porque el sueño de ayer es la esperanza de hoy y la realidad de mañana
Robert H. Goddard

6.1. Conclusiones

El trabajo desarrollado supone un proceso exhaustivo y enriquecedor centrado en el diseño y simulación del robot móvil Eddie Robot utilizando herramientas avanzadas como ROS y Gazebo. El objetivo principal fue asegurar que el robot no solo fuera capaz de moverse correctamente, sino también de interactuar con su entorno mediante la integración de sensores como el Lidar y la cámara Kinect.

Durante el desarrollo del proyecto, se realizaron múltiples iteraciones de diseño y simulación para optimizar el rendimiento. Esto implicó ajustes en la mecánica del robot, la programación de sus movimientos y la calibración precisa de los sensores para garantizar mediciones exactas y confiables. La utilización de Gazebo como plataforma de simulación resultó fundamental, permitiendo la visualización detallada del comportamiento del robot y facilitando realizar pruebas de funcionamiento.

La integración del Lidar y la cámara Kinect amplió las capacidades perceptivas del robot, permitiéndole detectar obstáculos. Estos sensores también expandieron su utilidad en aplicaciones prácticas como la exploración de entornos desconocidos.

Además de los aspectos técnicos, este proyecto representa un desafío intelectual y académico que me ha permitido aplicar y consolidar los conocimientos teóricos adquiridos a lo largo de mi formación universitaria. A través del diseño, simulación y prueba del robot móvil Eddie Robot, he fortalecido mis habilidades en ingeniería de robots, resolución de problemas y gestión de proyectos, preparándome así para enfrentar nuevos desafíos en el campo de la robótica y la automatización.

En conclusión, este Trabajo de Fin de Grado no solo ha sido una demostración de habilidades técnicas y creativas, sino también un testimonio del compromiso y la dedicación necesarios para llevar a cabo proyectos innovadores en el campo de la ingeniería robótica.

Para facilitar el análisis del trabajo realizado, se ha puesto a disposición el código en GitHub. El acceso al repositorio se detalla en el siguiente enlace:

https://github.com/Ignix98/eddiebot_ws.git

La disponibilidad de estos recursos en GitHub fomenta la colaboración y el avance conti-

nuo en este campo de estudio.

6.2. Trabajo futuro

Para futuras continuaciones de este Trabajo de Fin de Grado, es importante abordar los desafíos técnicos que surgieron durante el proyecto. Debido a limitaciones con el robot físico, no se logró completar el ajuste preciso de los parámetros del modelo simulado para reflejar completamente el comportamiento del robot real, una meta inicialmente planteada. Resolver esta discrepancia es fundamental para mejorar la precisión y utilidad del Gemelo Digital del robot móvil Eddie Robot en futuras simulaciones.

Además, se propone ampliar las funcionalidades del Gemelo Digital integrando herramientas avanzadas como el mapeado del entorno en tiempo real utilizando el paquete gmapping en ROS (Robot Operating System). Esta adición permitiría al Gemelo Digital generar mapas detallados del entorno durante las simulaciones, utilizando datos de sensores como Lidar y cámaras. La implementación de gmapping no solo enriquecería la capacidad del Gemelo Digital para pruebas virtuales más realistas, sino que también facilitaría el desarrollo y la validación de algoritmos de navegación más sofisticados.

En resumen, las áreas de trabajo futuro incluyen la optimización de los parámetros del modelo simulado del Eddie Robot para mejorar su correspondencia con el comportamiento real, así como la expansión de las funcionalidades del Gemelo Digital mediante la implementación de herramientas avanzadas de mapeo como gmapping en ROS. Estos avances no solo fortalecerán la robustez del proyecto en entornos de investigación y desarrollo en robótica, sino que también abrirán nuevas posibilidades para explorar y avanzar en la tecnología de robots móviles.

Apéndice A

Diseño 2D

A.1. Kinect plate assembly

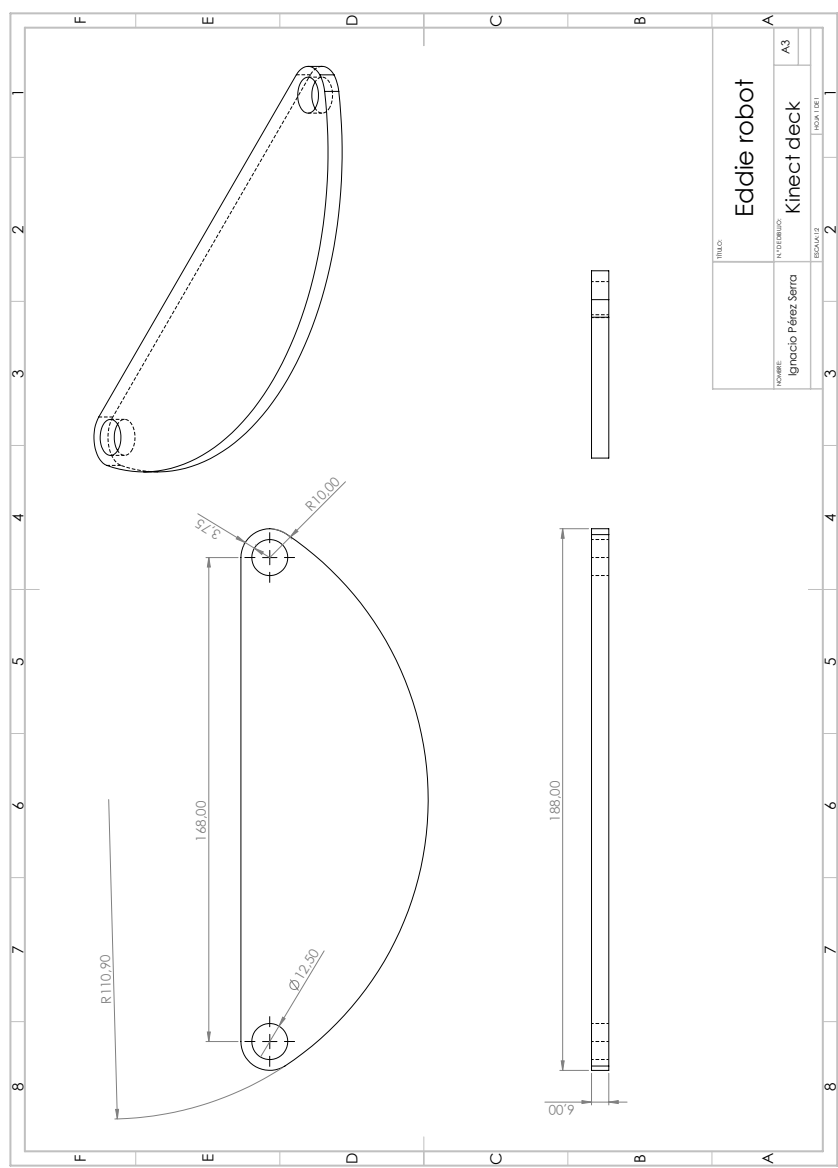


Figura A.1: Kinect deck

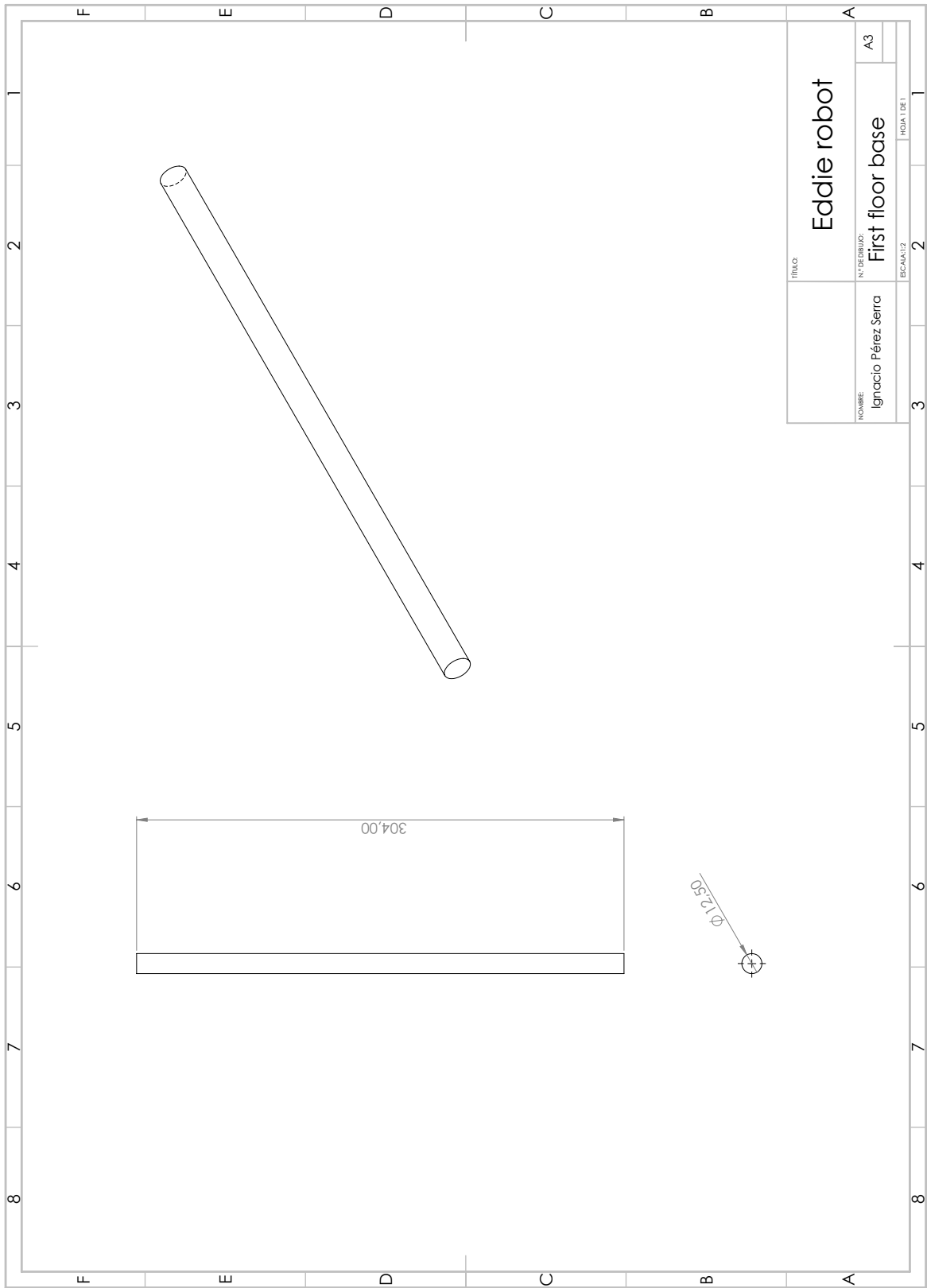


Figura A.2: Standoff kinect

A.2. Base

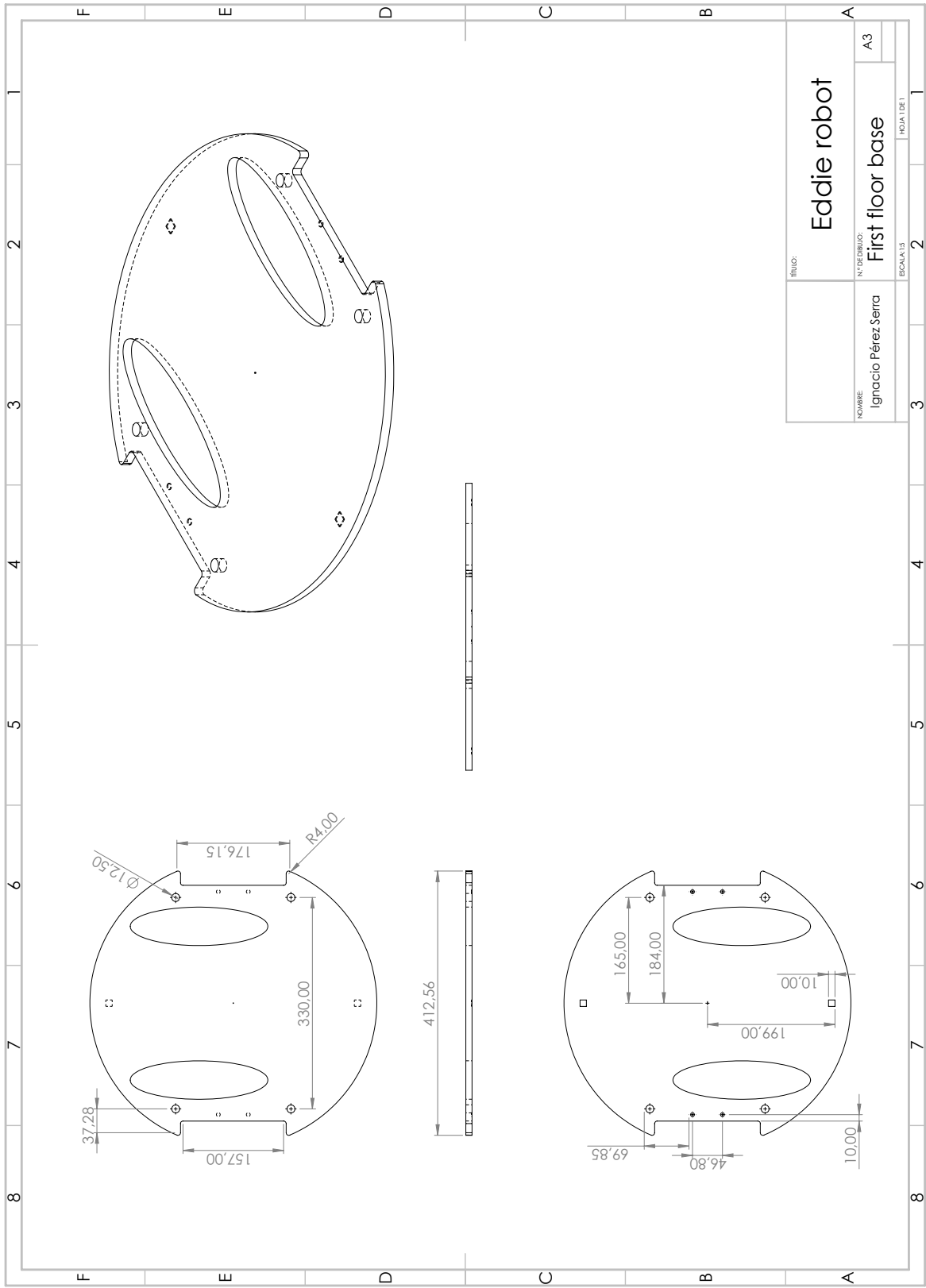


Figura A.3: First floor base

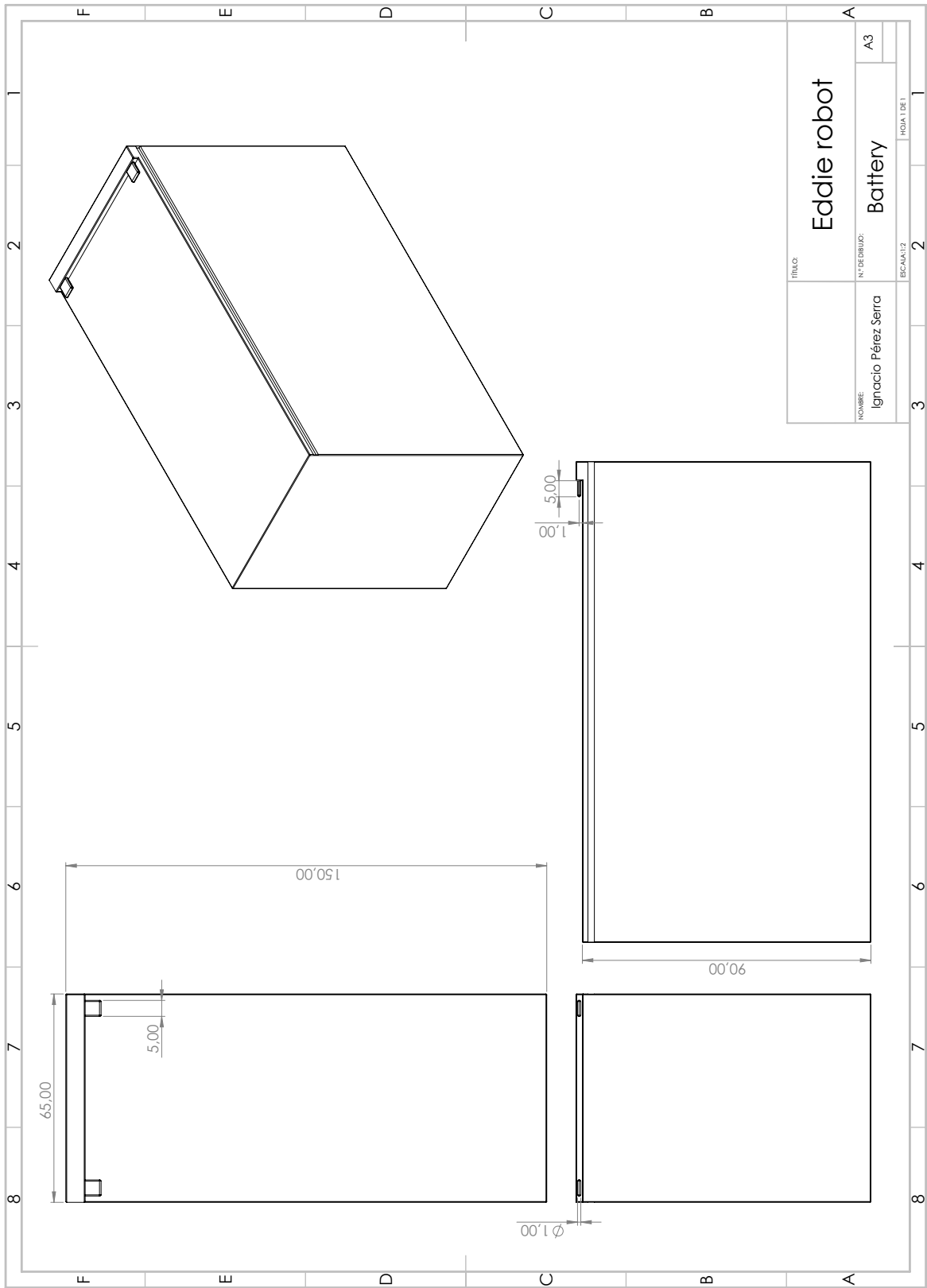


Figura A.4: Battery

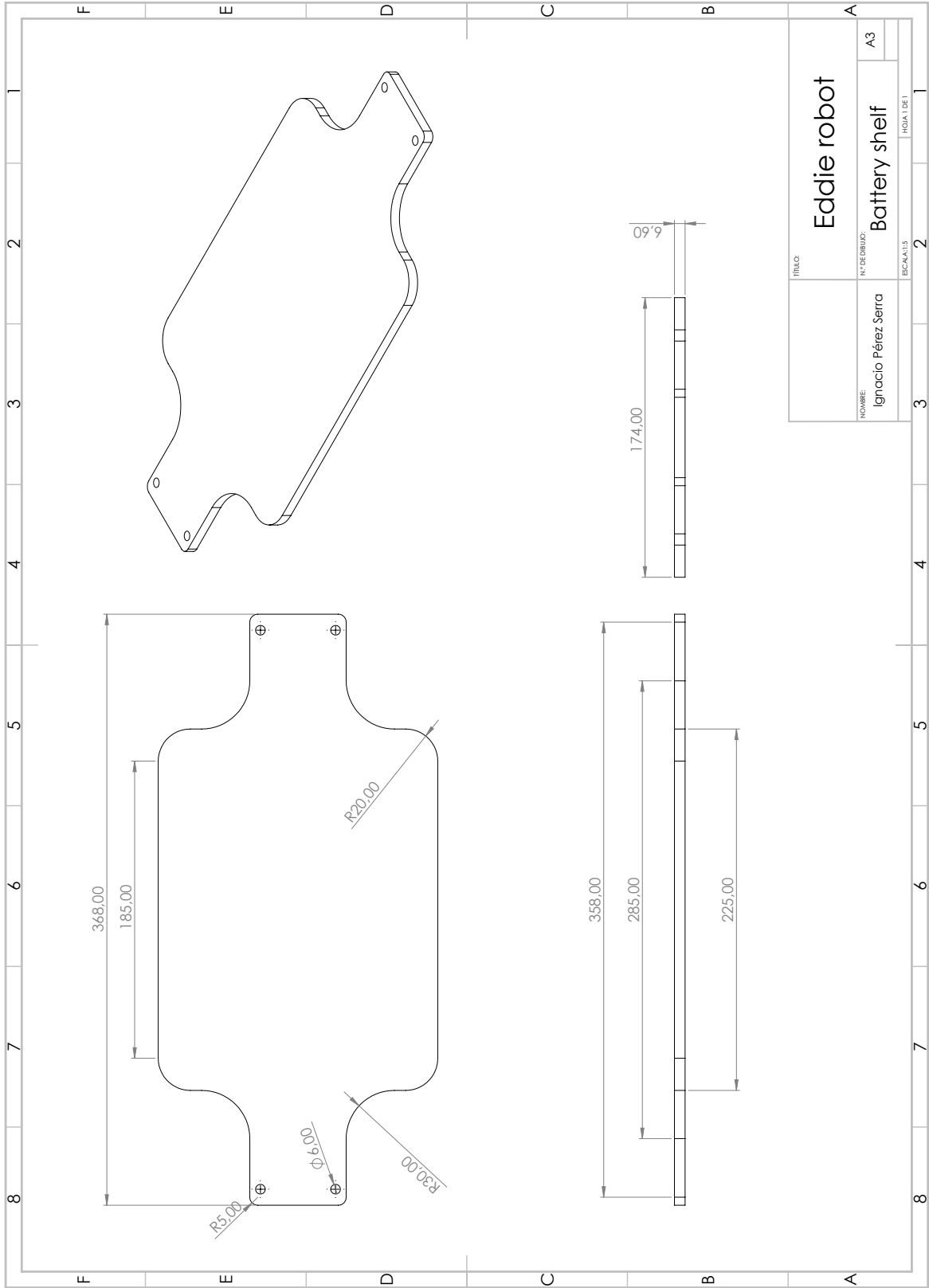


Figura A.5: Battery shelf

A.3. Caster wheel kit

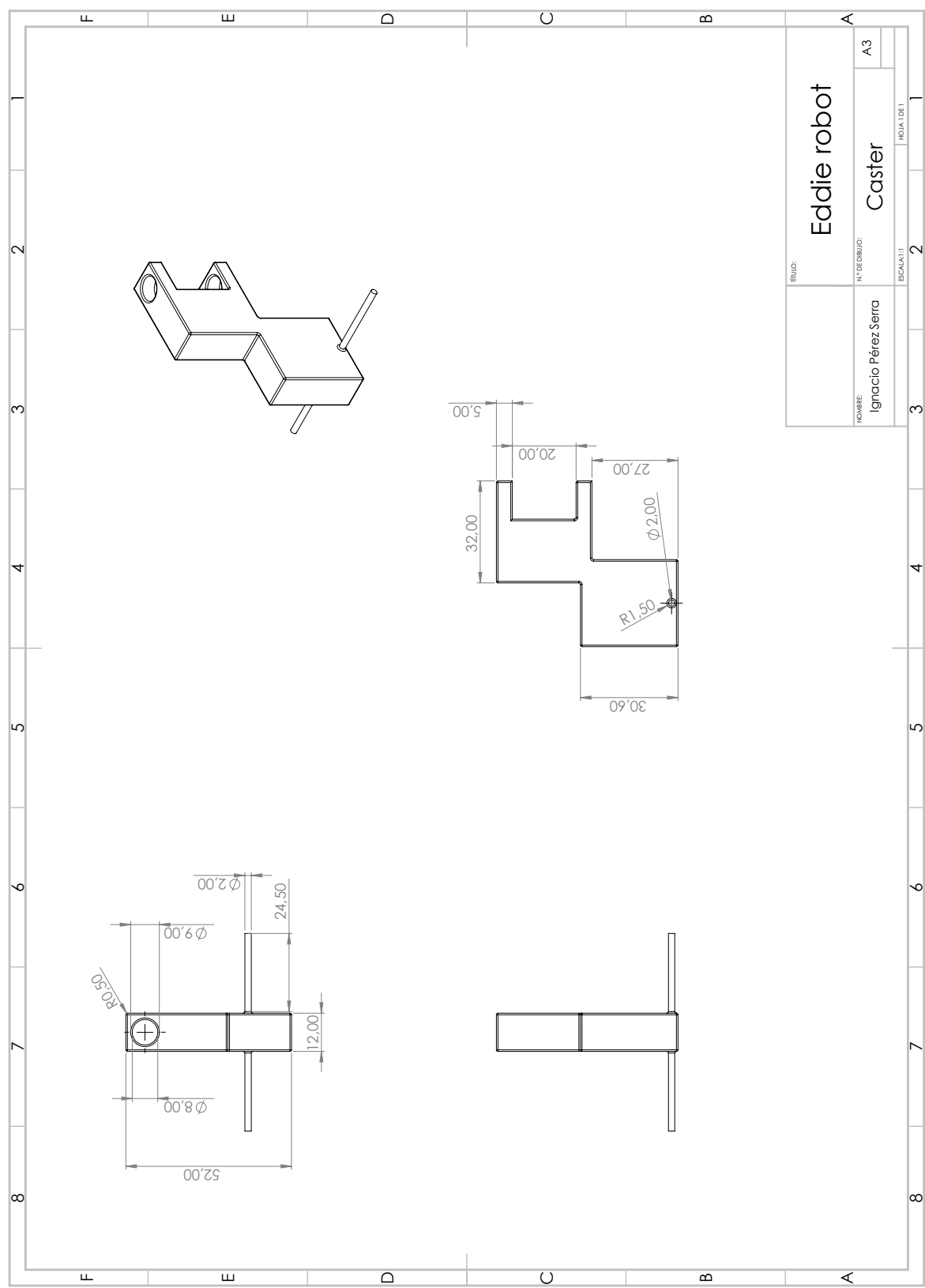


Figura A.6: Caster

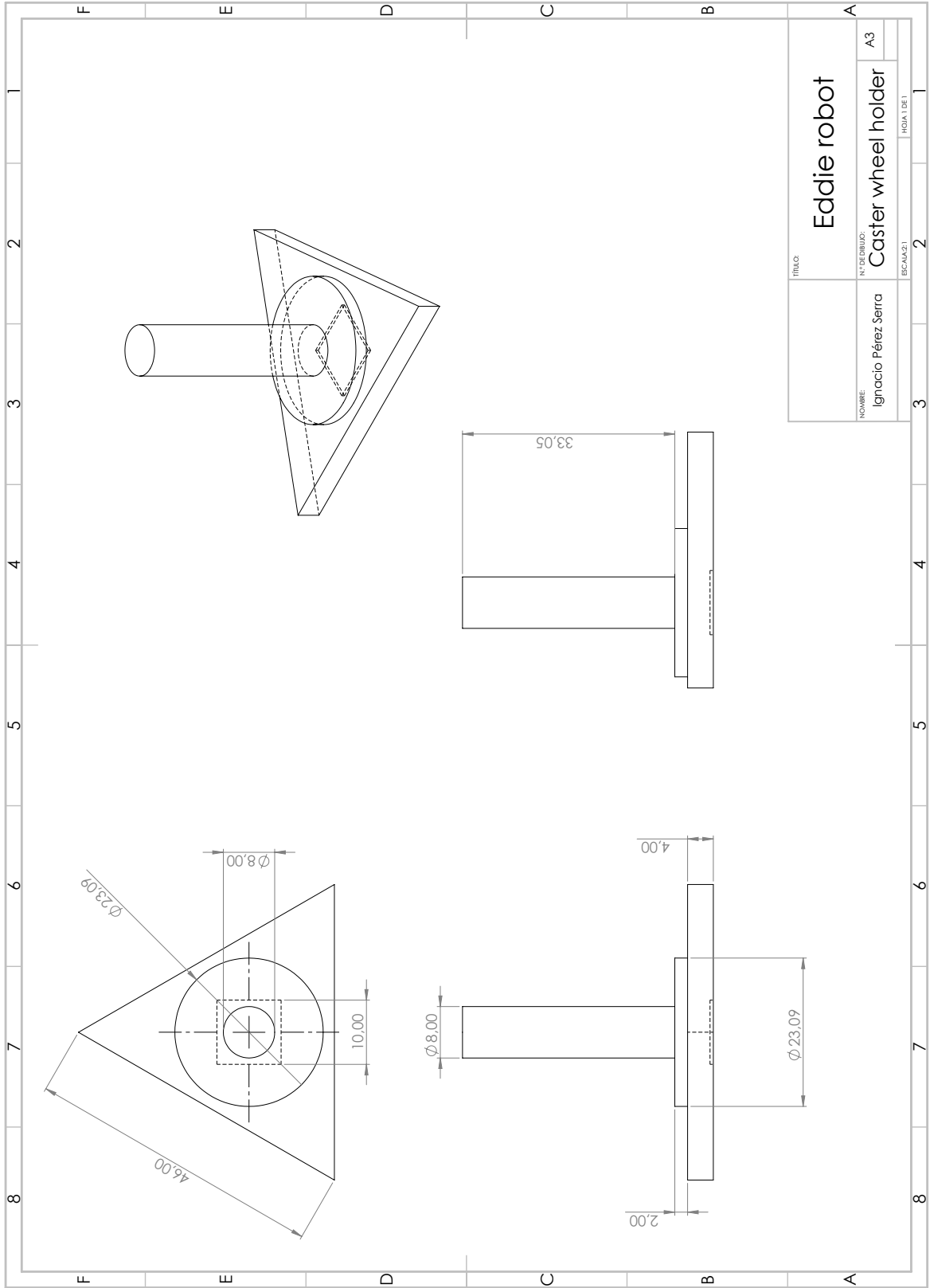


Figura A.7: Caster wheel holder

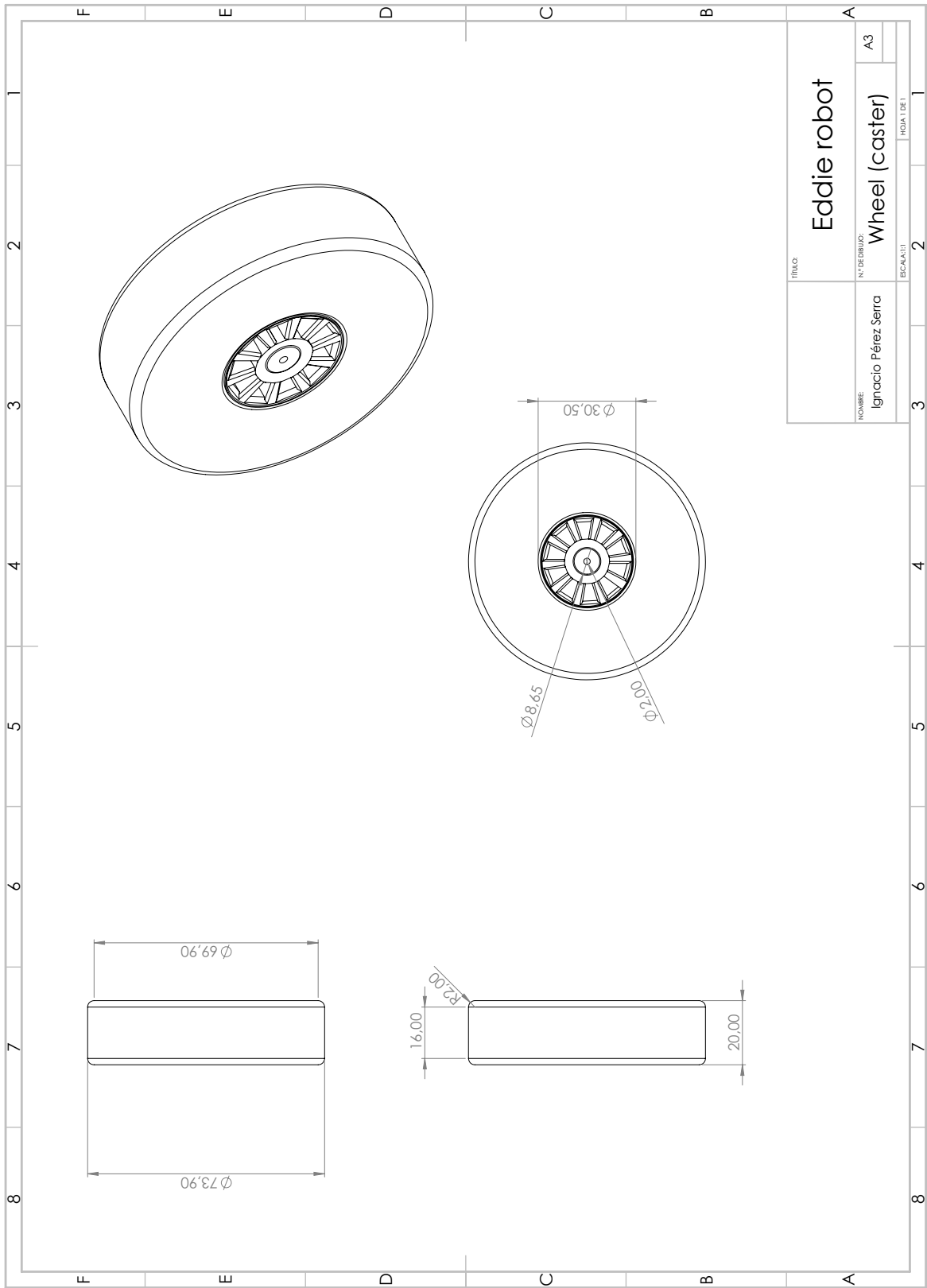


Figura A.8: wheel (caster)

A.4. Top base

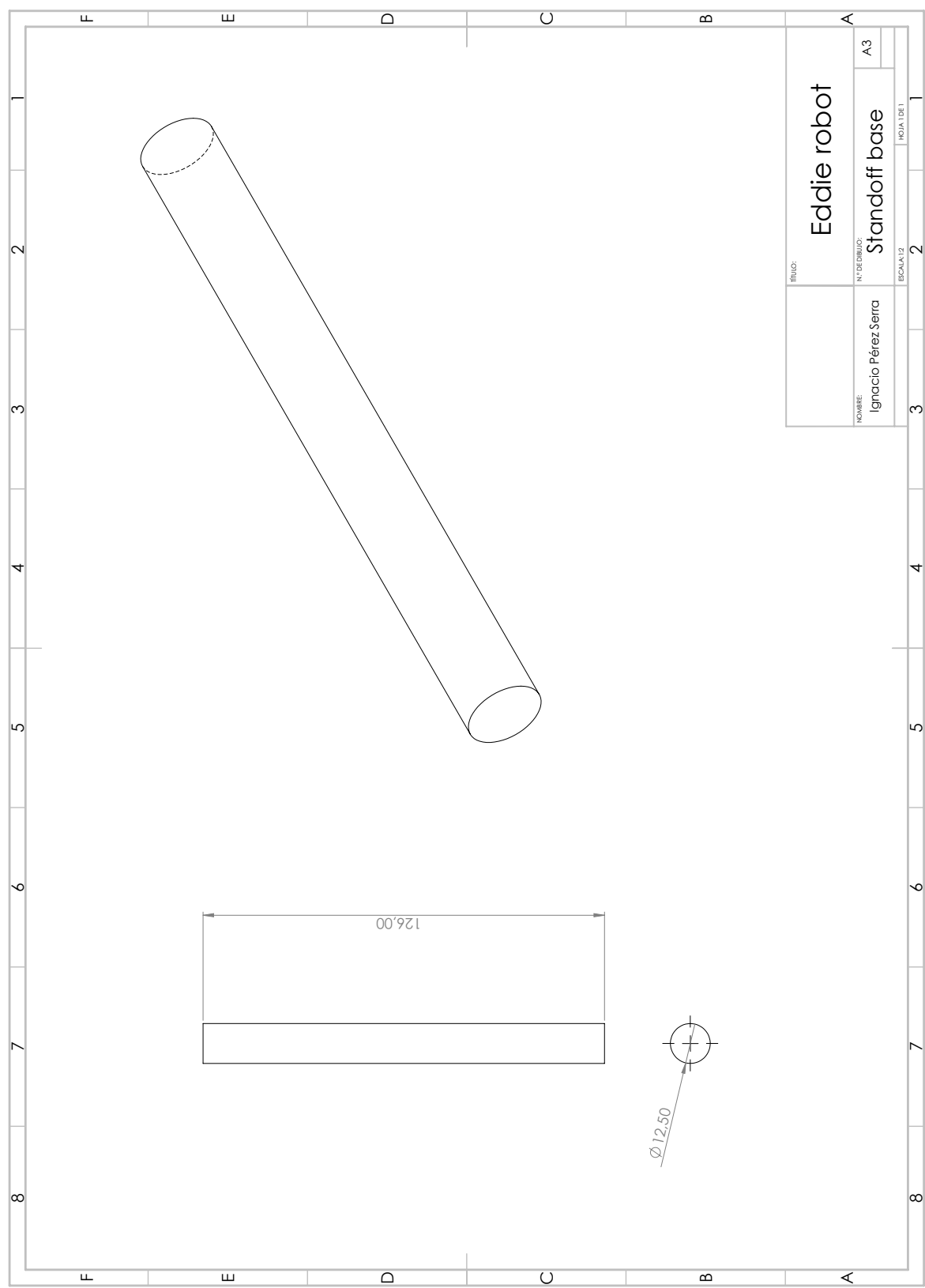


Figura A.9: Standoff base

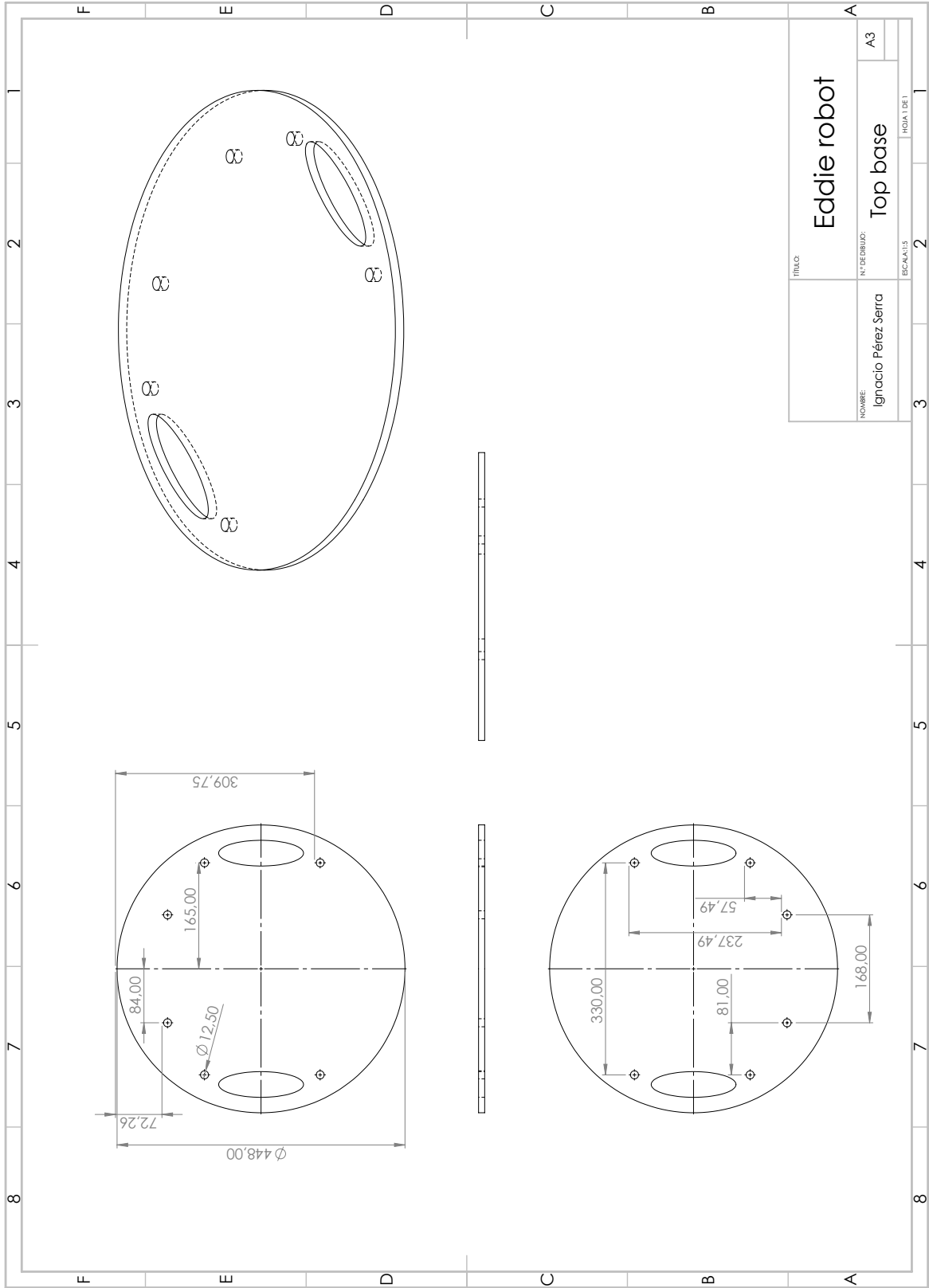


Figura A.10: Top base

A.5. Wheel + motor

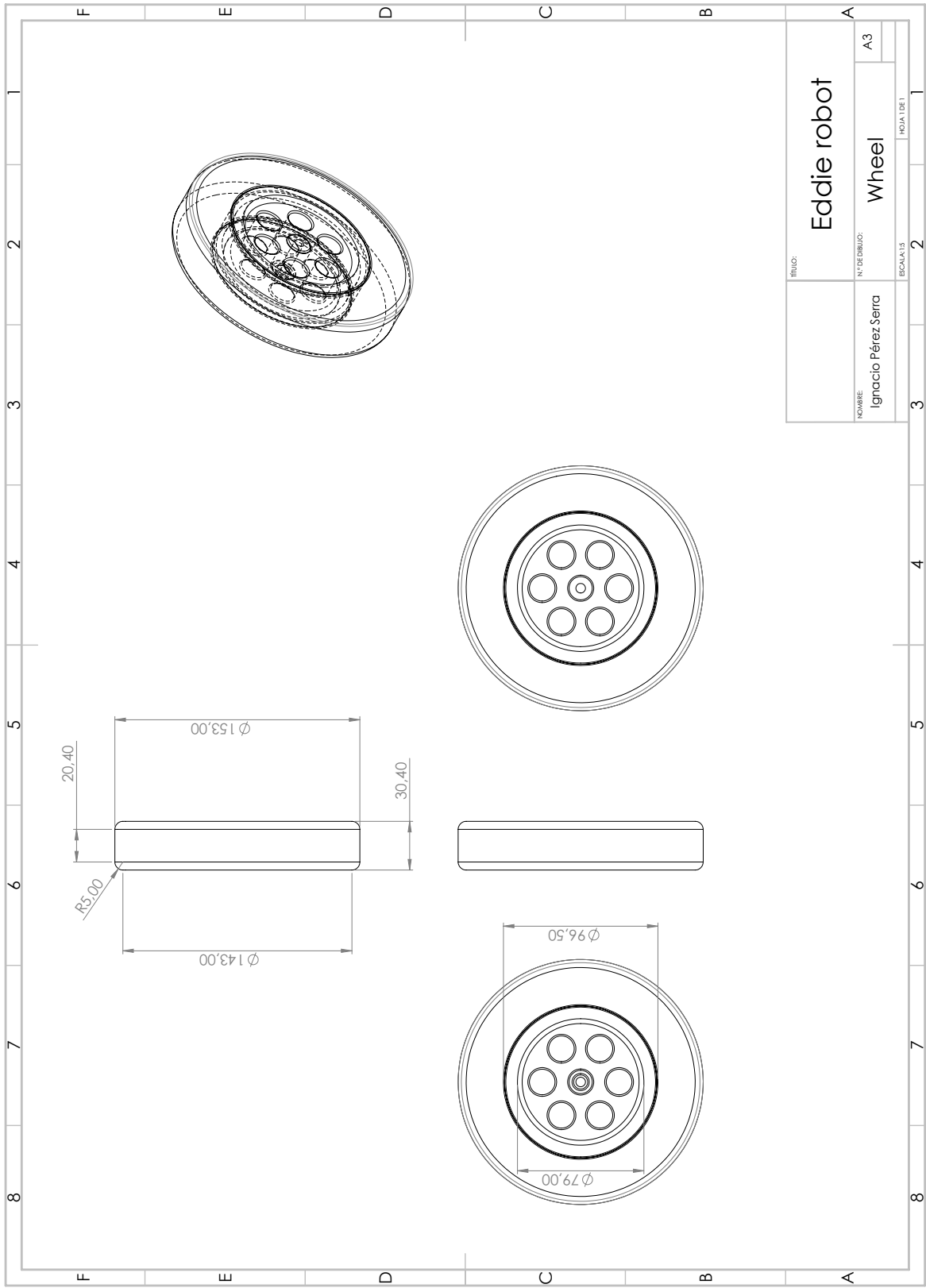


Figura A.11: Wheel

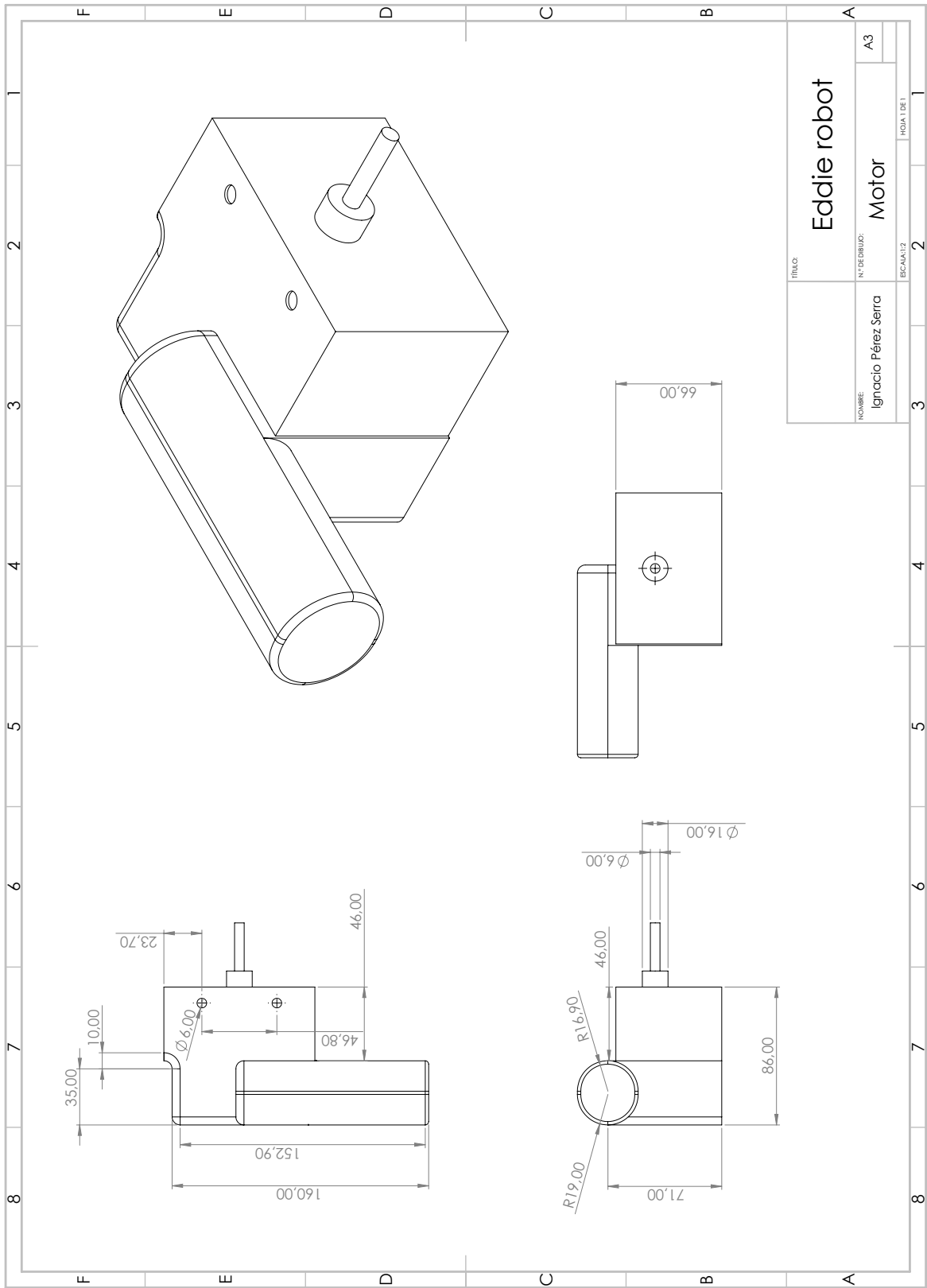


Figura A.12: Motor

A.6. Carcasa de la pantalla

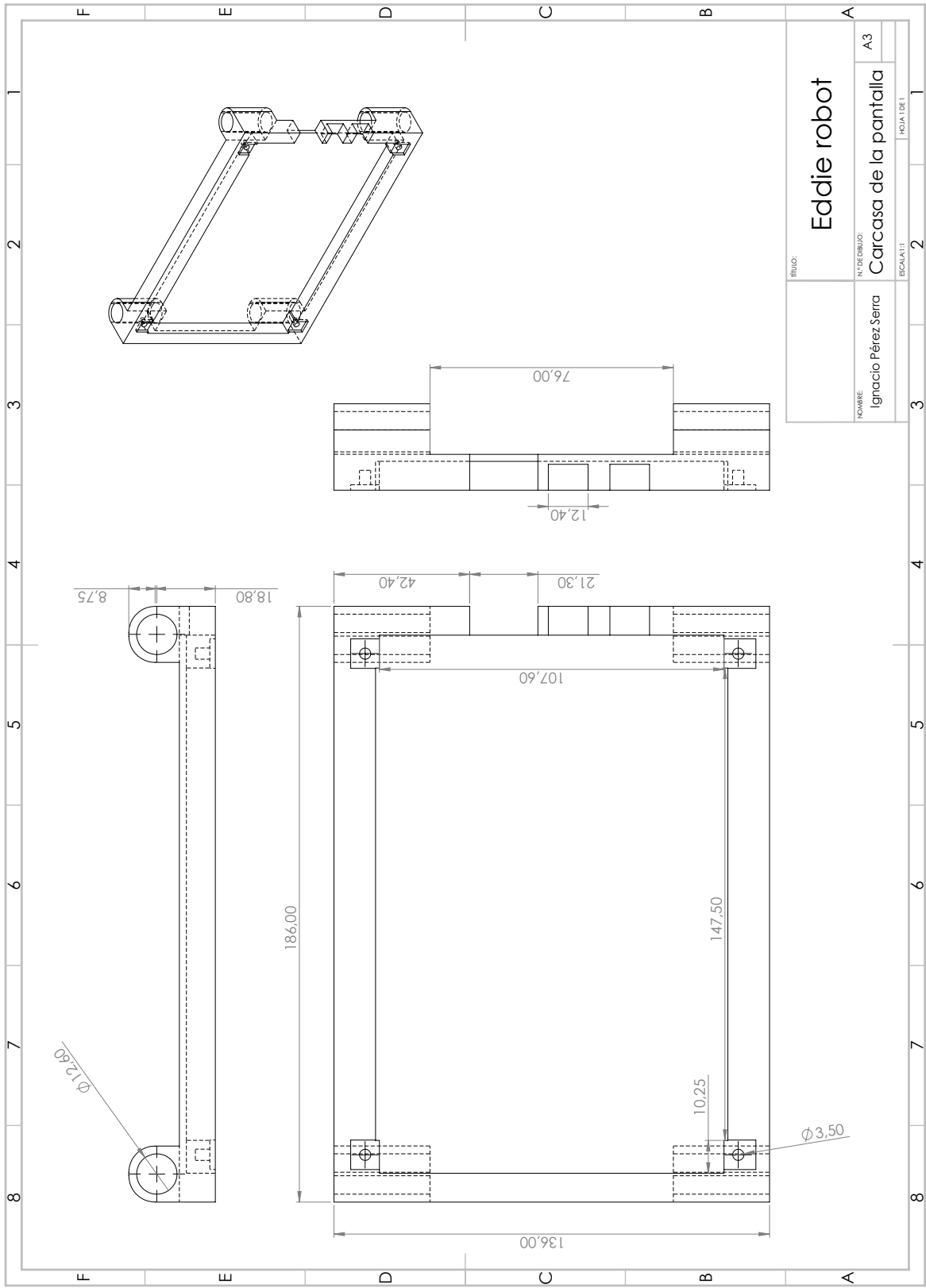


Figura A.13: Carcasa de la pantalla en 2D

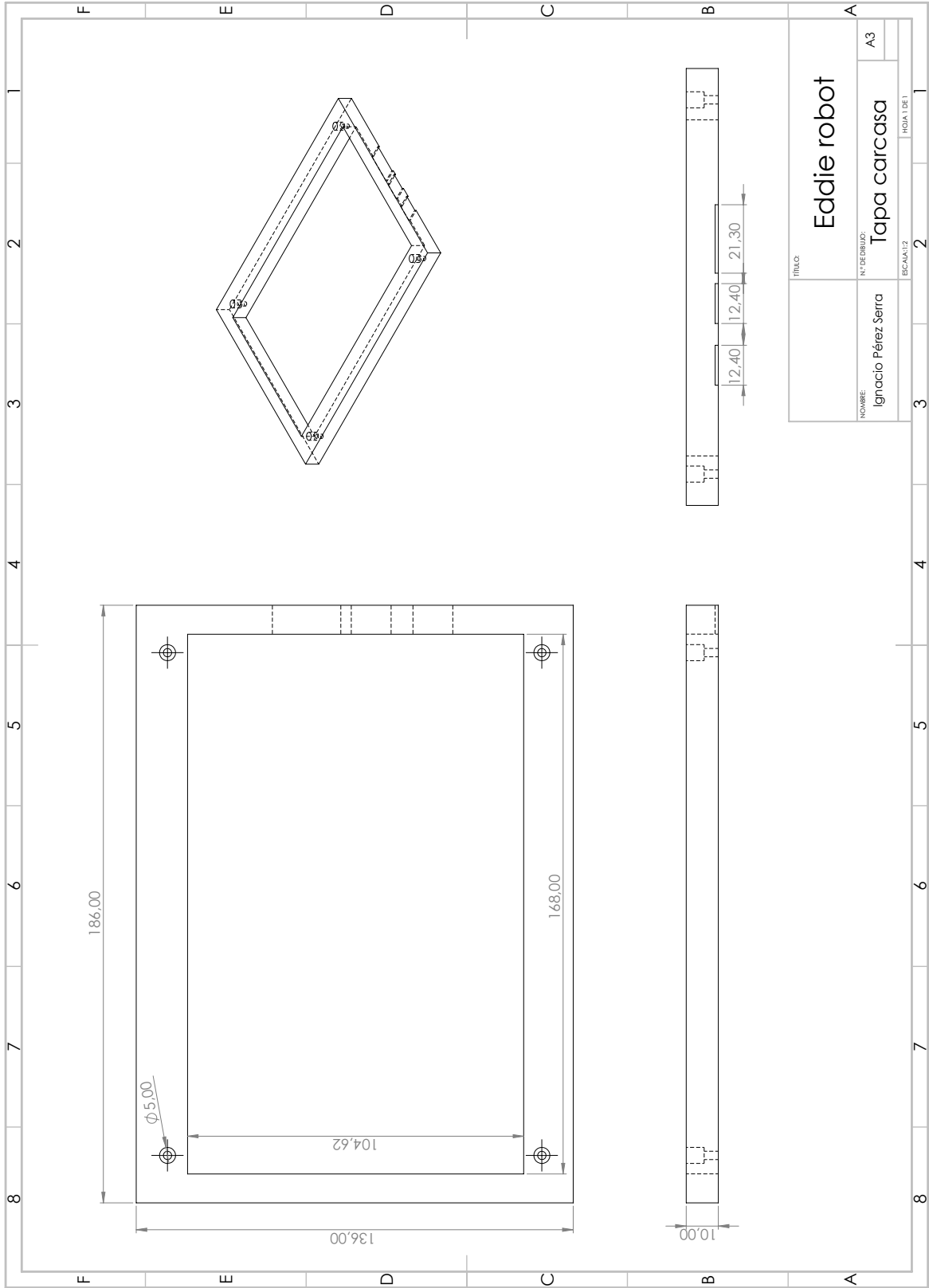


Figura A.14: Tapa de la carcasa en 2D

Apéndice B

Diseño 3D

B.1. Kinect plate assembly

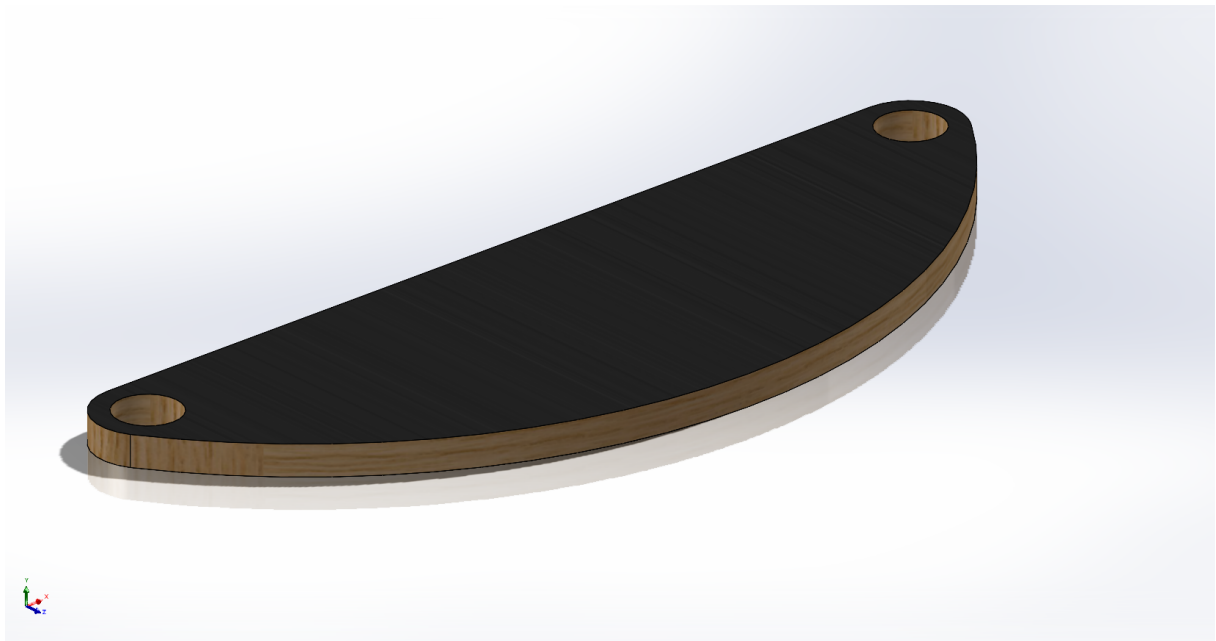


Figura B.1: Kinect deck

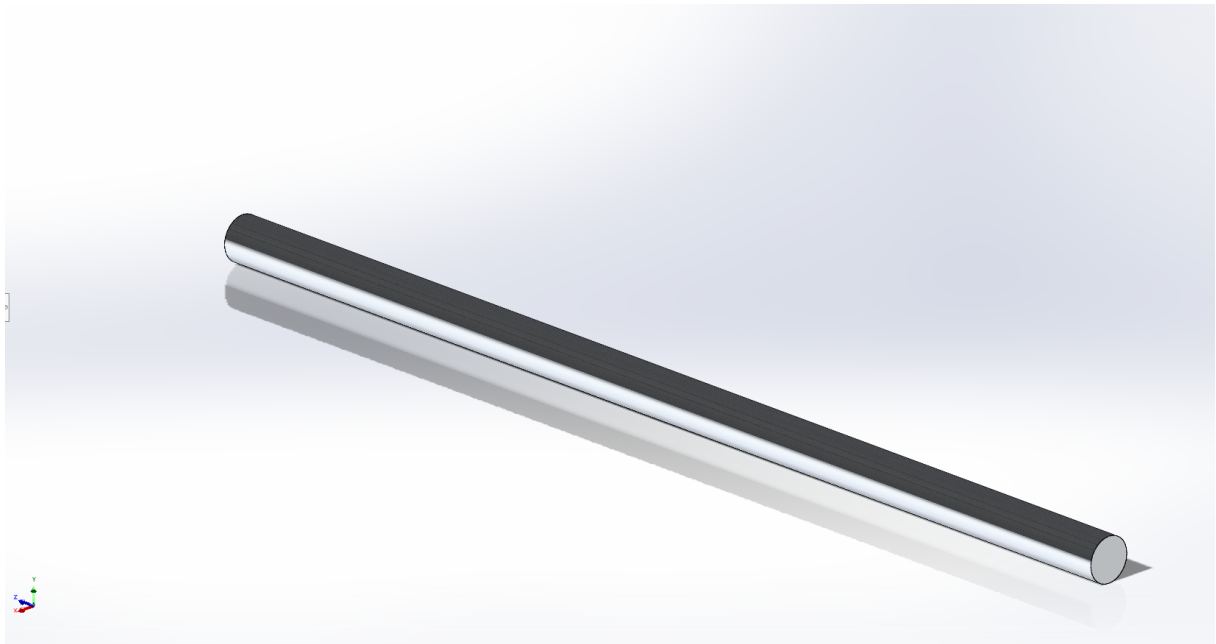


Figura B.2: Standoff kinect

B.2. Base

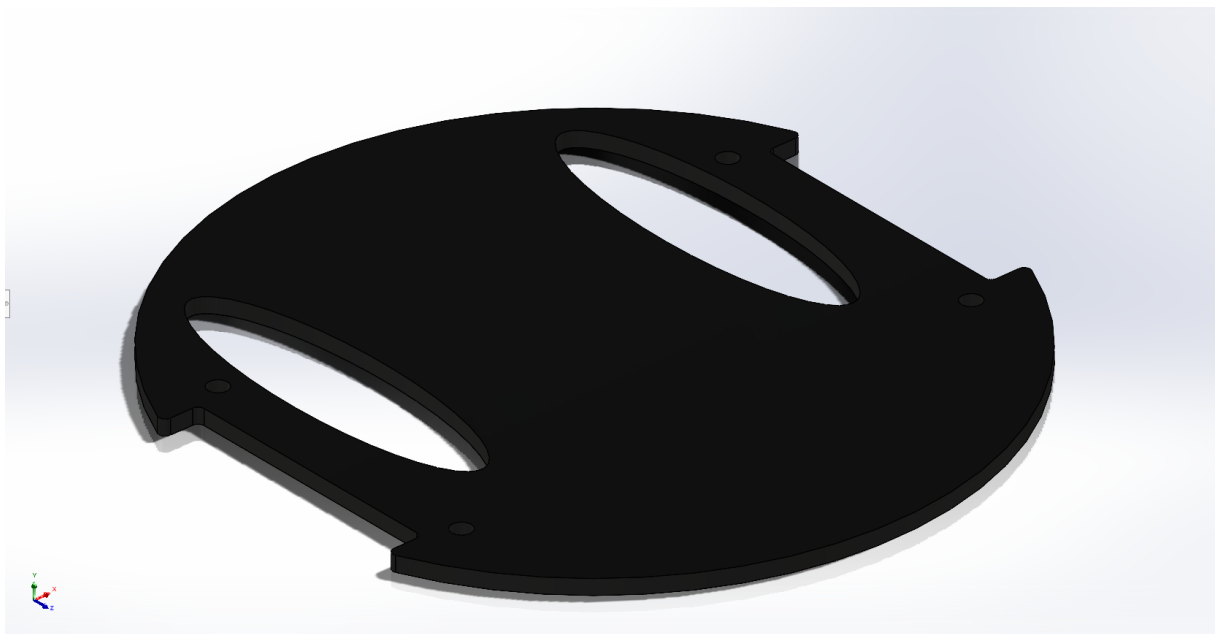


Figura B.3: First floor base

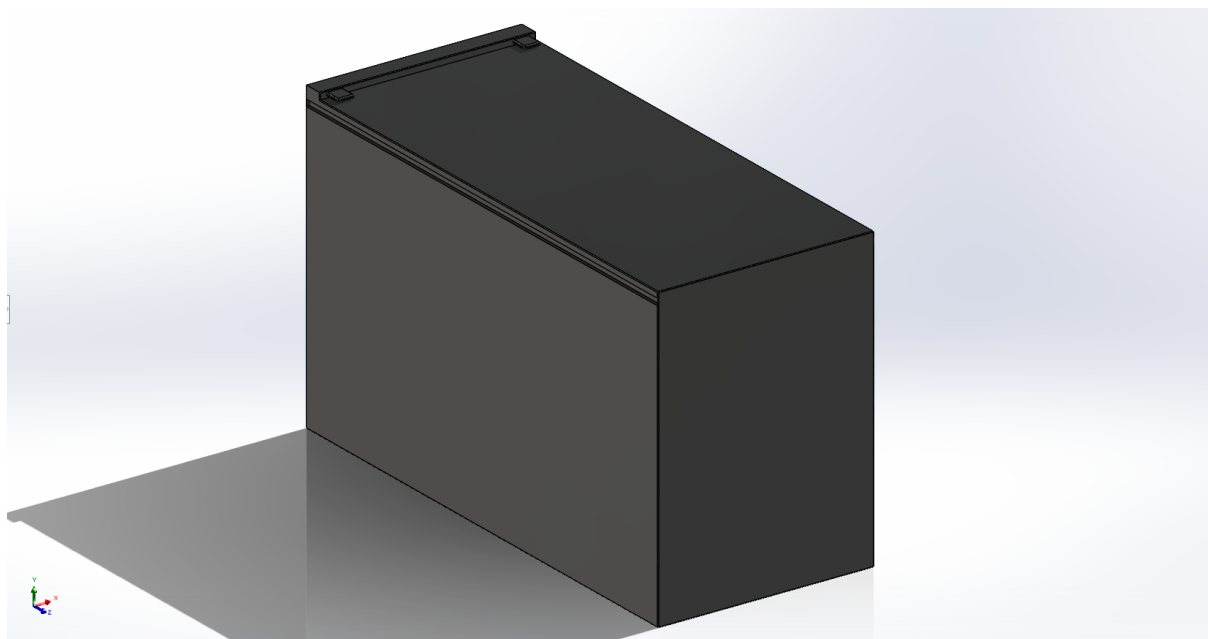


Figura B.4: Battery



Figura B.5: Battery shelf

B.3. Caster wheel kit

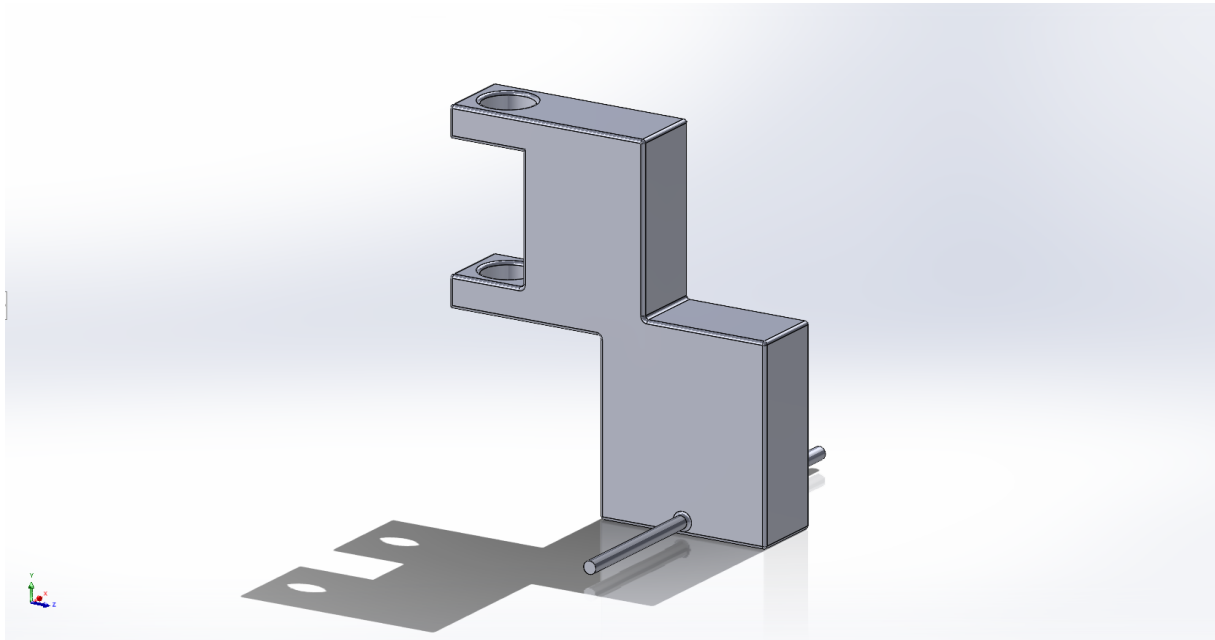


Figura B.6: Caster

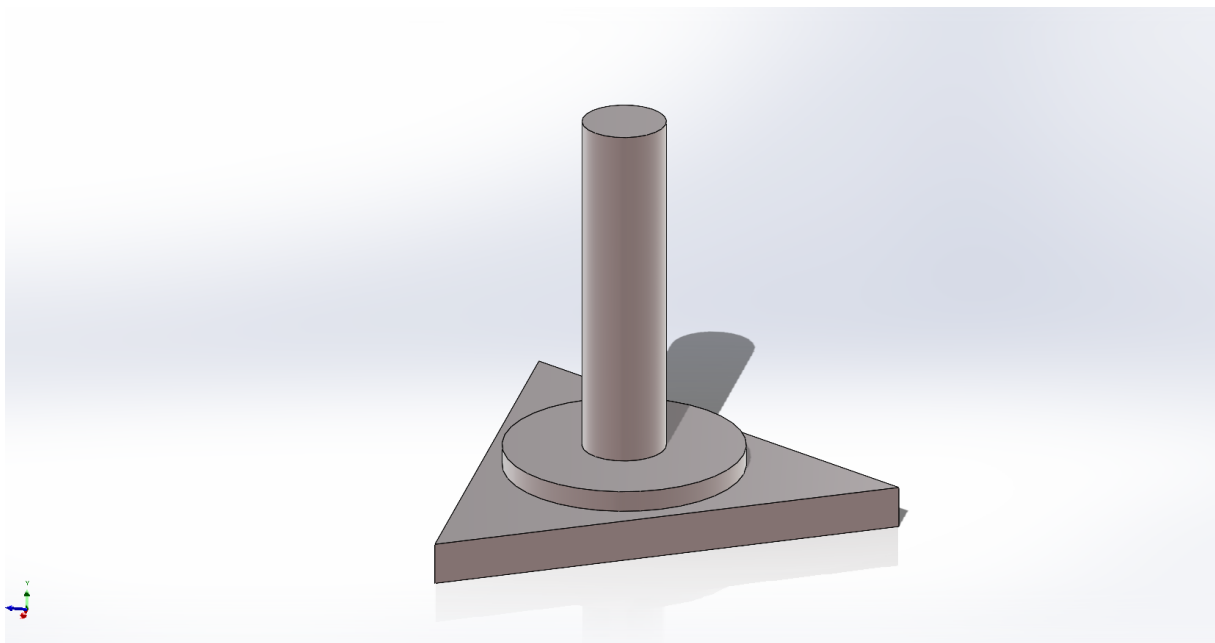


Figura B.7: Caster wheel holder

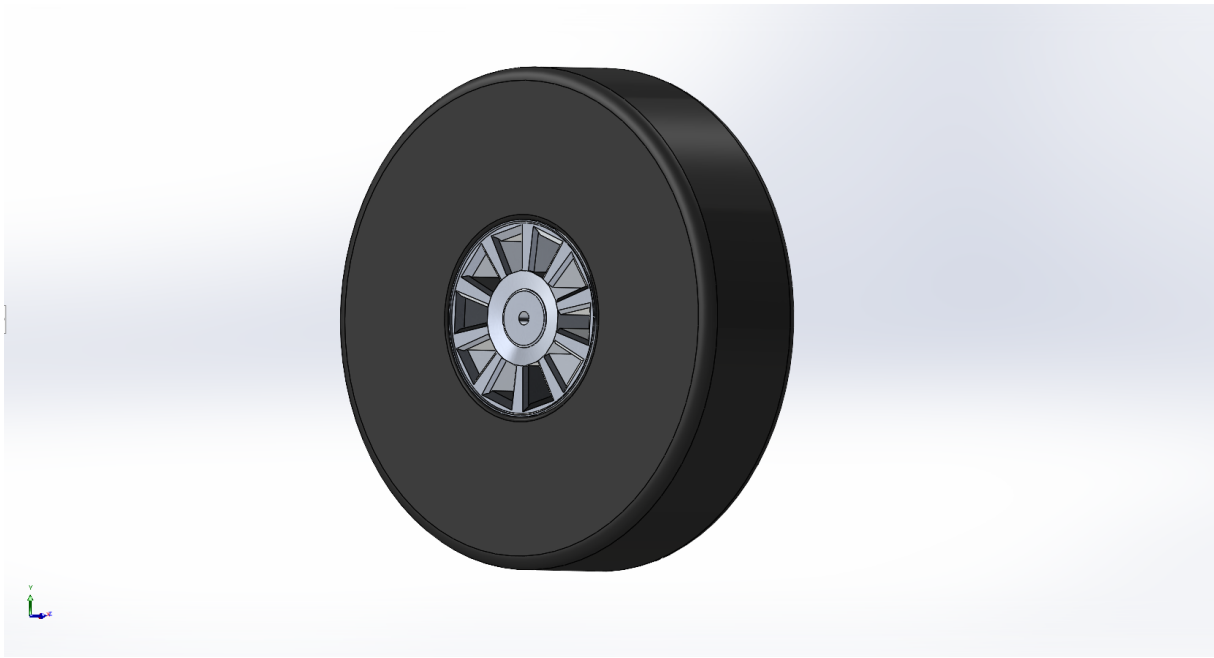


Figura B.8: wheel (caster)

B.4. Top base

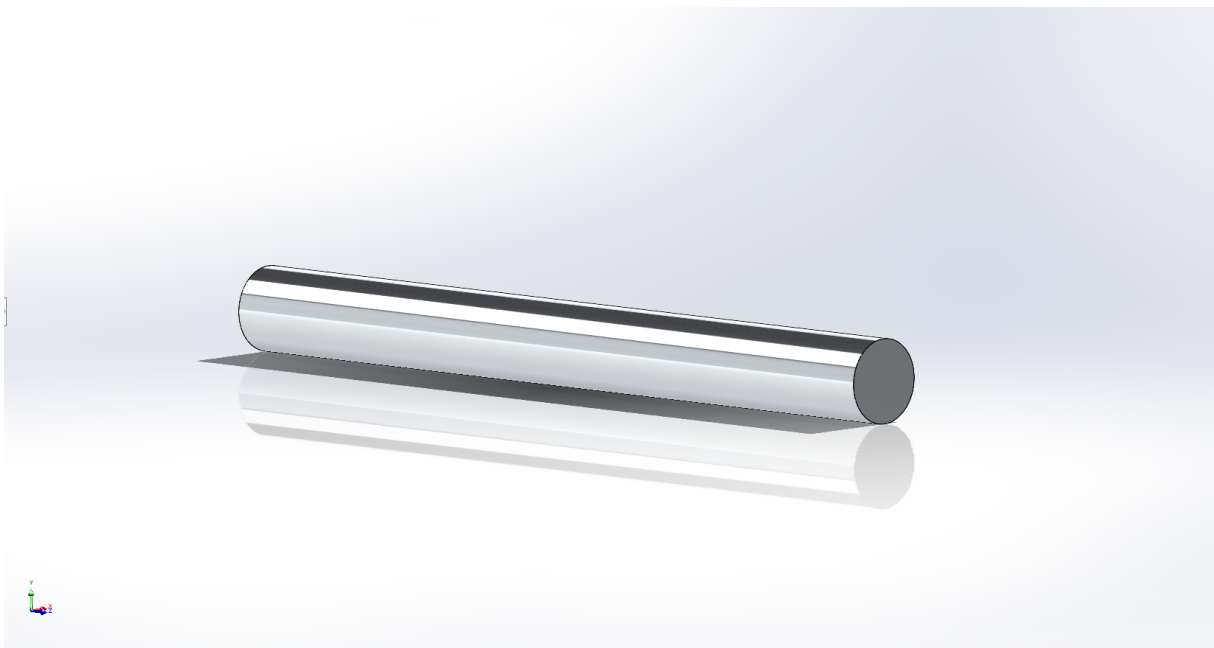


Figura B.9: Standoff base

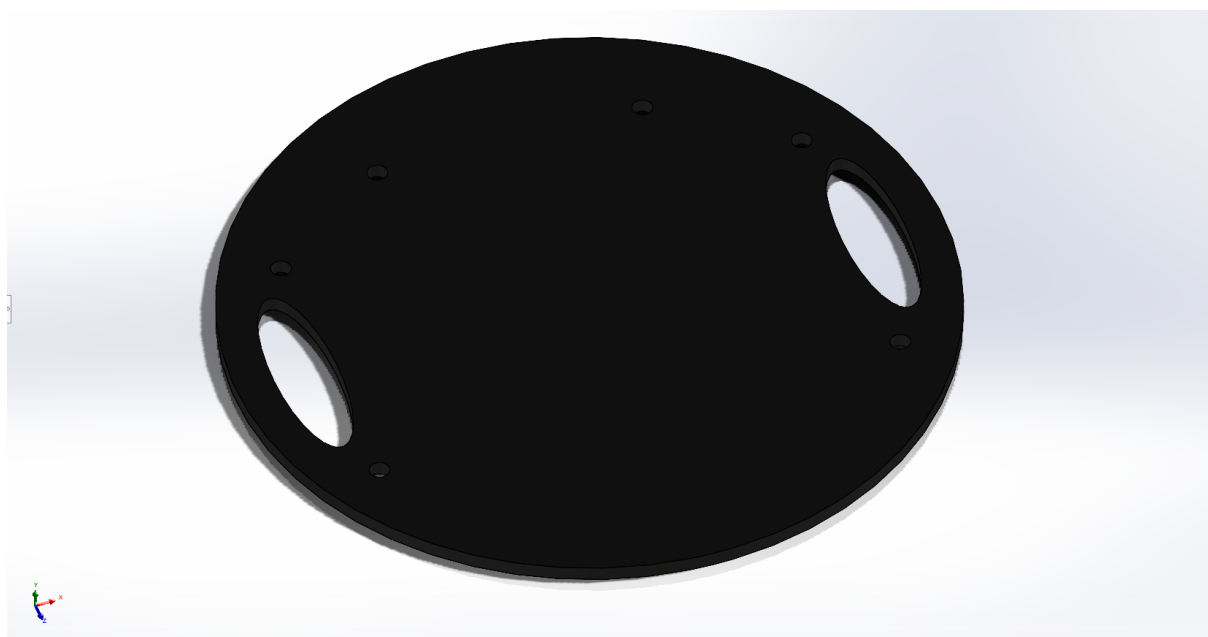


Figura B.10: Top base

B.5. Wheel + motor

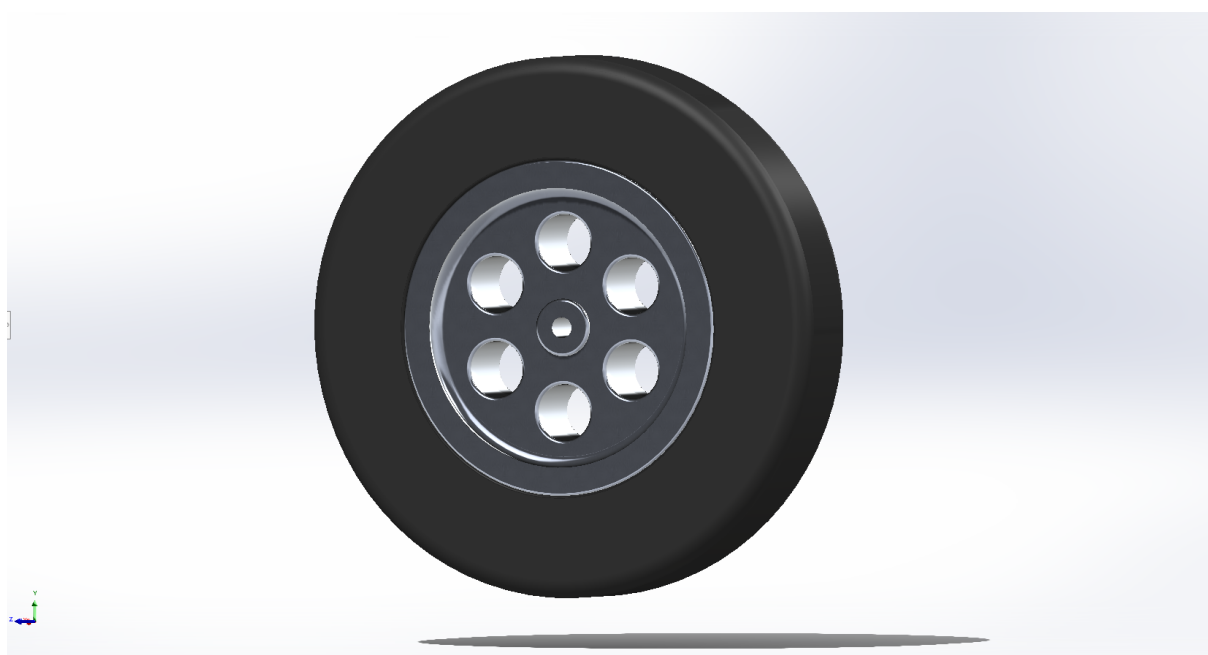


Figura B.11: Wheel - Vista delantera



Figura B.12: Wheel - Vista trasera

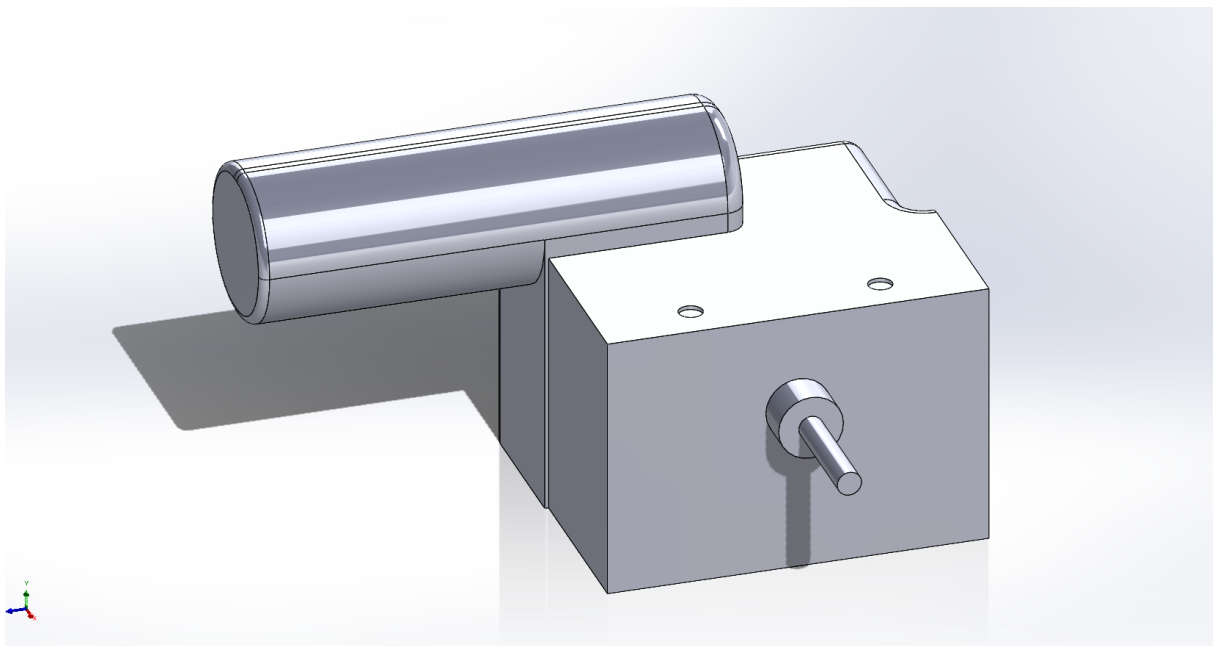


Figura B.13: Motor

Apêndice C

Código

C.1. URDF

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <robot name="eddiebot" xmlns:xacro="http://ros.org/wiki/xacro">
3   <xacro:include filename="$(find eddiebot_description)/urdf/common_properties.xacro
4     "/>
5   <xacro:include filename="$(find eddiebot_description)/urdf/eddiebot_gazebo.xacro"/>
6   <xacro:include filename="inertial_macros.xacro"/>
7
8   <link name="base_footprint"/>
9   <link name="base_link" >
10     <inertial>
11       <origin
12         xyz="6.40516959288934E-06 9.88983228679408E-08 0.0234596082010042"
13         rpy="0 0 0" />
14       <mass value="1.958" />
15       <inertia
16         ixx="0.01950247"
17         ixy="0.0"
18         ixz="-0.00020457"
19         iyy="0.02561711"
20         iyz="0.0"
21         izz="0.04157802" />
22     </inertial>
23     <visual>
24       <origin
25         xyz="0 0 0"
26         rpy="0 0 0" />
27       <geometry>
28         <mesh
29           filename="package://eddiebot_description/meshes/base_link.STL" />
30       </geometry>
31       <material name="dark"/>
32     </visual>
33     <collision>
34       <origin
35         xyz="0 0 0"
36         rpy="0 0 0" />
37       <geometry>
38         <box size="0.174 0.368 0.0762"/>
39     </geometry>
40   </collision>
```

```
40 </link>
41 <joint name="base_joint"
42   type="fixed">
43   <origin
44     xyz="-0.00188188452335199 -0.000513601526006199 0.06700000000013061"
45     rpy="0 0 0" />
46   <parent
47     link="base_footprint" />
48   <child
49     link="base_link" />
50   <axis
51     xyz="0 0 0" />
52 </joint>
53 <link name="base_top_link">
54   <inertial>
55     <origin
56       xyz="0.000307130677125385 1.10241172166653E-07 0.134982649934056"
57       rpy="0 0 0" />
58     <mass
59       value="1.724" />
60     <inertia
61       ixx="0.02119577"
62       ixy="0.0"
63       ixz="8.22e-06"
64       iyy="0.02278843"
65       iyz="0"
66       izz="0.04313801" />
67   </inertial>
68   <visual>
69     <origin
70       xyz="0 0 0"
71       rpy="0 0 0" />
72     <geometry>
73       <mesh
74         filename="package://eddiebot_description/meshes/base_top_link.STL" />
75     </geometry>
76     <material name="dark"/>
77   </visual>
78   <collision>
79     <origin
80       xyz="0.015 0 0.07"
81       rpy="0 0 0" />
82     <geometry>
83       <cylinder length="0.1356" radius="0.223"/>
84     </geometry>
85   </collision>
86 </link>
87 <joint name="base_top_joint"
88   type="fixed">
89   <origin
90     xyz="-0.00879392436820687 0 0.03665"
91     rpy="0 0 0" />
92   <parent
93     link="base_link" />
94   <child
95     link="base_top_link" />
96   <axis
97     xyz="0 0 0" />
98 </joint>
```

```

99 <link name="kinect_plate_link">
100   <inertial>
101     <origin
102       xyz="-0.00741329248753544 -0.00100077555843242 0.219296009980792"
103       rpy="0 0 0" />
104     <mass
105       value="0.272"/>
106     <inertia
107       ixx="0.00414787"
108       ixy="-1e-08"
109       ixz="0.00018088"
110       iyy="0.00286104"
111       iyz="2e-08"
112       izz="0.00139722" />
113   </inertial>
114   <visual>
115     <origin
116       xyz="0 0 0"
117       rpy="0 0 0" />
118     <geometry>
119       <mesh
120         filename="package://eddiebot_description/meshes/kinect_plate_link.STL" />
121     </geometry>
122     <material name="dark"/>
123   </visual>
124   <collision>
125     <origin
126       xyz="0 0 0.15"
127       rpy="0 0 0" />
128     <geometry>
129       <box size="0.065 0.188 0.310"/>
130     </geometry>
131   </collision>
132 </link>
133 <joint name="kinect_plate_joint"
134   type="fixed">
135   <origin
136     xyz="-0.145492267835786 0.000999999997519332 0.1406"
137     rpy="0 0 0" />
138   <parent
139     link="base_top_link" />
140   <child
141     link="kinect_plate_link" />
142   <axis
143     xyz="0 0 0" />
144 </joint>
145 <link name="motor_right_link">
146   <inertial>
147     <origin
148       xyz="-9.69481684120234E-05 6.09797385771327E-05 0.0406295446299301"
149       rpy="0 0 0" />
150     <mass
151       value="1.046" />
152     <inertia
153       ixx="0.00121916"
154       ixy="-0.00026888"
155       ixz="0.000388"
156       iyy="0.00203021"
157       iyz="0.00015076"

```

```

158     izz="0.00209411" />
159 </inertial>
160 <visual>
161   <origin
162     xyz="0 0 0"
163     rpy="0 0 0" />
164   <geometry>
165     <mesh
166       filename="package://eddiebot_description/meshes/motor_right_link.STL" />
167     </geometry>
168
169     <material name ="orange"/>
170   </visual>
171 </link>
172 <joint name="motor_right_joint"
173   type="fixed">
174   <origin
175     xyz="-0.02 -0.13727 -0.0320000000013062"
176     rpy="0 0 0" />
177   <parent
178     link="base_link" />
179   <child
180     link="motor_right_link" />
181   <axis
182     xyz="0 0 0" />
183 </joint>
184 <link name="wheel_right_link">
185   <inertial>
186     <origin
187       xyz="-1.66533453693773E-16 6.50521303491303E-18 0.0124886849718492"
188       rpy="0 0 0" />
189     <mass
190       value="0.388" />
191     <inertia
192       ixx="0.00065987"
193       ixy="0"
194       ixz="0"
195       iyy="0.00065987"
196       iyz="0"
197       izz="0.00126426" />
198   </inertial>
199   <visual>
200     <origin
201       xyz="0 0 0"
202       rpy="0 0 0" />
203     <geometry>
204       <mesh
205         filename="package://eddiebot_description/meshes/wheel_right_link.STL" />
206       </geometry>
207     <material name ="black"/>
208   </visual>
209   <collision>
210     <origin
211       xyz="0 0 0"
212       rpy="0 0 0" />
213     <geometry>
214       <cylinder radius="0.0765" length="0.0304"/>
215     </geometry>
216   </collision>

```

```

217 </link>
218 <joint name="wheel_right_joint"
219   type="continuous">
220   <origin
221     xyz="0.0132060756317932 -0.08677 0.0415"
222     rpy="1.5707963267949 1.5707963267949 3.14159265358979" />
223   <parent
224     link="motor_right_link" />
225   <child
226     link="wheel_right_link" />
227   <axis
228     xyz="0 0 1" />
229   <dynamics
230     damping="0"
231     friction="0" />
232 </joint>
233 <link name="motor_left_link">
234   <inertial>
235     <origin
236       xyz="-6.94825791193628E-06 -1.6822235779354E-05 0.0406295391896895"
237       rpy="0 0 0" />
238     <mass
239       value="1.046" />
240     <inertia
241       ixx="0.00121916"
242       ixy="0.00026888"
243       ixz="-0.000388"
244       iyy="0.00203021"
245       iyz="0.00015076"
246       izz="0.00209411" />
247     </inertial>
248     <visual>
249       <origin
250         xyz="0 0 0"
251         rpy="0 0 0" />
252       <geometry>
253         <mesh
254           filename="package://eddiebot_description/meshes/motor_left_link.STL" />
255         </geometry>
256         <geometry>
257           <cylinder radius="0.0765" length="0.0304"/>
258         </geometry>
259         <material name="orange"/>
260       </visual>
261 </link>
262 <joint name="motor_left_joint"
263   type="fixed">
264   <origin
265     xyz="-0.02009 0.13727 -0.032"
266     rpy="0 0 0" />
267   <parent
268     link="base_link" />
269   <child
270     link="motor_left_link" />
271   <axis
272     xyz="0 0 0" />
273 </joint>
274 <link name="wheel_left_link">
275   <inertial>

```

```

276     <origin
277         xyz="3.05311331771918E-16 -1.3834419720915E-16 0.0124886849718492"
278         rpy="0 0 0" />
279     <mass
280         value="0.388" />
281     <inertia
282         ixx="0.00065987"
283         ixy="0"
284         ixz="0"
285         iyy="0.00065987"
286         iyz="0"
287         izz="0.00126426" />
288 </inertial>
289 <visual>
290     <origin
291         xyz="0 0 0"
292         rpy="0 0 0" />
293     <geometry>
294         <mesh
295             filename="package://eddiebot_description/meshes/wheel_left_link.STL" />
296         </geometry>
297     <material
298         name="black">
299     </material>
300 </visual>
301 <collision>
302     <origin
303         xyz="0 0 0"
304         rpy="0 0 0" />
305     <geometry>
306         <cylinder radius="0.0765" length="0.0304"/>
307     </geometry>
308 </collision>
309 </link>
310 <joint name="wheel_left_joint"
311     type="continuous">
312     <origin
313         xyz="0.0132060756317932 0.08677 0.0415"
314         rpy="-1.5707963267949 -1.5707963267949 3.14159265358979" />
315     <parent
316         link="motor_left_link" />
317     <child
318         link="wheel_left_link" />
319     <axis
320         xyz="0 0 -1" />
321     <dynamics
322         damping="0"
323         friction="0" />
324 </joint>
325 <link name="battery_right_link">
326     <inertial>
327         <origin
328             xyz="-3.826065107051E-07 8.65721453095908E-05 0.0322499971323714"
329             rpy="0 0 0" />
330         <mass
331             value="2.5145" />
332         <inertia
333             ixx="0.00258294"
334             ixy="-7.81e-06"

```

```

335     ixz="0"
336     iyy="0.00560653"
337     iyz="0"
338     izz="0.00642109" />
339 </inertial>
340 <visual>
341   <origin
342     xyz="0 0 0"
343     rpy="0 0 0" />
344   <geometry>
345     <mesh
346       filename="package://eddiebot_description/meshes/battery_right_link.STL" />
347     </geometry>
348     <material name ="blue"/>
349   </visual>
350 </link>
351 <joint name="battery_right_joint"
352   type="fixed">
353   <origin
354     xyz="-0.006876 0.0454279216675194 -0.032"
355     rpy="0 0 0" />
356   <parent
357     link="base_link" />
358   <child
359     link="battery_right_link" />
360   <axis
361     xyz="0 0 0" />
362 </joint>
363 <link name="battery_left_link">
364   <inertial>
365     <origin
366       xyz="-3.82608826103407E-07 -2.41552581282251E-06 0.0325000028300059"
367       rpy="0 0 0" />
368     <mass
369       value="2.5145" />
370     <inertia
371       ixx="0.00258294"
372       ixy="7.81e-06"
373       ixz="0.0"
374       iyy="0.00560653"
375       iyz="0.0"
376       izz="0.00642109" />
377   </inertial>
378   <visual>
379     <origin
380       xyz="0 0 0"
381       rpy="0 0 0" />
382     <geometry>
383       <mesh
384         filename="package://eddiebot_description/meshes/battery_left_link.STL" />
385       </geometry>
386       <material name ="blue"/>
387     </visual>
388   </link>
389 <joint name="battery_left_joint"
390   type="fixed">
391   <origin
392     xyz="-0.006576000000000001 -0.0452620783324806 -0.032"
393     rpy="0 0 0" />

```

```

394   <parent
395     link="base_link" />
396   <child
397     link="battery_left_link" />
398   <axis
399     xyz="0 0 0" />
400 </joint>
401 <link name="base_rear_caster_link">
402   <inertial>
403     <origin
404       xyz="2.46370895906622E-10 0.0243123370668157 0.0165207797650175"
405       rpy="0 0 0" />
406     <mass
407       value="0.076" />
408     <inertia
409       ixx="0.00003100"
410       ixy="0.00000000"
411       ixz="0.00000000"
412       iyy="0.00002130"
413       iyz="-0.00000933"
414       izz="0.00001206" />
415   </inertial>
416   <visual>
417     <origin
418       xyz="0 0 0"
419       rpy="0 0 0" />
420     <geometry>
421       <mesh
422         filename="package://eddiebot_description/meshes/base_rear_caster_link.STL"
423       />
424     </geometry>
425     <material name="orange"/>
426   </visual>
427 </link>
428 <joint name="base_rear_caster_joint"
429   type="continuous">
430   <origin
431     xyz="-0.200793924368207 0 0.0114749999986939"
432     rpy="3.14159265358979 0 -1.5707963267949" />
433   <parent
434     link="base_link" />
435   <child
436     link="base_rear_caster_link" />
437   <axis
438     xyz="0 0 -1" />
439 </joint>
440 <link name="base_front_caster_link">
441   <inertial>
442     <origin
443       xyz="-0.00074358459420179 -0.024300963323452 0.016520779818389"
444       rpy="0 0 0" />
445     <mass
446       value="0.076" />
447     <inertia
448       ixx="0.00003099"
449       ixy="-0.00000030"
450       ixz="0.00000029"
451       iyy="0.00002131"
452       iyz="0.00000932"

```



```

452     izz="0.00001206" />
453 </inertial>
454 <visual>
455   <origin
456     xyz="0 0 0"
457     rpy="0 0 0" />
458   <geometry>
459     <mesh
460       filename="package://eddiebot_description/meshes/base_front_caster_link.STL"
461     />
462   </geometry>
463   <material name="orange"/>
464 </visual>
465 </link>
466 <joint name="base_front_caster_joint"
467   type="continuous">
468   <origin
469     xyz="0.187206075631793 0 0.0114749999986939"
470     rpy="-3.14159265358979 0 -1.60138574717821" />
471   <parent
472     link="base_link" />
473   <child
474     link="base_front_caster_link" />
475   <axis
476     xyz="0 0 -1" />
477 </joint>
478 <link name="wheels_rear_caster_link">
479   <inertial>
480     <origin
481       xyz="-9.07934302185609E-08 -1.14285528513003E-07 1.36766179853476E-07"
482       rpy="0 0 0" />
483     <mass
484       value="0.038" />
485     <inertia
486       ixx="0.00003227"
487       ixy="0.00000000"
488       ixz="0.00000000"
489       iyy="0.00003227"
490       iyz="0.00000000"
491       izz="0.00002691" />
492   </inertial>
493   <visual>
494     <origin
495       xyz="0 0 0"
496       rpy="0 0 0" />
497     <geometry>
498       <mesh
499         filename="package://eddiebot_description/meshes/wheels_rear_caster_link.STL"
500       />
501     </geometry>
502     <material name="black"/>
503   </visual>
504   <collision>
505     <origin
506       xyz="0 0 0"
507       rpy="0 0 0" />
508     <geometry>
509       <cylinder radius="0.03696" length="0.06297"/>
510     </geometry>

```

```

509     </collision>
510 </link>
511 <joint name="wheels_rear_caster_joint"
512     type="continuous">
513     <origin
514         xyz="0 0.032344678103671 0.041525"
515         rpy="-1.5707963267949 0 1.5707963267949" />
516     <parent
517         link="base_rear_caster_link" />
518     <child
519         link="wheels_rear_caster_link" />
520     <axis
521         xyz="0 0 1" />
522     <dynamics
523         damping="0"
524         friction="0" />
525 </joint>
526 <link name="wheels_front_caster_link">
527     <inertial>
528         <origin
529             xyz="-9.07934300520274E-08 -1.14285528568514E-07 1.36766177948749E-07"
530             rpy="0 0 0" />
531         <mass
532             value="0.038" />
533         <inertia
534             ixx="0.00003227"
535             ixy="0.00000000"
536             ixz="0.00000000"
537             iyy="0.00003226"
538             iyz="0.00000000"
539             izz="0.00002691" />
540     </inertial>
541     <visual>
542         <origin
543             xyz="0 0 0"
544             rpy="0 0 0" />
545         <geometry>
546             <mesh
547                 filename="package://eddiebot_description/meshes/wheels_rear_caster_link.STL"
548             />
549             </geometry>
550             <material name="black"/>
551     </visual>
552     <collision>
553         <origin
554             xyz="0 0 0"
555             rpy="0 0 0" />
556         <geometry>
557             <cylinder radius="0.03696" length="0.06297"/>
558         </geometry>
559     </collision>
560 </link>
561 <joint name="wheels_front_caster_joint"
562     type="continuous">
563     <origin
564         xyz="-0.001 -0.0323446781036711 0.0415249999999999"
565         rpy="-1.5707963267949 0 1.54020690641159" />
566     <parent
567         link="base_front_caster_link" />

```

```

567   <child
568     link="wheels_front_caster_link"/>
569     <axis
570       xyz="0 0 1" />
571     <dynamics
572       damping="0"
573       friction="0" />
574   </joint>
575   <link name="screen_case_link">
576     <inertial>
577       <origin
578         xyz="0.00999443346163434 -0.0018961166873752 0.0682350661912288"
579         rpy="0 0 0" />
580       <mass
581         value="0.403" />
582       <inertia
583         ixx="0.00003227"
584         ixy="0.0"
585         ixz="0.0"
586         iyy="0.00003227"
587         iyz="0.0"
588         izz="0.00002691" />
589     </inertial>
590     <visual>
591       <origin
592         xyz="0 0 0"
593         rpy="0 0 0" />
594       <geometry>
595         <mesh
596           filename="package://eddiebot_description/meshes/screen_case_link.STL" />
597         </geometry>
598       <material name="red"/>
599     </visual>
600     <collision>
601       <origin
602         xyz="0 0 0"
603         rpy="0 0 0" />
604       <geometry>
605         <mesh
606           filename="package://eddiebot_description/meshes/screen_case_link.STL" />
607         </geometry>
608       </collision>
609     </link>
610   <joint name="screen_case_joint" type="fixed">
611     <origin
612       xyz="0.00475 -0.000999999997519367 0.148601341910205"
613       rpy="0 0 0" />
614     <parent
615       link="kinect_plate_link" />
616     <child
617       link="screen_case_link" />
618     <axis
619       xyz="0 1 0" />
620   </joint>
621
622   <link name="screen_link">
623     <visual>
624       <origin
625         xyz="0 0 0"

```

```

626     rpy="0 0 0" />
627     <geometry>
628     <mesh
629         filename="package://eddiebot_description/meshes/screen_link.STL" />
630     </geometry>
631     <material name="black"/>
632 </visual>
633 </link>
634 <joint name="screen_joint"
635     type="fixed">
636     <origin
637         xyz="0.0045 0 0.0154"
638         rpy="3.1416 0 1.5708" />
639     <parent
640         link="screen_case_link" />
641     <child
642         link="screen_link" />
643     <axis
644         xyz="0 0 1" />
645 </joint>
646 <!-- IMU -->
647 <joint name="imu_joint" type="fixed">
648     <parent link="base_link"/>
649     <child link="imu_link"/>
650     <origin xyz="-0.032 0 0.068" rpy="0 0 0"/>
651 </joint>
652 <link name="imu_link"/>
653
654 <!-- LIDAR -->
655 <joint name="laser_joint" type="fixed">
656     <parent link="base_top_link"/>
657     <child link="laser_frame"/>
658     <origin xyz="0 0 0.125" rpy="0 3.14 0"/>
659 </joint>
660
661 <link name="laser_frame">
662     <visual>
663         <origin xyz="0 0 0" rpy="-1.57 0 0"/>
664         <geometry>
665             <mesh
666                 filename="package://eddiebot_description/meshes/rplidar.stl" scale="0.0005
667                 0.0005 0.0005"/>
668             </geometry>
669             <material name="dark"/>
670         </visual>
671         <collision>
672             <geometry>
673                 <cylinder radius="0.05" length="0.04"/>
674             </geometry>
675         </collision>
676         <xacro:inertial_cylinder mass="0.1" length="0.04" radius="0.05">
677             <origin xyz="0 0 0" rpy="0 0 0"/>
678         </xacro:inertial_cylinder>
679 </link>
680
681 <!-- Kinect -->
682 <!-- -->
683 <link name="pos_kinect_Link"/>

```

```

684 <joint name="pos_kinect_joint"
685   type="fixed">
686   <origin
687     xyz="0.01 -0.00100184102795236 0.36199"
688     rpy="0 0 0" />
689   <parent
690     link="kinect_plate_link" />
691   <child
692     link="pos_kinect_Link" />
693   <axis
694     xyz="0 0 0" />
695 </joint>
696 <!-- -->
697   <xacro:property name="cam_px" value="0" />
698   <xacro:property name="cam_py" value="0" />
699   <xacro:property name="cam_pz" value="0" />
700   <xacro:property name="cam_or" value="0" />
701   <xacro:property name="cam_op" value="0" />
702   <xacro:property name="cam_oy" value="0" />
703
704   <xacro:include filename="$(find eddiebot_description)/urdf/kinect.urdf.xacro" />
705   <xacro:sensor_kinect parent="pos_kinect_Link" cam_px="${cam_px}" cam_py="${cam_py}"
706     cam_pz="${cam_pz}" cam_or="${cam_or}" cam_op="${cam_op}" cam_oy="${cam_oy}"/>
707 </robot>

```

C.2. Kinect URDF

```

1 <?xml version="1.0"?>
2 <robot name="sensor_openni" xmlns:xacro="http://ros.org/wiki/xacro">
3
4   <!-- Parameterised in part by the values in turtlebot_properties.urdf.xacro -->
5   <xacro:macro name="sensor_kinect" params="parent cam_px cam_py cam_pz cam_or cam_op
6     cam_oy">
7     <joint name="camera_rgb_joint" type="fixed">
8       <origin xyz="${cam_px} ${cam_py} ${cam_pz}" rpy="${cam_or} ${cam_op} ${cam_oy}
9       <parent link="${parent}"/>
10      <child link="camera_rgb_frame" />
11    </joint>
12    <link name="camera_rgb_frame"/>
13
14    <joint name="camera_rgb_optical_joint" type="fixed">
15      <origin xyz="0 0 0" rpy="${-M_PI/2} 0 ${-M_PI/2}" />
16      <parent link="camera_rgb_frame" />
17      <child link="camera_rgb_optical_frame" />
18    </joint>
19    <link name="camera_rgb_optical_frame"/>
20
21    <joint name="camera_joint" type="fixed">
22      <origin xyz="-0.031 ${-cam_py} -0.016" rpy="0 0 0"/>
23      <parent link="camera_rgb_frame"/>
24      <child link="camera_link"/>
25    </joint>
26    <link name="camera_link">
27      <visual>
28        <origin xyz="0 0 0" rpy="0 0 ${M_PI/2}"/>

```

```

28     <geometry>
29       <mesh filename="package://eddiebot_description/meshes/kinect/kinect.dae"/>
30     </geometry>
31   </visual>
32   <collision>
33     <origin xyz="0.0 0.0 0.0" rpy="0 0 0"/>
34     <geometry>
35       <box size="0.07271 0.27794 0.073"/>
36     </geometry>
37   </collision>
38   <inertial>
39     <mass value="0.564" />
40     <origin xyz="0 0 0" />
41     <inertia ixx="0.003881243" ixy="0.0" ixz="0.0"
42             iyy="0.000498940" iyz="0.0"
43             izz="0.003879257" />
44   </inertial>
45 </link>
46
47   <!-- The fixed joints & links below are usually published by static_transformers
48   launched by the OpenNi launch
49   files. However, for Gazebo simulation we need them, so we add them here.
50   (Hence, don't publish them additionally!) -->
51 <joint name="camera_depth_joint" type="fixed">
52   <origin xyz="0 ${2 * -cam_py} 0" rpy="0 0 0" />
53   <parent link="camera_rgb_frame" />
54   <child link="camera_depth_frame" />
55 </joint>
56 <link name="camera_depth_frame"/>
57
58 <joint name="camera_depth_optical_joint" type="fixed">
59   <origin xyz="0 0 0" rpy="${-M_PI/2} 0 ${-M_PI/2}" />
60   <parent link="camera_depth_frame" />
61   <child link="camera_depth_optical_frame" />
62 </joint>
63 <link name="camera_depth_optical_frame"/>
64
65 </xacro:macro>
66 </robot>

```

C.3. Gazebo

```

1 <?xml version="1.0"?>
2 <robot name="eddiebot" xmlns:xacro="http://ros.org/wiki/xacro">
3
4   <xacro:property name="M_PI" value="3.14159265359"/>
5
6   <!-- Definición de argumentos para la visualización del láser e IMU -->
7   <xacro:arg name="laser_visual" default="false"/>
8   <xacro:arg name="imu_visual" default="false"/>
9
10  <!-- Solo se va a configurar el material -->
11
12  <gazebo reference="base_link">
13    <material>Gazebo/DarkGrey</material>
14  </gazebo>

```

```

15 <gazebo reference="base_top_link">
16   <material>Gazebo/DarkGrey</material>
17 </gazebo>
18
19 <gazebo reference="kinect_plate_link">
20   <material>Gazebo/Grey</material>
21 </gazebo>
22
23 <gazebo reference="motor_left_link">
24   <material>Gazebo/Orange</material>
25 </gazebo>
26
27 <gazebo reference="motor_right_link">
28   <material>Gazebo/Orange</material>
29 </gazebo>
30
31 <gazebo reference="battery_left_link">
32   <material>Gazebo/Black</material>
33 </gazebo>
34
35 <gazebo reference="battery_right_link">
36   <material>Gazebo/Black</material>
37 </gazebo>
38
39 <gazebo reference="screen_case_link">
40   <material>Gazebo/Red</material>
41 </gazebo>
42
43 <gazebo reference="screen_link">
44   <material>Gazebo/FlatBlack</material>
45 </gazebo>
46
47
48 <!-- ||||| -->
49
50 <!-- Configuración de las propiedades físicas y material para la rueda izquierda -->
51 <gazebo reference="wheel_left_link">
52   <mu1>0.9</mu1>
53   <mu2>0.9</mu2>
54   <kp>1000000.0</kp>
55   <kd>100.0</kd>
56   <fdirection value="1 0 0"/>
57   <minDepth>0.001</minDepth>
58   <maxVel>1.0</maxVel>
59   <material>Gazebo/FlatBlack</material>
60 </gazebo>
61
62 <!-- Configuración de las propiedades físicas y material para la rueda derecha -->
63 <gazebo reference="wheel_right_link">
64   <mu1>0.9</mu1>
65   <mu2>0.9</mu2>
66   <kp>1000000.0</kp>
67   <kd>100.0</kd>
68   <fdirection value="1 0 0"/>
69   <minDepth>0.001</minDepth>
70   <maxVel>1.0</maxVel>
71   <material>Gazebo/FlatBlack</material>
72 </gazebo>
73

```

```

74  <!-- Configuración de las propiedades físicas y material -->
75  <gazebo reference="base_front_caster_link">
76    <mu1 value="0.1"/>
77    <mu2 value="0.1"/>
78    <kp value="1000000.0"/>
79    <kd value="100.0"/>
80    <minDepth>0.001</minDepth>
81    <maxVel>1.0</maxVel>
82    <material>Gazebo/Orange</material>
83  </gazebo>
84
85  <!-- Configuración de las propiedades físicas y material -->
86  <gazebo reference="base_rear_caster_link">
87    <mu1 value="0.1"/>
88    <mu2 value="0.1"/>
89    <kp value="1000000.0"/>
90    <kd value="100.0"/>
91    <minDepth>0.001</minDepth>
92    <maxVel>1.0</maxVel>
93    <material>Gazebo/Orange</material>
94  </gazebo>
95
96  <!-- Configuración de las propiedades físicas y material para las ruedas delanteras -->
97  <gazebo reference="wheels_front_caster_link">
98    <mu1 value="0.1"/>
99    <mu2 value="0.1"/>
100    <kp value="1000000.0"/>
101    <kd value="100.0"/>
102    <minDepth>0.001</minDepth>
103    <maxVel>1.0</maxVel>
104    <material>Gazebo/FlatBlack</material>
105  </gazebo>
106
107
108  <!-- Configuración de las propiedades físicas y material para las ruedas traseras -->
109  <gazebo reference="wheels_rear_caster_link">
110    <mu1 value="0.1"/>
111    <mu2 value="0.1"/>
112    <kp value="1000000.0"/>
113    <kd value="100.0"/>
114    <minDepth>0.001</minDepth>
115    <maxVel>1.0</maxVel>
116    <material>Gazebo/FlatBlack</material>
117  </gazebo>
118
119  <!-- Configuración del sensor IMU -->
120  <gazebo reference="imu_link">
121    <sensor type="imu" name="imu">
122      <always_on>true</always_on>
123      <visualize>$(arg imu_visual)</visualize>
124    </sensor>
125    <material>Gazebo/FlatBlack</material>
126  </gazebo>
127
128  <!-- Configuración del plugin de control del robot diferencial -->
129  <gazebo>
130    <plugin name="eddiebot_controller" filename="libgazebo_ros_diff_drive.so">

```



```

131     <commandTopic>cmd_vel</commandTopic>
132     <odometryTopic>odom</odometryTopic>
133     <odometryFrame>odom</odometryFrame>
134     <odometrySource>world</odometrySource>
135     <publishOdomTF>true</publishOdomTF>
136     <robotBaseFrame>base_footprint</robotBaseFrame>
137     <publishWheelTF>false</publishWheelTF>
138     <publishTf>true</publishTf>
139     <publishWheelJointState>true</publishWheelJointState>
140     <legacyMode>false</legacyMode>
141     <updateRate>40</updateRate>
142     <leftJoint>wheel_left_joint</leftJoint>
143     <rightJoint>wheel_right_joint</rightJoint>
144     <wheelSeparation>0.44808</wheelSeparation>
145     <wheelDiameter>0.153</wheelDiameter>
146     <wheelAcceleration>0.5</wheelAcceleration>
147     <wheelTorque>1.25</wheelTorque>
148     <rosDebugLevel>na</rosDebugLevel>
149 </plugin>
150 </gazebo>
151
152 <!-- Configuración del plugin del sensor IMU -->
153 <gazebo>
154   <plugin name="imu_plugin" filename="libgazebo_ros_imu.so">
155     <alwaysOn>true</alwaysOn>
156     <bodyName>imu_link</bodyName>
157     <frameName>imu_link</frameName>
158     <topicName>imu</topicName>
159     <serviceName>imu_service</serviceName>
160     <gaussianNoise>0.0</gaussianNoise>
161     <updateRate>0</updateRate>
162     <imu>
163       <noise>
164         <type>gaussian</type>
165         <rate>
166           <mean>0.0</mean>
167           <stddev>2e-4</stddev>
168           <bias_mean>0.0000075</bias_mean>
169           <bias_stddev>0.0000008</bias_stddev>
170         </rate>
171         <accel>
172           <mean>0.0</mean>
173           <stddev>1.7e-2</stddev>
174           <bias_mean>0.1</bias_mean>
175           <bias_stddev>0.001</bias_stddev>
176         </accel>
177       </noise>
178     </imu>
179   </plugin>
180 </gazebo>
181
182 <!-- LIDAR -->
183 <gazebo reference="laser_frame">
184   <material>Gazebo/FlatBlack</material>
185
186   <sensor name="laser" type="ray">
187     <alwaysOn>true</alwaysOn>
188     <pose> 0 0 0 0 0 0 </pose>
189     <visualize>true</visualize>

```

```

190     <update_rate>10</update_rate>
191     <ray>
192         <scan>
193             <horizontal>
194                 <samples>360</samples>
195                 <min_angle>-3.14</min_angle>
196                 <max_angle>3.14</max_angle>
197             </horizontal>
198         </scan>
199         <range>
200             <min>0.3</min>
201             <max>12</max>
202         </range>
203     </ray>
204     <plugin name="laser_controller" filename="libgazebo_ros_laser.so">
205         <topicName>scan</topicName>
206         <frameName>laser_frame</frameName>
207     </plugin>
208 </sensor>
209 </gazebo>
210
211 <!-- Kinect sensor for simulation -->
212 <!--<xacro:sim_3dsensor_kinect/>-->
213 <gazebo reference="pos_kinect_Link">
214     <sensor type="depth" name="camera">
215         <always_on>true</always_on>
216         <update_rate>20.0</update_rate>
217         <camera>
218             <horizontal_fov>${60.0*M_PI/180.0}</horizontal_fov>
219             <image>
220                 <format>B8G8R8</format>
221                 <width>640</width>
222                 <height>480</height>
223             </image>
224             <clip>
225                 <near>0.05</near>
226                 <far>8.0</far>
227             </clip>
228         </camera>
229         <plugin name="kinect_camera_controller" filename="libgazebo_ros_openni_kinect.so
">
230             <cameraName>camera</cameraName>
231             <alwaysOn>true</alwaysOn>
232             <updateRate>0</updateRate>
233             <imageTopicName>rgb/image_raw</imageTopicName>
234             <depthImageTopicName>depth/image_raw</depthImageTopicName>
235             <pointCloudTopicName>depth/points</pointCloudTopicName>
236             <cameraInfoTopicName>rgb/camera_info</cameraInfoTopicName>
237             <depthImageCameraInfoTopicName>depth/camera_info</
depthImageCameraInfoTopicName>
238             <frameName>camera_depth_optical_frame</frameName>
239             <baseline>0.1</baseline>
240             <distortion_k1>0.0</distortion_k1>
241             <distortion_k2>0.0</distortion_k2>
242             <distortion_k3>0.0</distortion_k3>
243             <distortion_t1>0.0</distortion_t1>
244             <distortion_t2>0.0</distortion_t2>
245             <pointCloudCutoff>0.4</pointCloudCutoff>
246         </plugin>

```

```

247     </sensor>
248 </gazebo>
249
250 <!-- Configuración del plugin del sensor p3d -->
251 <gazebo>
252   <plugin name="p3d_plugin" filename="libgazebo_ros_p3d.so">
253     <bodyName>base_link</bodyName>
254     <frameName>world</frameName>
255     <topicName>pose_3d</topicName>
256     <gaussianNoise>0.0</gaussianNoise>
257     <updateRate>0</updateRate>
258   </plugin>
259 </gazebo>
260 </robot>

```

C.4. Gazebo launch

```

1 <launch>
2   <!-- Definición de argumentos -->
3   <arg name="model" default="$(find eddiebot_description)/urdf/eddiebot.urdf.xacro
4     "/>
5   <!-- Define la ubicación del archivo URDF del robot -->
6   <arg name="world" default="empty_world"/>
7   <!-- Define el mundo de Gazebo que se utilizará -->
8
9   <!-- Parámetro de descripción del robot -->
10  <param name="robot_description" command="$(find xacro)/xacro --inorder $(arg model
11    )"/>
12
13  <!-- Se define un parámetro que contendrá la descripción del robot. La descripción
14    se obtiene ejecutando
15    el comando xacro para procesar el archivo URDF especificado en el argumento
16    model -->
17
18  <!-- Inclusión de archivo de lanzamiento de Gazebo -->
19  <include file="$(find gazebo_ros)/launch/empty_world.launch">
20    <!-- Se incluye el archivo de lanzamiento empty_world.launch proporcionado por
21    el paquete gazebo_ros -->
22    <arg name="paused" value="false"/>
23    <arg name="use_sim_time" value="true"/>
24    <arg name="gui" value="true"/>
25    <arg name="headless" value="false"/>
26    <arg name="debug" value="false"/>
27    <arg name="world_name" value="$(arg world)"/>
28    <!-- Se establecen varios parámetros para configurar el entorno de Gazebo,
29    como pausado, uso de tiempo
30    simulado, interfaz gráfica, etc. El mundo especificado en el argumento
31    world se carga en este lanzamiento -->
32  </include>
33
34  <!-- Nodo de spawn del modelo -->
35  <node pkg="gazebo_ros" type="spawn_model" name="spawn_model"
36    args="-unpause -urdf -model robot -param robot_description"
37    output="screen" respawn="false"/>
38
39  <!-- Se lanza el nodo robot_state_publisher del paquete robot_state_publisher, que
40    publica el estado del robot

```

```

32     en el sistema de coordenadas del mundo -->
33     <node pkg="robot_state_publisher" type="robot_state_publisher" name="
robot_state_publisher">
34         <param name="publish_frequency" type="double" value="50.0" />
35     </node>
36 </launch>

```

C.5. RVIZ Launch

```

1 <launch>
2     <arg name="rviz_config_file" default="$(find eddiebot_description)/config/
robot_eddie.rviz"/>
3     <!-- Nodo para publicar el estado de las articulaciones del robot -->
4     <node pkg="joint_state_publisher_gui" type="joint_state_publisher_gui" name="
joint_state_publisher_gui">
5         <param name="use_gui" value="true"/>
6         <param name="rate" value="50"/>
7     </node>
8     <!-- Nodo para visualizar el modelo del robot en RViz -->
9     <node pkg="rviz" type="rviz" name="rviz" args="-d $(arg rviz_config_file)"/>
10 </launch>

```

C.6. Script Bash para Guardar Datos de Pose 3D y Odom en archivos .txt

```

1 #!/bin/bash
2 # Nombre del archivo: guardar_datos.sh
3
4 # Ruta de los archivos de salida en el escritorio
5 ODOM_OUTPUT_FILE="$HOME/Escritorio/scripts/odom_data.txt"
6 POSE_OUTPUT_FILE="$HOME/Escritorio/scripts/pose_3d_data.txt"
7
8 # Función para guardar los datos de un tópico
9 guardar_topico() {
10     local topico=$1
11     local archivo=$2
12
13     echo "Guardando datos de $topico en $archivo"
14     rostopic echo $topico > $archivo &
15 }
16
17 # Guardar datos de /odom
18 guardar_topico /odom $ODOM_OUTPUT_FILE
19
20 # Guardar datos de /pose_3d
21 guardar_topico /pose_3d $POSE_OUTPUT_FILE
22
23 # Esperar 15 segundos
24 echo "Grabando datos por 15 segundos..."
25 sleep 15
26
27 # Detener la grabación de datos
28 echo "Deteniendo la grabación de datos"

```

```
29 pkill -f "rostopic echo"  
30  
31 echo "Grabación completada. Los datos se han guardado en los archivos especificados en  
    el escritorio."
```


Apéndice D

Impresora Bambu Lab X1 Carbon

La impresora Bambulab X1 Carbon es una impresora 3D avanzada que ofrece capacidad para imprimir en varios materiales y colores. Destaca por su impresión de alta calidad con una resolución Lidar de $7\text{ }\mu\text{m}$, lo que garantiza detalles precisos y acabados finos en los modelos impresos. Incorpora un sistema CoreXY de alta velocidad con una aceleración notable de $20,000\text{ mm/s}^2$, lo que permite tiempos de impresión más rápidos y eficientes. Además, cuenta con doble nivelación automática de cama, lo que facilita la preparación y optimización del entorno de impresión para resultados consistentemente precisos [28]. (Ver Figura D.1).



Figura D.1: Bambu Lab X1 Carbon

Una vez se tiene el archivo en formato STL, STEP, etc. Se necesita convertir este objeto a un archivo G-Code, este código contiene instrucciones detalladas para la impresora 3D, indicando cómo se debe depositar el material capa por capa para construir el objeto. Para ello se ha utilizado el slicer Bambu Studio, el cual pertenece a BambuLab.

A continuación se detallan los pasos seguidos en el uso del slicer para preparar la tapa de la carcasa.

1. Abrir el Bambu Studio. Pantalla de inicio detallada en Figura D.2

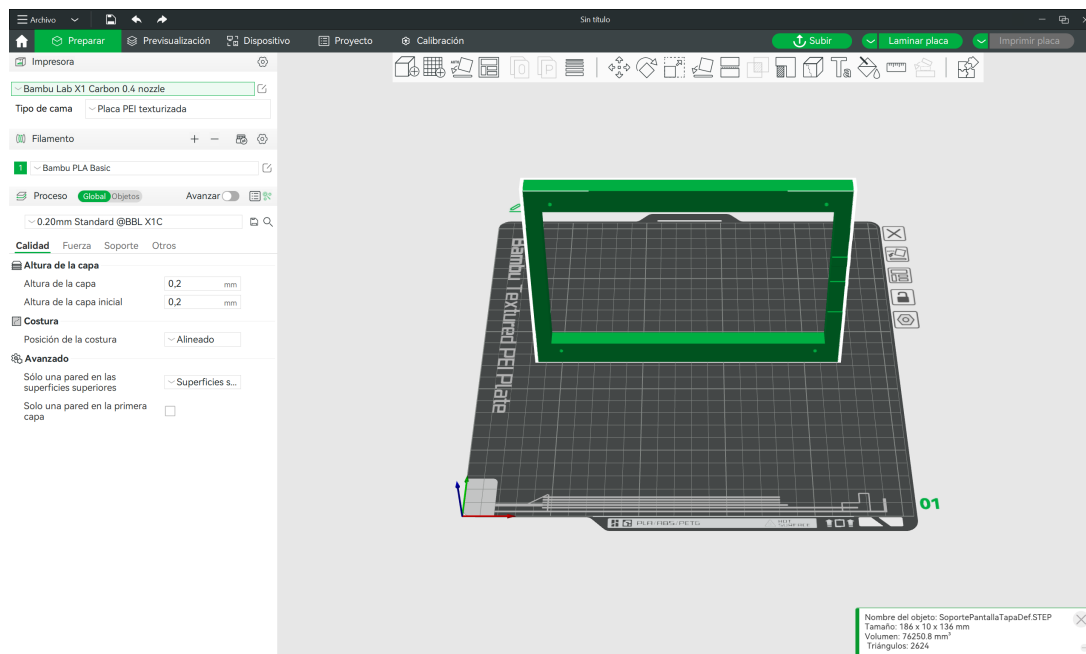


Figura D.2: Pantalla de inicio

2. Orientación de la pieza. Ver Figura D.3

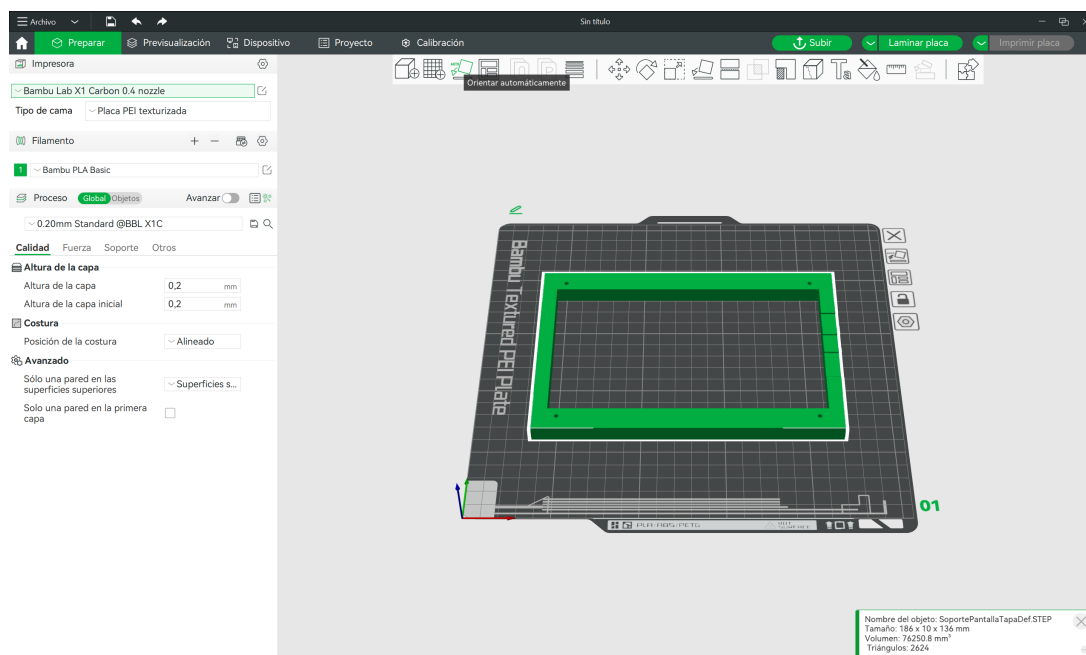


Figura D.3: Orientación de la pantalla

3. Ajuste de parámetros para impresión correcta. Ver Figura D.4.

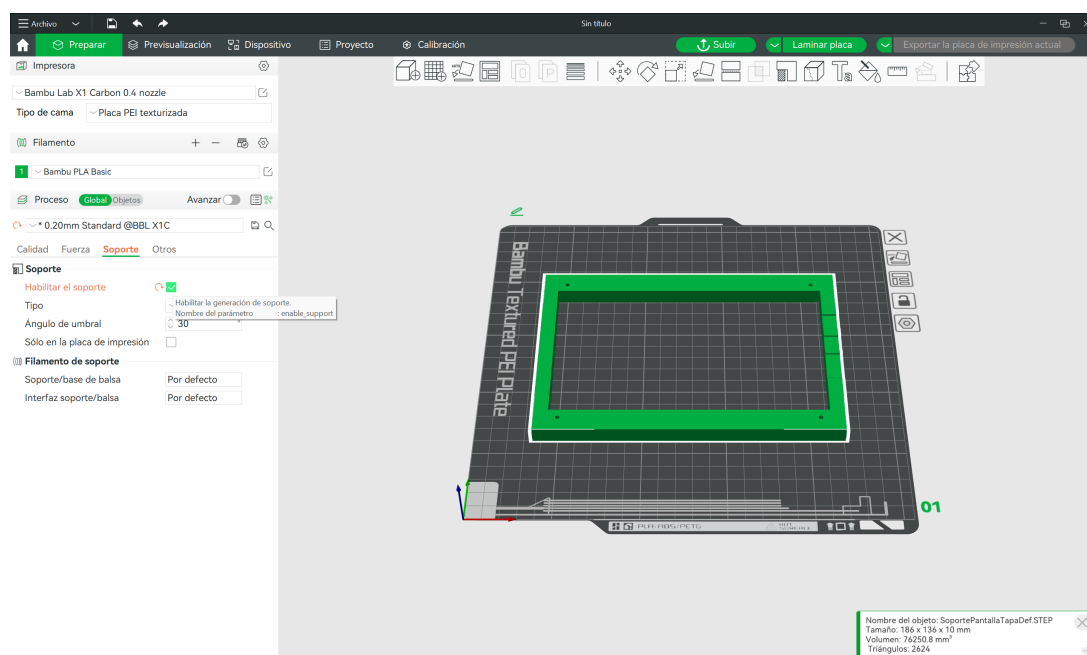


Figura D.4: Ajuste de parámetros

4. Exportación del archivo G-Code. Ver Figura D.5.

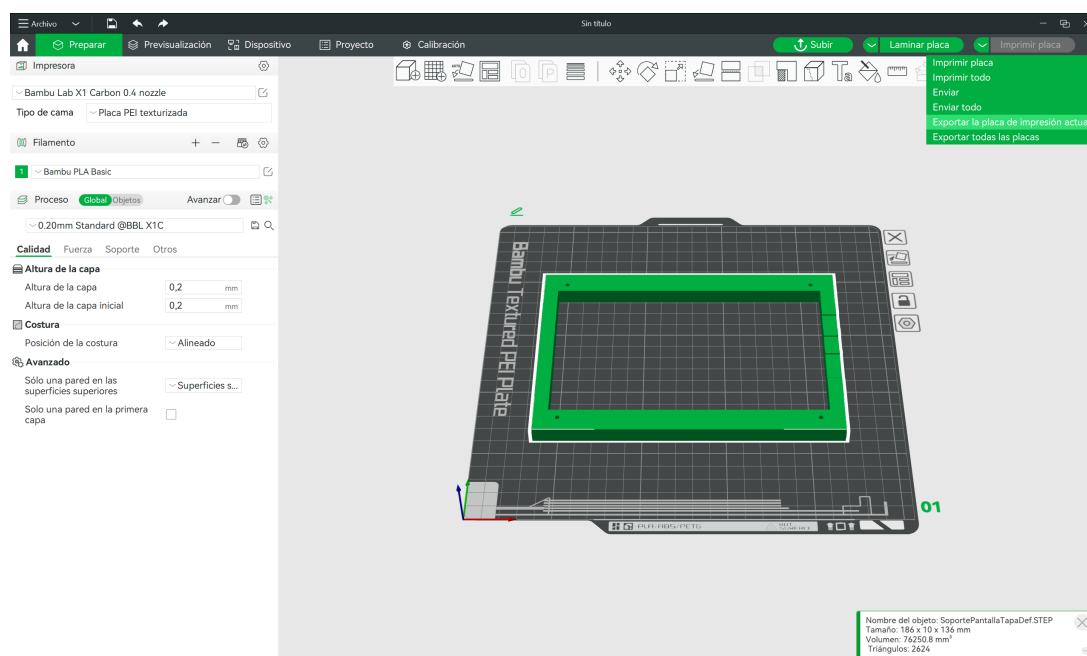


Figura D.5: Botón para exportar a G-Code

5. Comprobación de la correcta exportación del archivo G-Code en pantalla final. Ver Figura D.6

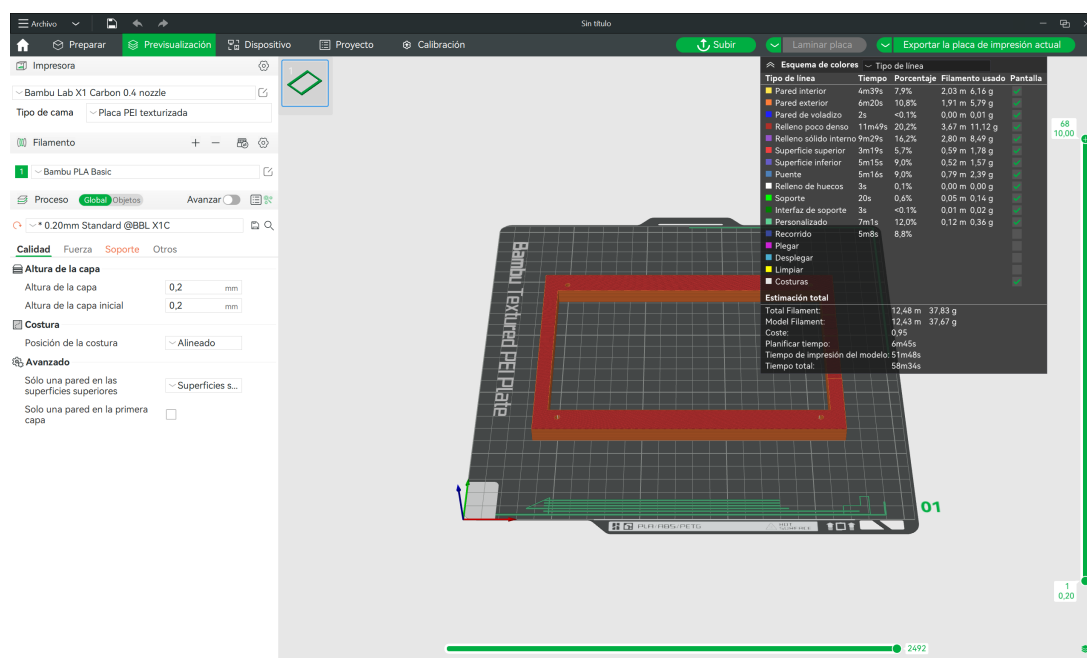


Figura D.6: Objeto exportado a G-Code

Una vez exportado el archivo G-Code, se procede a pasarlo a la impresora Bambu Lab X1, a través de una tarjeta SD. A continuación se detallan los pasos para la impresión.

1. Encender la impresora 3D. Ver en Figura D.7 la pantalla de inicio.



Figura D.7: Pantalla de inicio de la impresora

2. Desde el icono de la carpeta del menú lateral, seleccionar la tarjeta SD y el archivo a imprimir. Ver Figura D.8.

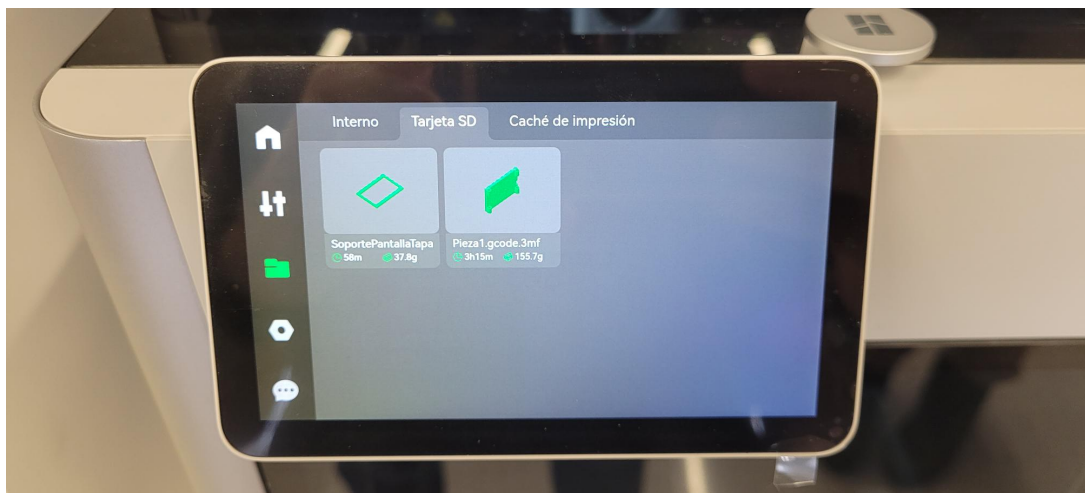


Figura D.8: Selección del archivo

3. Tras seleccionar el archivo aparece la pantalla detallada en la Figura D.9, en la cual se debe de pulsar en “Imprimir ahora”.



Figura D.9: Pantalla previa a la impresión

4. Antes de iniciar la impresión, se realizan una serie de pasos automáticamente para la correcta impresión de la pieza. Ver Figura D.10.

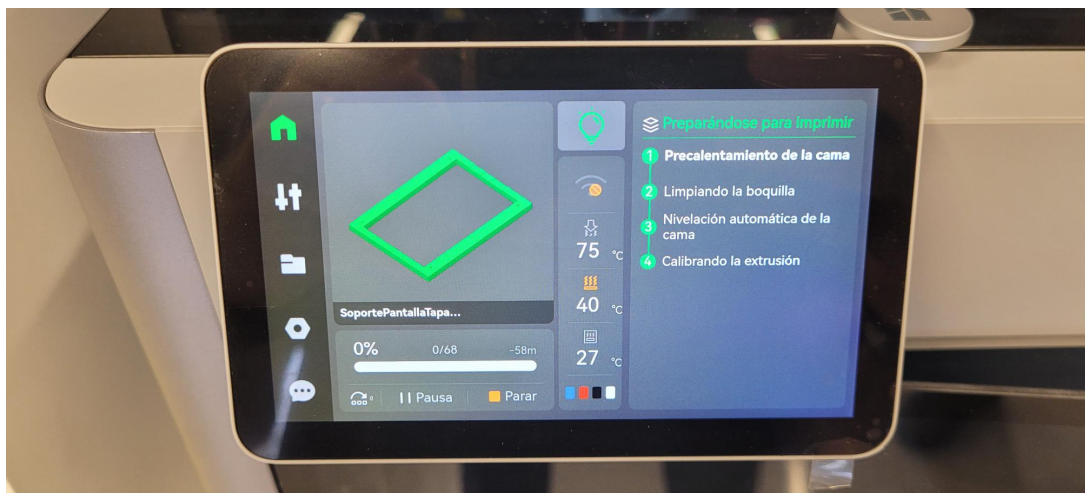


Figura D.10: Preparación de la impresión

En la Figura D.11 se muestra la impresora Bambu Lab X1 con una de las piezas, tras finalizar.



Figura D.11: Bambu Lab X1

Por último, en la Figura D.12 se muestra una de las piezas finales impresas.



Figura D.12: Ejemplo de pieza impresa con sus soportes

Bibliografía

- [1] R. Raj and A. Kos, “A comprehensive study of mobile robot: History, developments, applications, and future research perspectives,” *Applied Sciences*, vol. 12, no. 14, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/14/6951>
- [2] N. Ambrosetti, ““a great body of machinery”: From robots to mecha and back,” in *Explorations in the History and Heritage of Machines and Mechanisms*, M. Ceccarelli and R. López-García, Eds. Cham: Springer International Publishing, 2022, pp. 345–359.
- [3] P. Simionescu and D. Beale, “Optimum synthesis of the four-bar function generator in its symmetric embodiment: the ackermann steering linkage,” *Mechanism and Machine Theory*, vol. 37, no. 12, pp. 1487–1504, 2002. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0094114X0200071X>
- [4] S. Soni, T. Mistry, and J. Hanath, “Experimental analysis of mecanum wheel and omni wheel,” *International Journal of Innovative Science, Engineering & Technology*, vol. 1, no. 3, pp. 292–295, 2014.
- [5] A. Stefek, T. V. Pham, V. Krivanek, and K. L. Pham, “Energy comparison of controllers used for a differential drive wheeled mobile robot,” *IEEE Access*, vol. 8, pp. 170 915–170 927, 2020.
- [6] N. Melik and N. Slimane, “Autonomous navigation with obstacle avoidance of tricycle mobile robot based on fuzzy controller,” in *2015 4th International Conference on Electrical Engineering (ICEE)*, 2015, pp. 1–4.
- [7] K. Kozłowski and D. Pazderski, “Modeling and control of a 4-wheel skid-steering mobile robot,” *International journal of applied mathematics and computer science*, vol. 14, no. 4, pp. 477–496, 2004.
- [8] P. Iñigo-Blasco, F. D. del Rio, M. C. Romero-Ternero, D. Cagigas-Muñiz, and S. Vicente-Diaz, “Robotics software frameworks for multi-agent robotic systems development,” *Robotics and Autonomous Systems*, vol. 60, no. 6, pp. 803–821, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889012000322>
- [9] T. H. Collett, B. A. MacDonald, and B. P. Gerkey, “Player 2.0: Toward a practical robot programming framework,” in *Proceedings of the Australasian conference on robotics and automation (ACRA 2005)*, vol. 145. Citeseer Citeseer, 2005.
- [10] M. Montemerlo, N. Roy, and S. Thrun, “Perspectives on standardization in mobile robot programming: the carnegie mellon navigation (carmen) toolkit,” in *Proceedings*

- 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, vol. 3, 2003, pp. 2436–2441 vol.3.
- [11] A. Brooks, T. Kaupp, A. Makarenko, S. Williams, and A. Orebäck, *Orca: A Component Model and Repository*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 231–251. [Online]. Available: https://doi.org/10.1007/978-3-540-68951-5_13
 - [12] G. Metta, P. Fitzpatrick, and L. Natale, “Yarp: yet another robot platform,” *International Journal of Advanced Robotic Systems*, vol. 3, no. 1, p. 8, 2006.
 - [13] H. Bruyninckx, “Open robot control software: the orocos project,” in *Proceedings 2001 ICRA. IEEE international conference on robotics and automation (Cat. No. 01CH37164)*, vol. 3. IEEE, 2001, pp. 2523–2528.
 - [14] “Gazebo,” <https://gazebo.org/about>.
 - [15] “CoppeliaSim,” <https://coppeliarobotics.com>.
 - [16] “Webots,” <https://cyberbotics.com>.
 - [17] “RoboDK,” <https://robodk.com/es/>.
 - [18] “Unity,” <https://unity.com/es/solutions/automotive-transportation-manufacturing/robotics>.
 - [19] “SolidWorks,” <https://www.solidworks.com/es>.
 - [20] “Fusion360,” <https://www.autodesk.es/products/fusion-360/>.
 - [21] “FreeCAD,” <https://www.freecad.org/index.php>.
 - [22] “Blender,” <https://www.blender.org>.
 - [23] “Cinema4D,” <https://www.maxon.net/es/cinema-4d>.
 - [24] “Maya,” <https://www.autodesk.es/products/maya/overview?term=1-YEAR&tab=subscription>.
 - [25] “SolidWorks to URDF Exporter,” http://wiki.ros.org/sw_urdf_exporter.
 - [26] “GKjohnson (URDF viewer),” <https://gkjohnson.github.io/urdf-loaders/javascript/example/bundle/index.html>.
 - [27] “ROS - Robotic Operating System,” <https://www.ros.org/>.
 - [28] “Bambu Lab X1 Carbon,” <https://eu.store.bambulab.com/es-es/products/x1-carbon-3d-printer>.